

Delta Repository Access API

Table of Contents

1. Conventions used in this document	6
2. Introduction	6
2.1. Out of scope	7
2.2. Command overview	7
3. Description	8
3.1. Repository characteristics	8
3.1.1. Contents	9
3.1.2. Conflict resolution	9
3.2. Client characteristics	9
3.3. Channel characteristics	10
3.4. Participation characteristics	10
3.5. Event sequence numbers	10
3.6. Subscription characteristics	10
3.6.1. Composite events	11
3.7. Message characteristics	11
3.7.1. Message correlation	12
3.7.2. Unsuccessful commands	12
3.7.3. Splittable messages	12
3.7.4. Additional Info	14
3.8. Characteristics of manipulation of a list	14
3.8.1. Moving a node (without replacing)	15
3.8.2. Moving a node and replacing another node	17
4. Scenarios	19
4.1. Virgin repository, first time client connects	19
4.2. Client reconnects to known partition	20
4.3. Client changes known node with remote update	21
4.4. Client changes known node with local update	22
4.5. Client tries to change invalid node id	23
4.6. Client tries to change unknown node with failed event	24
4.7. Client with remote update changes property to same value as it already has	25
4.8. Repository has bug, needs to tell client it couldn't apply its commands	26
4.9. Concurrent changes to known node with local update	27
4.10. Concurrent changes to known node with remote update	29
4.11. Split query response	29
5. Messages	31
5.1. Parameter syntax	31

5.2. Parameter types	31
5.2.1. New nodes	33
5.2.2. Existing nodes	33
5.2.3. Reused nodes	33
5.2.4. How to address references	34
5.3. Message groups	34
5.3.1. Create node messages	34
5.3.2. Remove node messages	34
5.3.3. Add value messages	35
5.3.4. Add node messages	35
5.3.5. Change value messages	35
5.3.6. Replace node messages	35
5.3.7. Delete value messages	35
5.3.8. Delete node messages	35
5.3.9. Move or replace messages	36
5.3.10. Move-only messages	36
5.3.11. Move and replace messages	36
5.3.12. Move or replace child messages	37
5.3.13. Move or replace annotation messages	37
5.3.14. Move or replace from other parent	37
5.3.15. Move or replace in same parent	38
5.3.16. Move or replace from other containment	38
5.3.17. Move or replace in same containment	38
5.4. Errors	38
5.4.1. Error additional info types	39
5.4.2. Generic errors	39
5.4.2.1. Invalid participation	39
5.4.2.2. Message syntax incorrect	39
5.4.2.3. Message kind unknown	39
5.4.2.4. Missing required message parameter	40
5.4.2.5. Unknown node	40
5.4.2.6. The given parent of a node is not its parent	40
5.4.2.7. Index out of range	40
5.4.3. Chunk errors	41
5.4.3.1. Invalid anchor node	41
5.4.3.2. Duplicate node id	41
5.4.3.3. Parent / containment mismatch	41
5.4.3.4. Circular containment	42
5.4.4. Create node errors	42
5.4.4.1. Node already exists	42
5.4.5. Move errors	42

5.4.5.1. Old and new parent identical	42
5.4.5.2. Old and new containment identical	42
5.4.5.3. Old and new index identical	43
5.4.5.4. Index offset is invalid	43
5.4.6. Annotation errors	43
5.4.6.1. Annotation at index is not the given annotation	43
5.4.7. Containment errors	44
5.4.7.1. Unknown containment	44
5.4.7.2. Child at index is not the given node	44
5.4.8. Reference errors	44
5.4.8.1. Unknown reference	44
5.4.8.2. Reference already set	44
5.4.8.3. Reference not set	45
5.4.8.4. Reference at index is not the given reference	45
5.4.8.5. Undefined reference target	46
5.4.9. Property errors	46
5.4.9.1. Unknown property	46
5.4.9.2. Property already set	46
5.4.9.3. Property not set	46
5.4.10. Subscription errors	47
5.4.10.1. Mixed Inform about / Subscribe to Changing Partitions	47
5.4.10.2. Already subscribed to partition	47
5.4.10.3. Not subscribed to partition	47
5.4.10.4. Invalid depth limit	47
5.4.10.5. Unsupported protocol version	48
5.4.10.6. Unknown repository	48
5.4.11. Splittable message errors	48
5.4.11.1. No active split message	48
5.4.11.2. Duplicate continued chunk	48
5.4.11.3. Unknown splittable message in continued chunk	48
5.4.11.4. Invalid splittable message in continued chunk	49
5.4.11.5. Interrupted continued chunk	49
5.4.11.6. Composite command contains splittable	49
5.5. Custom messages	49
5.5.1. Custom queries	50
5.5.2. Custom commands	50
5.5.3. Custom events	50
5.6. Queries	51
5.6.1. Special responses	51
5.6.1.1. Error response	51
5.6.1.2. Continued query response	51

5.6.2. Subscription	52
5.6.2.1. Subscribe to partition changes	52
5.6.2.2. Inform about partition changes	53
5.6.2.3. Subscribe to partition contents	53
5.6.2.4. Unsubscribe from partition contents	54
5.6.3. Participation	55
5.6.3.1. Sign On (start a participation)	55
5.6.3.2. Sign Off (end a participation)	55
5.6.3.3. Reconnect (resume an existing participation)	56
5.6.4. Miscellaneous	57
5.6.4.1. Get available ids	57
5.6.4.2. List partitions	57
5.6.4.3. List and subscribe partitions	58
5.7. Command — event pairs overview	58
5.7.1. Partitions	60
5.7.2. Nodes	60
5.7.3. Properties	60
5.7.4. Children	61
5.7.5. Annotations	64
5.7.6. References	67
5.8. Commands	68
5.8.1. Continued Command	68
5.8.2. Partitions	69
5.8.2.1. Add partition	69
5.8.2.2. Delete partition	70
5.8.3. Nodes	71
5.8.3.1. Change classifier	71
5.8.4. Properties	71
5.8.4.1. Add property	71
5.8.4.2. Delete property	72
5.8.4.3. Change property	73
5.8.5. Children	73
5.8.5.1. Add child	73
5.8.5.2. Delete child	75
5.8.5.3. Replace child	76
5.8.5.4. Move child from containment in other parent	77
5.8.5.5. Move child from other containment in same parent	79
5.8.5.6. Move child in same containment in same parent	81
5.8.5.7. Move child from containment in other parent and replace existing child	82
5.8.5.8. Move child from other containment in same parent and replace existing child	84
5.8.5.9. Move child in same containment in same parent and replace existing child	86

5.8.6. Annotations	87
5.8.6.1. Add annotation	87
5.8.6.2. Delete annotation	89
5.8.6.3. Replace annotation	89
5.8.6.4. Move annotation from other parent	91
5.8.6.5. Move annotation in same parent	92
5.8.6.6. Move annotation from other parent and replace existing annotation	93
5.8.6.7. Move annotation in same parent and replace existing annotation	95
5.8.7. References	96
5.8.7.1. Add reference	96
5.8.7.2. Delete reference	97
5.8.7.3. Change reference	98
5.8.8. Composite	100
5.8.8.1. CompositeCommand	100
5.9. Events	101
5.9.1. Continued Event	101
5.9.2. Partitions	102
5.9.2.1. Partition added	102
5.9.2.2. Partition deleted	102
5.9.3. Nodes	103
5.9.3.1. Classifier changed	103
5.9.4. Properties	103
5.9.4.1. Property added	103
5.9.4.2. Property deleted	104
5.9.4.3. Property changed	104
5.9.5. Children	105
5.9.5.1. Child added	105
5.9.5.2. Child deleted	106
5.9.5.3. Child replaced	106
5.9.5.4. Child moved from containment in other parent	107
5.9.5.5. Child moved from other containment in same parent	108
5.9.5.6. Child moved in same containment in same parent	108
5.9.5.7. Child moved from containment in other parent and replaced existing child	109
5.9.5.8. Child moved from other containment in same parent and replaced existing child	110
5.9.5.9. Child moved in same containment in same parent and replaced existing child	111
5.9.6. Annotations	112
5.9.6.1. Annotation added	112
5.9.6.2. Annotation deleted	112
5.9.6.3. Annotation replaced	113
5.9.6.4. Annotation moved from other parent	114
5.9.6.5. Annotation moved in same parent	115

5.9.6.6. Annotation moved from other parent and replaced existing annotation	115
5.9.6.7. Annotation moved in same parent and replaced existing annotation	116
5.9.7. References	117
5.9.7.1. Reference added	117
5.9.7.2. Reference deleted	117
5.9.7.3. Reference changed	118
5.9.8. Miscellaneous	119
5.9.8.1. CompositeEvent	119
5.9.8.2. No-op Event	120
5.9.8.3. ErrorEvent	120
Appendix A: Delta JSON Schema	121

The delta API **version 2026.1** is used to store and retrieve changes to nodes. It is intended for fine-grained changes to single values. it is **not** intended as a bulk API that updates large chunks of the model at once.

1. Conventions used in this document

- ALL-CAPS key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in [BCP14 \(RFC2119, RFC8174\)](#) when, and only when, they appear in all capitals, as shown here.
- Footnotes refer to more discussions and rationale, but are non-normative.

2. Introduction

Delta API assumes a bidirectional communication channel between a [client](#) and a [repository](#). The client initiates the channel with the repository to open a [participation](#)^[1]. Both parties communicate through the established channel via *messages*.

Delta API supports these message categories, loosely following CQRS pattern^{[2][3]}:

- [Queries](#) initiated/requested by the client, with synchronous response by the repository. A query requests some information from the repository without changing the repository's contents.

The repository gathers all information needed to answer the query, and sends the information back. The repository might reply to invalid queries with a failure message. We also use queries for managing [participations](#).^[4]

- [Commands](#) sent by the client, with no immediate response by the repository. A command requests some change to the repository.

The repository processes the command asynchronously, and eventually broadcasts the effect(s) as *event(s)*.

- **Events** initiated/broadcast by the repository, without response by the client. An event describes some change that has been applied to the repository's contents. Events correlate to their originating command(s)^[5]; this command might have been sent by this client or another client.^[6]

All clients receive the same *successful* events (given the same subscription scope and disregarding **event sequence numbers**). Only the sending client receives events originating from **unsuccessful commands**.

A client describes the interesting part of the repository's contents by specifying its **subscription scope**. The client receives only events on nodes within that scope.

A typical workflow for a client would be:

1. Client **connects** to repository and starts a participation.
2. Client **subscribes** to some partitions with subscription queries. The response contains a **serialization chunk** with the current contents of these partitions.
3. Client initiates its editor or processing with the chunk.
4. Client **sends commands** to request changes to some nodes in the subscribed partitions.
5. Client **receives events** with the actual changes that have been applied to the subscribed nodes.
6. Client updates its editor or processing with the event's contents.
7. Client receives events originating from other clients' commands.
8. Client updates its editor or processing with the event's contents.
9. Client **ends** the participation.

2.1. Out of scope

This specification precisely describes the semantic content of each message. We don't describe the binding to any specific protocol (e.g. JSON over WebSockets, Protobuf over HTTP, message queues).

2.2. Command overview

We specified the semantics of each command and event precisely, leading to lots of similar, but different commands/events.^[7]

This table provides an overview of actions (rows) and element they apply to (columns).

Action	Child	Annotation	Reference	Property	Partition / Classifier
Add	Add child	Add annotation	Add reference ^{[8][9]}	Add property	Add partition

Action	Child	Annotation	Reference	Property	Partition / Classifier
Delete	Delete child	Delete annotation	Delete reference	Delete property	Delete partition
Change	Replace child	Replace annotation	Change reference	Change property	Change classifier
Move from other node	Move child from containment in other parent	Move annotation from other parent			
Move from other feature in same node	Move child from other containment in same parent				
Move in same feature in same node	Move child in same containment in same parent	Move annotation in same parent			
Move from other node and replace existing ^[10]	Move child from containment in other parent and replace existing child	Move annotation from other parent and replace existing annotation			
Move from other feature in same node and replace existing ^[10]	Move child from other containment in same parent and replace existing child				
Move in same feature in same node and replace existing ^[10]	Move child in same containment in same parent and replace existing child	Move annotation in same parent and replace existing annotation			
Composite	CompositeCommand				

3. Description

3.1. Repository characteristics

A repository is a set of partitions, and a set of APIs, amongst them this delta API. The repository has an ID that must be unique for all repositories in the same context. LionWeb doesn't specify the scope of the context.^[11]

A repository is “its own universe”: nodes within it can’t reference nodes, or be referenced from nodes from other repositories. All nodes within the same repository **MUST** have unique IDs. Nodes can refer to nodes from the LionCore and built-in languages.

3.1.1. Contents

The repository is always the ultimate arbiter of its contents.^[12] It can decide to apply each command verbatim, with some tweaks (e.g. because of [concurrent changes](#)), or not at all (e.g. because the command would introduce containment loops).

3.1.2. Conflict resolution

Each repository might introduce its own strategy how to handle commands.^[3] For example, assume client 1 sends a `DeleteChild` command to delete node `X`, and then client 2 sends a `SetProperty` command on node `X`. The repository has to *resolve that conflict* somehow.

- Repository A applies all commands in order, thus deletes node `X` and ignores the second command, i.e. does not set any property, but attaches both commands' `commandId` to the resulting `ChildDeleted` event.
- Repository B keeps an internal "trash can" of all deleted nodes. It deletes node `X` on the first command, but resurrects node `X` on the second command and changes the property as requested. It first emits a `ChildDeleted` event (correlated to the first command), but later `ChildAdded` and `PropertyChanged` events (both correlated to the second command).
- Repository C batches command processing. It looks at both messages together, and decides it's more important to keep the new property value around. It emits one `PropertyChanged` event, correlated with both commands.

Conflict resolution might depend on which commands a repository considers to arrive together, i.e. at the same time. A repository is free to chose its own interpretation of "at the same time". Examples include "never, every command stands on its own", "sliding window of 10 ms", or "within one processing cycle".

3.2. Client characteristics

A client is a program connecting to a [repository](#). Each client has exactly one [participation](#) with one repository at one point in time. The same client can have several consecutive participations with the same repository, but never at the same time.

A client might interact with several repositories, but they are completely independent (i.e. same as two clients connecting to two repositories).^[1]

A client stays up-to-date with the repository's contents if it applies all events in order.^[13] The order is defined by the event's sequence number.^[14]

A client can either decide to apply its own change immediately and filter out events based on that command, or wait for the correlated event to arrive and apply the change only then.

The client has the responsibility to guarantee uniqueness of [query ids](#) and [command ids](#) within the same participation.

Authentication and authorization are out of scope of this API.

3.3. Channel characteristics

Logically, the channel has independent parts for each [category](#) of message; in practice, we might transmit some or all of them through a shared channel.

This protocol is agnostic of its underlying transport or technical implementation. By default, this protocol uses JSON encoding over a text-based, single channel, stateful, bidirectional, asynchronous, reliable, order-preserving, and trustworthy transport.^[15] Any transport that fits these criteria (e.g. WebSocket^[16]) MUST use the same encoding and MUST adhere to the [Delta JSON Schema](#).^[17]

We assume the underlying protocol handles quality of service concerns (i.e. guaranteed in-order^[18] one-time complete delivery of messages), they are out of scope of this API. Outside of [Conflict resolution](#), a repository SHOULD apply commands in the received order.^[19]

3.4. Participation characteristics

Each participation has a *participation id*, unique for all concurrent participations. The repository assigns the participation id upon starting a new participation. The participation id MUST follow the same format as [identifier](#). It serves as a technical identifier.

A participation is transparent for reconnecting network connections, i.e. one participation can survive potential technical reconnects. A participation spans over time: if we would e.g. use REST, a participation spans over several REST requests/responses.

A participation [starts](#) by the client connecting to the repository via delta protocol. A participation ends either by the [client ending](#) it, or if the client disconnects involuntarily (e.g. by crashing or a connection failure).

A client can try to [reconnect](#) a participation if the client still knows a previously valid participation id, and the last [event sequence number](#) the client received.^[20]

3.5. Event sequence numbers

Each event contains a sequence number.^[14] This number is a positive integer that increments by one for each consecutive event, starting at 1.^[21] It is unique per participation.^[22] Events MUST be sent across the [channel](#) in increasing order of sequence number.^[18]

A [client](#) MUST use this number to order incoming events^[23], and SHOULD check whether the client missed any event.

3.6. Subscription characteristics

The subscription, or subscription scope, is the set of partitions a [participation](#) gets events for. A subscription does not have an identity on its own; it is bound to one participation.

Scope is defined by the node id of a [partition](#); it includes all changes to the partition and all its descendants (including annotation instances).^[24]

At the start of a participation, the subscription scope is empty. A client can change its subscription scope by

- [List and subscribe partitions](#) for requesting and subscribing to all existing partitions,
- [Inform about partition changes](#) for being informed about newly created partitions,
- [Subscribe to partition changes](#) for subscribing to newly created partitions,
- [Subscribe to partition contents](#) for subscribing to / [Unsubscribe from partition contents](#) for unsubscribing from a specific partition, or
- [Add partition](#) for adding a new partition.

A client can change the subscription scope at any time during a participation. However, a client MUST NOT mix [Inform about partition changes](#) and [Subscribe to partition changes](#) queries. If any [Inform about partition changes](#) query is active, the same client MUST NOT issue a [Subscribe to partition changes](#) query, and vice versa. Once all [Inform about partition changes](#) are disabled (i.e. both [creation](#) and [deletion](#) are [false](#)), the client MAY issue a [Subscribe to partition changes](#) query.^[25]

A client CAN send commands for any node id, independent of the client's subscription. Assume a client sends a command for a node that's part of a partition that client is not subscribed to. The repository treats that command the same as from a client that was subscribed to the containing partition, i.e. applies it in accordance with the repository's policy. The repository sends out the same events to subscribed clients, and *also* to the sending client.^[26]

This means a client:

- *always* gets the events for its own commands, event if it relates to a node outside the client's subscription scope.
- *can* get an event it didn't subscribe to, if the event originates in the client's own command.

3.6.1. Composite events

For a [composite event](#), arising from a [composite command](#), the following MUST hold:

- Every client gets a composite event with only the events it's subscribed to.
- No client gets an empty composite event.
- The client that sent the composite command always gets the complete composite event, modulo repo changes, e.g. conflict resolutions.

3.7. Message characteristics

Atomicity

Most messages are atomic; they cannot be split. The exceptions are [splittable messages](#).

Message identity

Each message has some id unique to the [participation](#) and [category](#):

- Queries have a [QueryId](#) (for [Continued query response](#) together with its [ContinuedChunk SequenceNumber](#)).
- Commands have a [CommandId](#) (for [Continued Command](#) together with its [ContinuedChunk SequenceNumber](#)).^[27]
- Events have [EventSequenceNumber](#) (for [Continued Event](#) together with its [ContinuedChunk SequenceNumber](#)).^{[28][29]}

Technical name

Every defined message has a unique *technical name*. The technical name is a valid programming language identifier as described for [IKeyed](#). Each binding to a specific protocol might use the technical name in a different way. For example, a JSON binding might have a member containing the technical name of the message, or a Protobuf binding might use the technical name as type name.

3.7.1. Message correlation

Each query uses its [QueryId](#) to correlate query *request* and *response*. Thus, query ids MUST be unique per participation.

Each event is correlated to one or more commands by its [CommandSource](#), consisting of the sending [ParticipationId](#) and the [CommandId](#) of the originating command.^[28] Thus, command ids MUST be unique per participation.^{[27][30]}

The repository assigns guaranteed unique [ParticipationIds](#).^[31]

An event can have more than one command source^[5], and it's possible that the originating command is *not* the event's twin. These situations mostly appear if the repository applied conflict resolution logic.

3.7.2. Unsuccessful commands

A command can be unsuccessful for the following reasons:

- The repository didn't understand the command, resulting in an [ErrorEvent](#).^[32]
- The client used the wrong command (e.g. [Add property](#) if the property has already been set, or [Move child from containment in other parent and replace existing child](#) if both the old and new parent are the same). This results in an [ErrorEvent](#).^[33]
- The command would not change anything in the repository, resulting in a [No-op Event](#).^[34]

The repository sends the event originating from an unsuccessful command *only* to the sending participation; it *does not* broadcast such events.^[35]

3.7.3. Splittable messages

All messages containing a [DeltaChunk](#).

A client or repository CAN decide to split up a *splittable message* if that message would exceed the acceptable message size. LionWeb does not define the acceptable message size, it SHOULD be a parameter for each implementation.

All *splittable messages* MAY contain a *SplitFlag* — typically named *split*. If the flag is unset or *false*, the splittable message is self-contained. If the flag is *true*, one or more *continued chunk* messages will follow.^[36]

Splittable messages include:

Queries

- *Subscribe to partition contents* response *contents*
- *List partitions* response *partitions*
- *List and subscribe partitions* response *partitions*

Commands

- *Add partition* *newPartition*
- *Add child* *newChild*
- *Replace child* *newChild*
- *Add annotation* *parent*
- *Replace annotation* *newAnnotation*

Events

- *Partition added* *newPartition*
- *Child added* *newChild*
- *Child replaced* *newChild*
- *Annotation added* *newAnnotation*
- *Annotation replaced* *newAnnotation*

Continued chunks

Continued chunks continue the chunk from the related *splittable message*. Each *continued chunk* refers to the *splittable message* it belongs to. The *continued chunk* contains a *DeltaChunk*, a *ContinuedChunkSequenceNumber*, and a *ContinuedChunkCompletedFlag* to signal whether it is the last *continued chunk*, or more will follow. If the flag is *true*, this is the last *continued chunk*. If the flag is *false*, one or more *continued chunk* messages will follow.^[36] A *DeltaChunk* from a continued chunk is to be “merged” into the chunk from the related *splittable message* simply by appending the contents of its top-level *nodes* field to the *nodes* field of the chunk of the related *splittable message*.^[37]

There MUST NOT be any message in the same *channel* between the *split message* and all related *continued chunks*. The continued chunks MUST be sent across the *channel* in increasing order of *ContinuedChunkSequenceNumber*.^[38]

A continued chunk MUST NOT appear as part of a *CompositeCommand* or *CompositeEvent*. (That also implies that the *SplitFlag* of a *splittable message* appearing inside a composite MUST either be unset or set to *false*.)^[39]

Continued chunks include:

- Continued query response
- Continued Command
- Continued Event

They can result in a [specific set of errors](#).

3.7.4. Additional Info

Each message has zero or more *additional infos* – i.e., these appear as an optionally empty list – consisting of:^{[40][41][42]}

- **kind** is an [identifier](#)-compatible string identifying the additional info type. Some kinds are pre-defined in this specification. A repository MAY reply with other, additional kinds.
- **distribute?** is an optional [boolean](#) flag with default value **false**. It is only valid in [Commands](#), i.e. MUST NOT be used for [Events](#) or [Queries](#). If present and set to **true**, the repository MUST copy the additional info to all events that result from the command, and distribute the additional info to all participants receiving the event.^[43]
- **message** is a human-readable string describing the additional info.
- **data** is a map with 0 or more arbitrary keys and values.^[44] All keys MUST be [identifier](#)-compatible and MUST be unique within the same additional info. All values MUST be strings. A *kind* might imply the presence of specific keys in **data**.

This additional info is symmetric to [additional infos in bulk API](#).

3.8. Characteristics of manipulation of a list

A number of commands and events deal with moving a node in one specific list^[45], and possibly replacing another node in the same list. Such a list will either consist of the child nodes of a [../metametamodel/metametamodel.pdf](#) on a **parent** node, or of the annotation nodes on a **parent** node. More precisely: from the viewpoint of the GPL that this functionality is implemented in, such a list consists of *pointers* to nodes (which happen to be objects). There's chance for confusion with regard to the indices before and after completion of the intended operation — more so than with commands and events that involve moving a node *between* lists.

There are two types of operations that involve manipulating a single list, and which benefit from clarification with regard to how they work position-wise:

1. Moving a node to a different position – the *target* index – in a list, moving other list nodes aside as necessary.
2. Moving a node to a different position – the *target* index – in a list, **and** replacing the list node at that target index. Note that for this operation, the moved node might not reside at that target index afterward, because of the replacing!

To avoid ambiguities in the description of such operations, and reduce the chance of misunderstanding the specification as much as possible, we chose to phrase such operations in

terms of an *index offset* — i.e., “in relative coordinates”.

For convenience, we first establish some notation/shorthand and definitions:

Integer ranges

We write $[lower..upper]$ for the set of integers consisting of: $lower$, $(lower + 1)$, $(lower + 2)$, ..., $upper$. The **integer range** $[lower..upper]$ includes $lower$ and $upper$ themselves.

Input data

For both scenarios above, assume we have a list of $n > 1$ distinct nodes. We also have an *old index* `oldIndex`, which MUST be an integer in the integer range $[0..(n - 1)]$: the position that the **node to move** appears at in the list. Finally, we have an *index offset* `indexOffset`, which MUST be an integer unequal to 0, and such that the *target index* (`oldIndex + indexOffset`) is in the integer range $[0..(n - 1)]$. `indexOffset` can be either positive or negative, meaning the movement will be towards higher positions (positive), or towards lower positions (negative).

3.8.1. Moving a node (without replacing)

This pertains to the following commands and events: [Move child in same containment in same parent](#), [Move annotation in same parent](#), [Child moved in same containment in same parent](#), [Annotation moved in same parent](#).

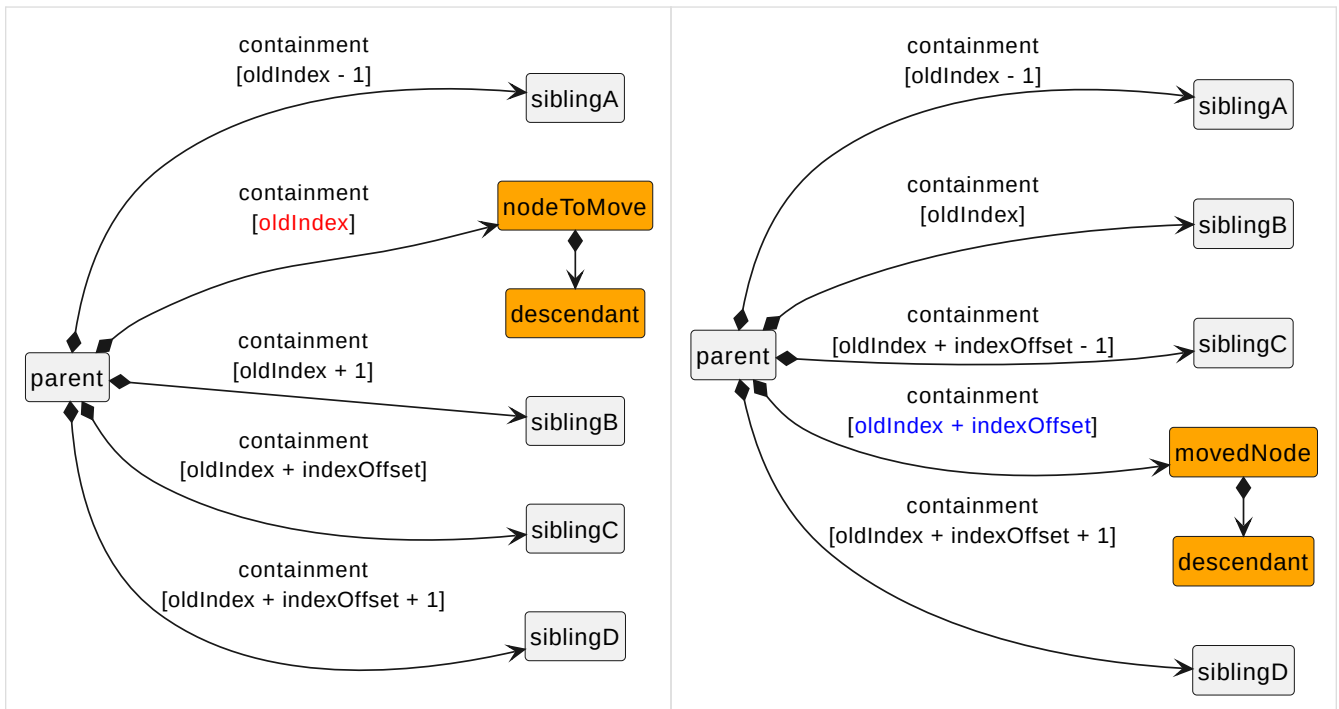
Set `newIndex = (oldIndex + indexOffset)`, which in this scenario is also the *target index*. The result of moving the node that’s initially at index `oldIndex` by `indexOffset` positions is that that node has been moved `indexOffset` positions up (if `indexOffset > 0`), or `(-indexOffset)` positions down (if `indexOffset < 0`). Afterward, the moved node resides at index `newIndex` in the list.

For clarity, we discuss two distinct “sub” scenarios depending on whether `indexOffset` is positive or negative.

Moving up

For `indexOffset > 0`:

Before	After
--------	-------

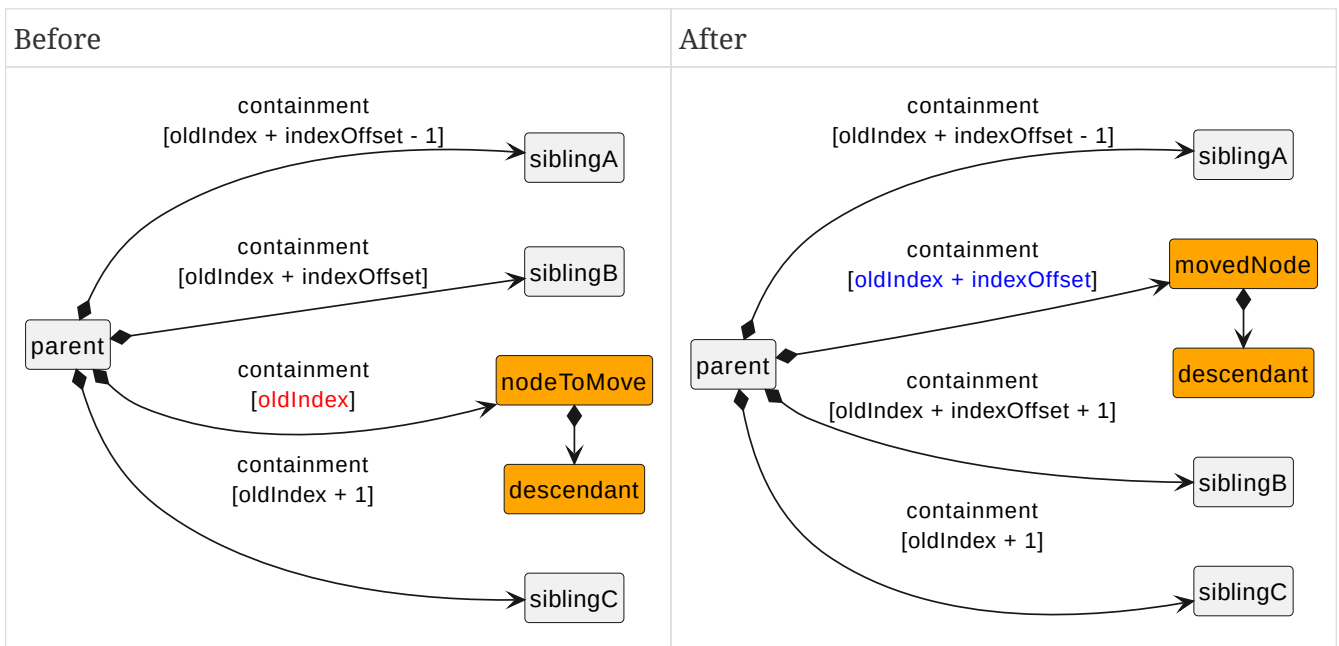


Example moving X with $indexOffset = 2$:

oldIndex=3: A[0] B[1] C[2] X[3] D[4] E[5] F[6] G[7] →
 newIndex=5: A[0] B[1] C[2] D[3] E[4] X[5] F[6] G[7]

Moving down

For $indexOffset < 0$:



Example moving X with $indexOffset = -2$:

oldIndex=3: A[0] B[1] C[2] X[3] D[4] E[5] F[6] G[7] →
 newIndex=1: A[0] X[1] B[2] C[3] D[4] E[5] F[6] G[7]

In both cases, only the nodes with initial indices in $[\min(oldIndex, newIndex)..max(oldIndex, newIndex)]$ are affected. If $indexOffset > 0$, then the nodes with initial indices in $[(oldIndex + 1)..newIndex]$ are moved to the next lower position. If $indexOffset < 0$, then the nodes with initial indices in $[newIndex..(oldIndex - 1)]$ are moved to the next higher position.

3.8.2. Moving a node and replacing another node

This pertains to the following commands and events: [Move child in same containment in same parent and replace existing child](#), [Move annotation in same parent and replace existing annotation](#), [Child moved in same containment in same parent and replaced existing child](#), [Annotation moved in same parent and replaced existing annotation](#).

We describe this scenario as a two-step process:

Set `targetIndex = (oldIndex + indexOffset)`. We don't use `newIndex` (yet): we want to reserve that name for the index that the moved node resides at after this operation, which can be different from `targetIndex`.

The node that appears at index `oldIndex` in the list is the node to move. The node that appears at index `targetIndex` in the list is the **node to replace**.

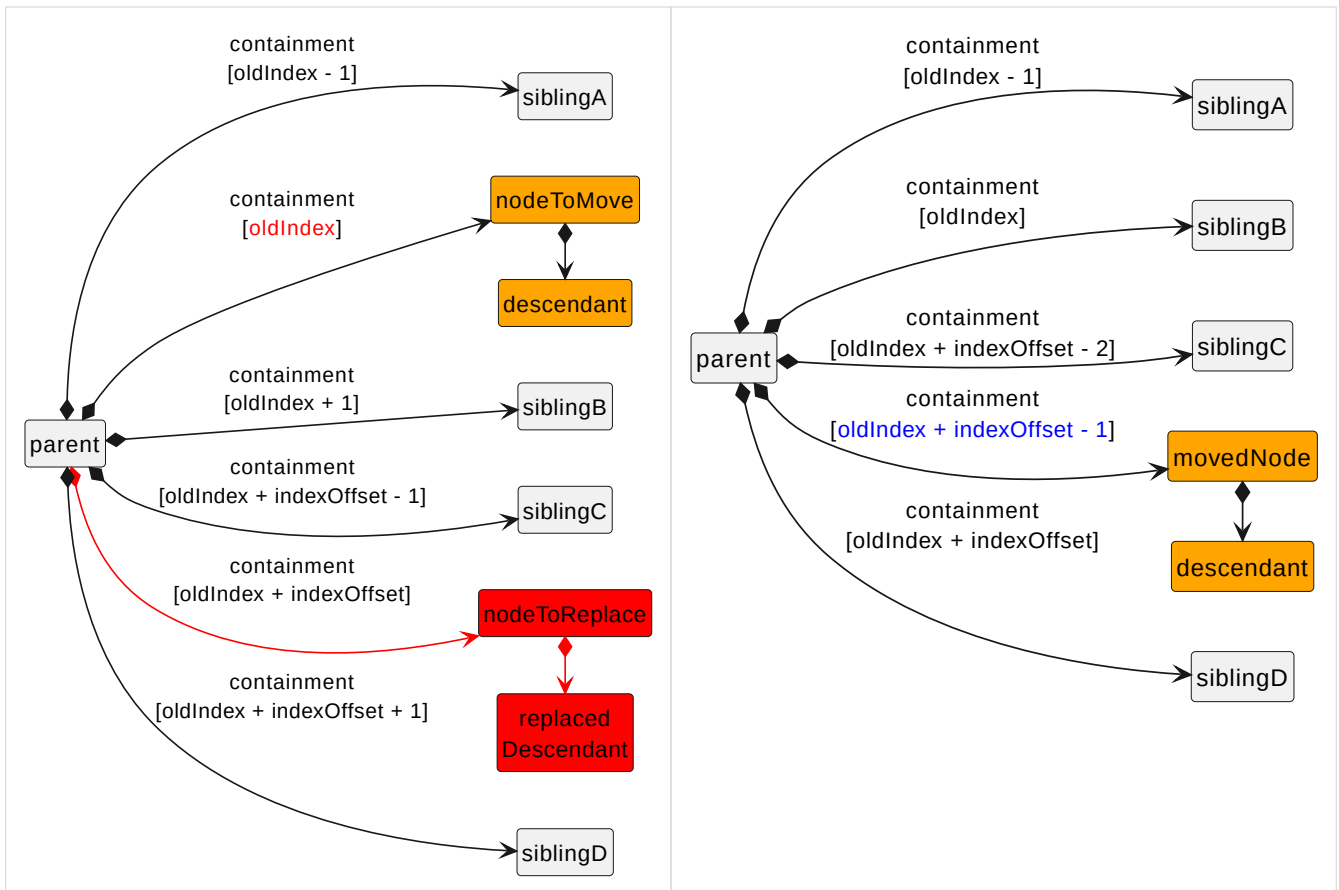
1. Set the node at index `targetIndex` to the node with initial index `oldIndex`. This action overwrites and effectively replaces the node to replace with the node to move. The node to replace gets deleted as a result. Now, the list contains the same number of nodes as before, just with two occurrences of the node to move, namely at indices `oldIndex` and `targetIndex`, and no occurrences of the node to replace.
2. The node with index `oldIndex` is removed from the list, with the nodes after it being moved to the next lower position — this includes the node moved in step 1 itself if (`indexOffset > 0`). Now, the list contains one less node than before.

For clarity, we discuss two distinct “sub” scenarios depending on whether `indexOffset` is positive or negative.

Moving up

For `indexOffset > 0`:

Before	After
--------	-------



The new index of the moved child can be calculated as follows: $\text{newIndex} = \text{oldIndex} + \text{indexOffset} - 1$.

Example moving **X** with $\text{indexOffset} = 3$, and replacing **F**:

$\text{oldIndex}=3$: A[0] B[1] C[2] X[3] D[4] E[5] F[6] G[7] →

$\text{indexOffset}=3$: A[0] B[1] C[2] D[3] E[4] X[5] G[6]

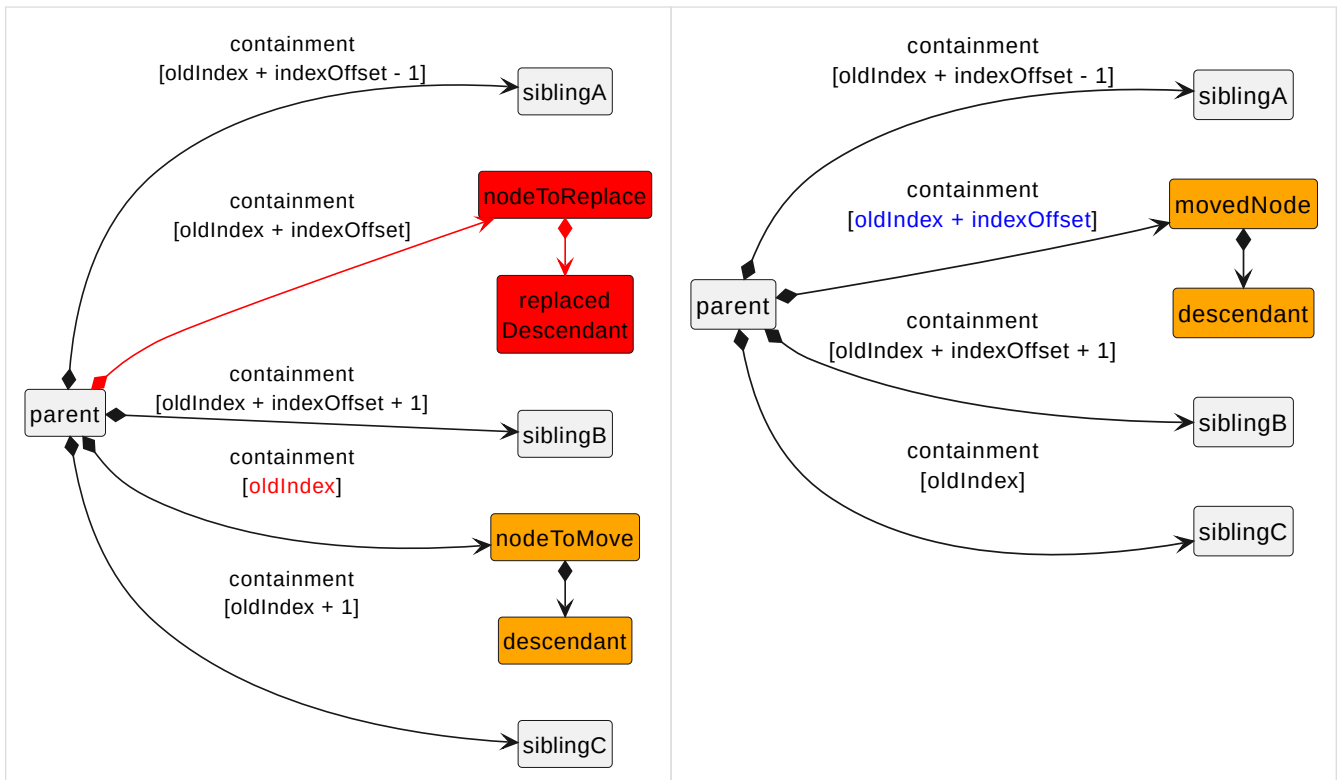
$\text{targetIndex}=6$, $\text{newIndex}=5$ and F is replaced with X

Note that moving and replacing another node with $\text{indexOffset}=1$ has the same effect as (just) deleting the node with index $(\text{oldIndex} + 1)$.

Moving down

For $\text{indexOffset} < 0$:

Before	After
--------	-------



The new index of the moved child can be calculated as follows: $\text{newIndex} = \text{oldIndex} + \text{indexOffset}$.

Example moving X with $\text{indexOffset} = -3$, and replacing B :

$\text{oldIndex}=4$: $A[0] \ B[1] \ C[2] \ D[3] \ X[4] \ E[5] \ F[6] \rightarrow$

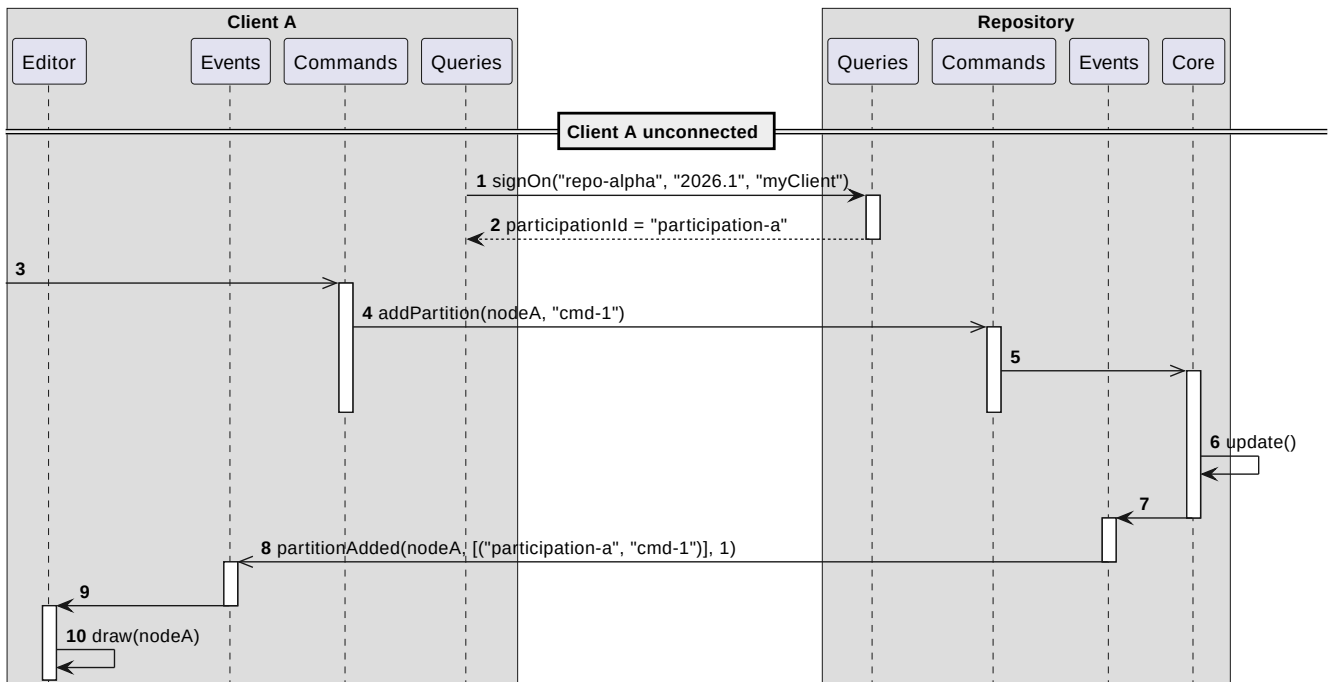
$\text{indexOffset}=-3$: $A[0] \ X[1] \ C[2] \ D[3] \ E[4] \ F[5]$

$\text{targetIndex}=1$, $\text{newIndex}=1$ and B is replaced with X

Note that moving and replacing another node with $\text{indexOffset}=-1$ has the same effect as (just) deleting the node with index ($\text{oldIndex} - 1$).

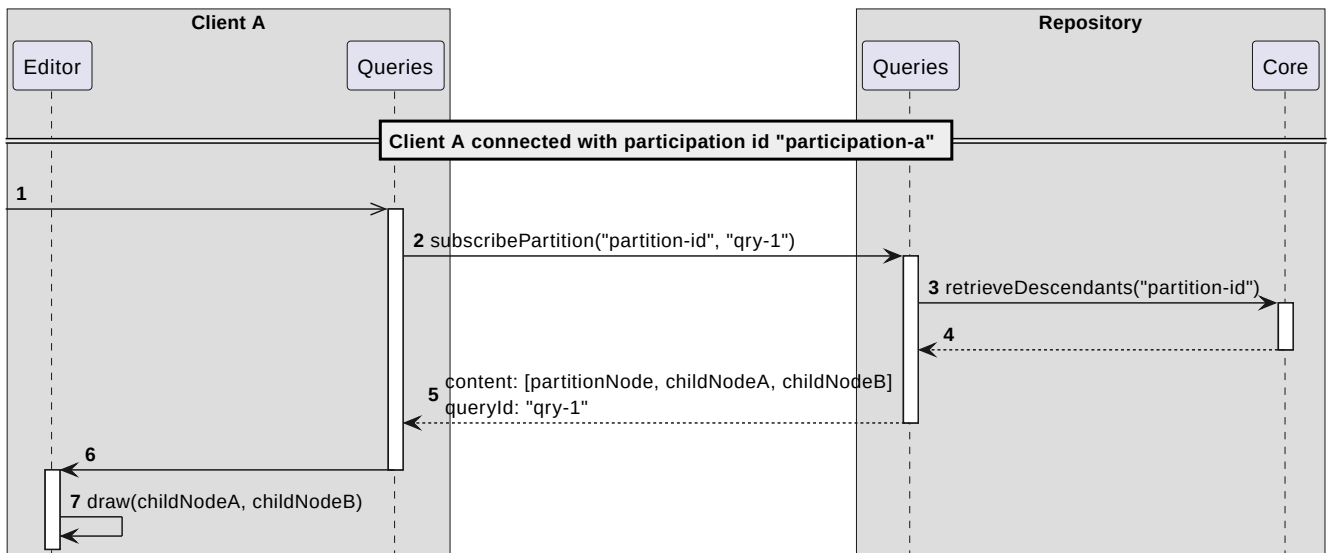
4. Scenarios

4.1. Virgin repository, first time client connects



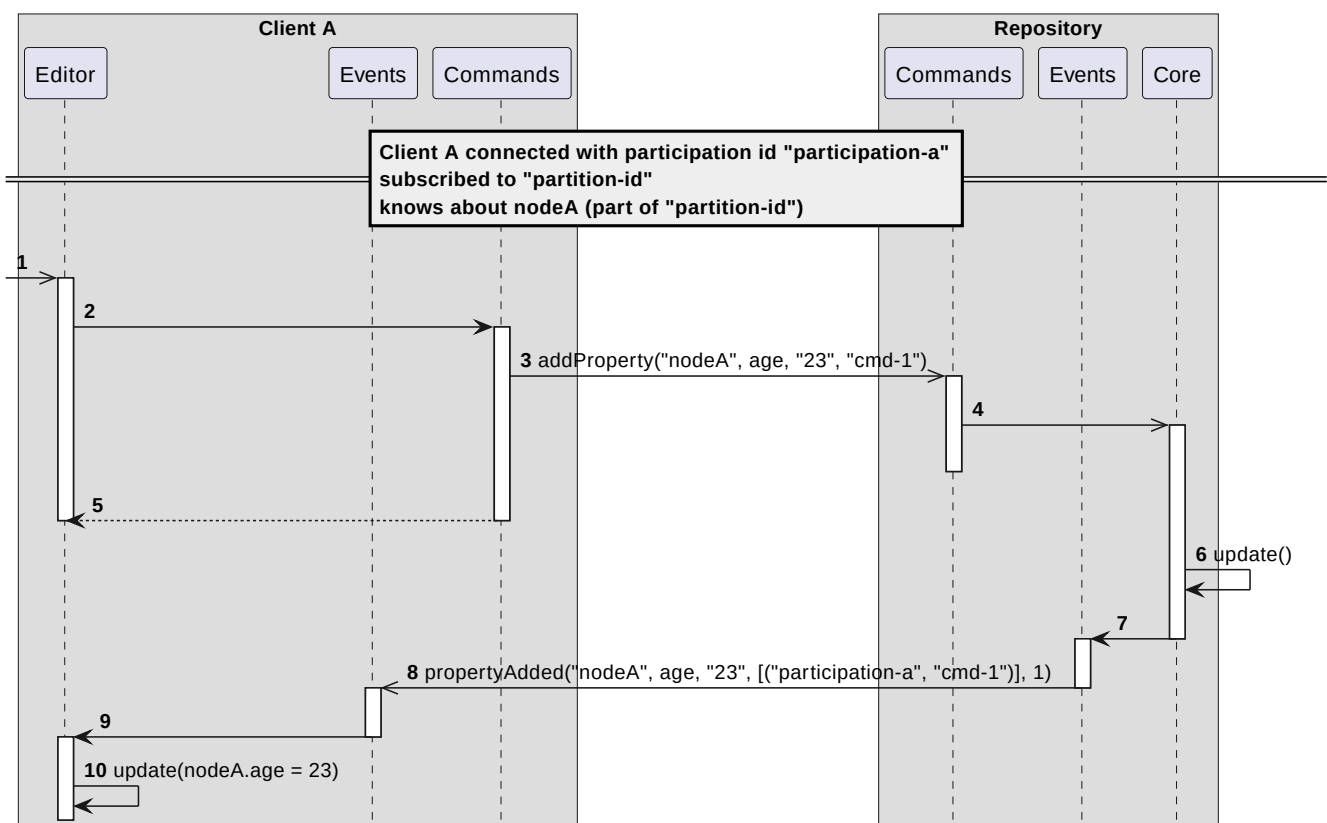
1. Client **signs on** to repository **repo-alpha**, with protocol version **2026.1**, and client id **myClient**.
2. Repository confirms participation with participation id **participation-a**.
3. Client initiates new partition.
4. Client sends **Add partition** command with **nodeA** payload (details omitted) and command id **cmd-1**.
5. Repository registers command for processing.
6. Repository updates internal representation.
7. Repository creates event for update.
8. Repository emits event **Partition added** with **nodeA** payload and sequence number **1**. It includes one **CommandSource** with value (**participation-a**, **cmd-1**)
9. Client receives event and forwards information to editor.
10. Client draws **nodeA**.

4.2. Client reconnects to known partition



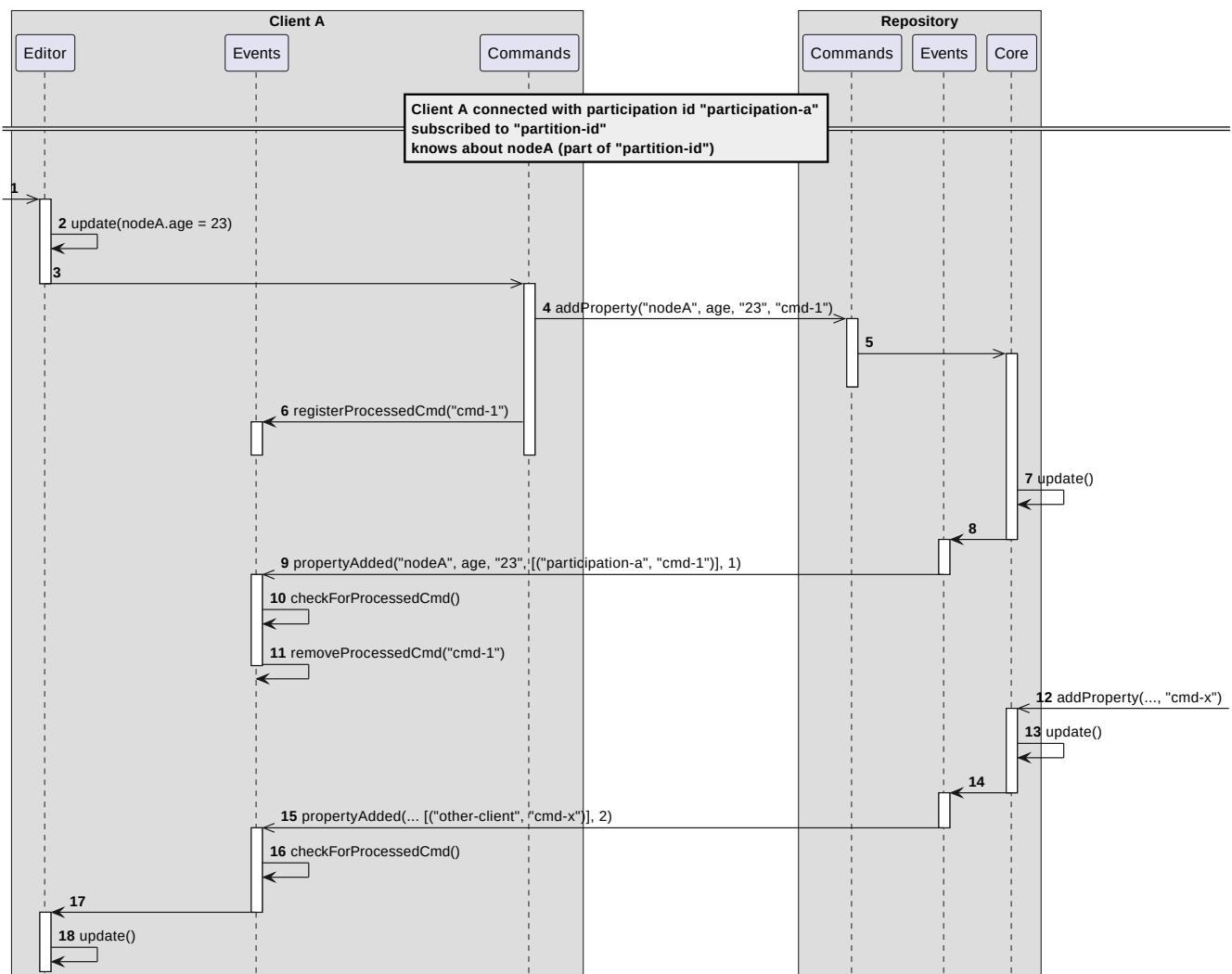
1. Client knows the relevant partition, wants to subscribe.
2. Client sends **Subscribe to partition contents** query with partition node id **partition-id**.
3. Repository retrieves all contents of partition with node id **partition-id**.
4. Repository found contents.
5. Repository responds to query with id **qry-1** with contents of 3 nodes: **partitionNode**, **childNodeA**, **childNodeB** (details omitted).
6. Client receives query result and forwards information to editor.
7. Client draws **childNodeA** and **childNodeB**. **partitionNode** is not drawn, it represents the canvas.

4.3. Client changes known node with remote update



1. User enters new property value.
2. Client editor forwards user action into command.
3. Client sends **Add property** command with **nodeA** node id, **age** property (details omitted), **23** value and command id **cmd-1**.
4. Repository registers command for processing.
5. Client informs editor of command submission.
6. Repository updates internal representation.
7. Repository creates event for update.
8. Repository emits event **Property added** with **nodeA** node id, **age** property (details omitted), **23** value and sequence number **1**. It includes one **CommandSource** with value (**participation-a**, **cmd-1**)
9. Client receives event and forwards information to editor.
10. Client updates **nodeA** with new property value.

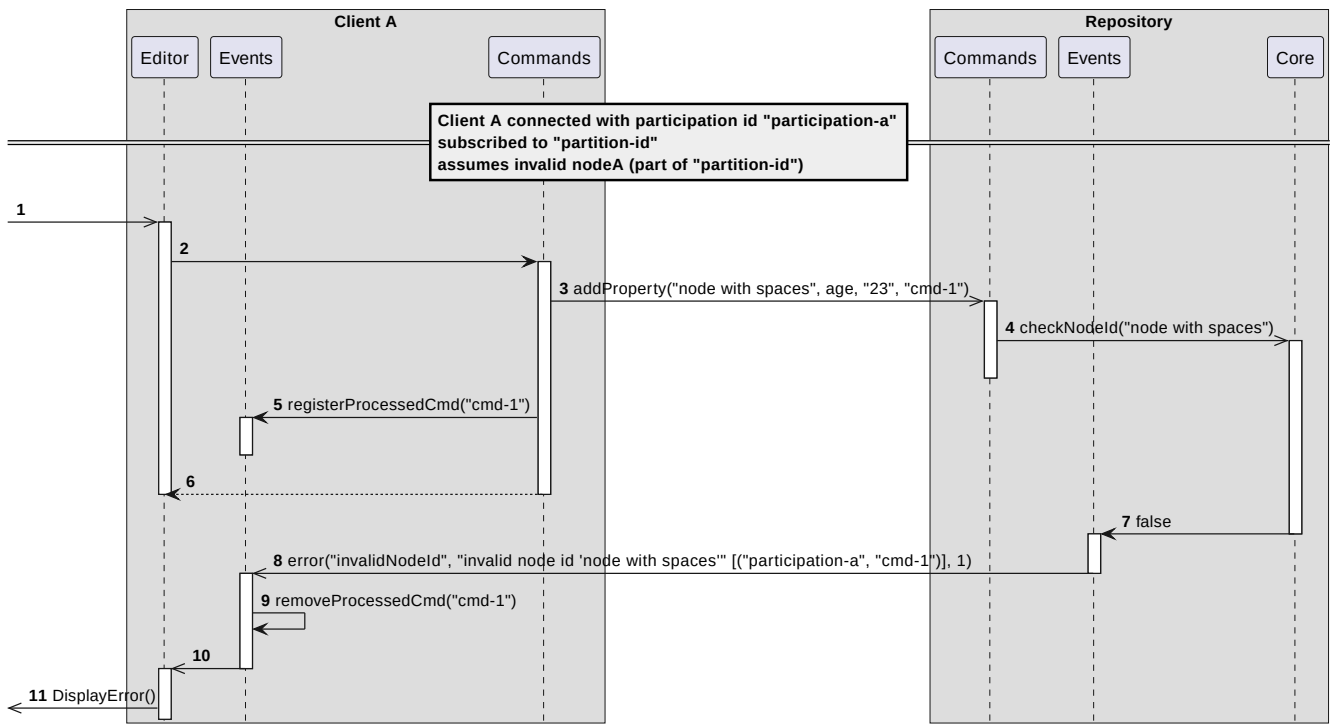
4.4. Client changes known node with local update



1. User enters new property value.

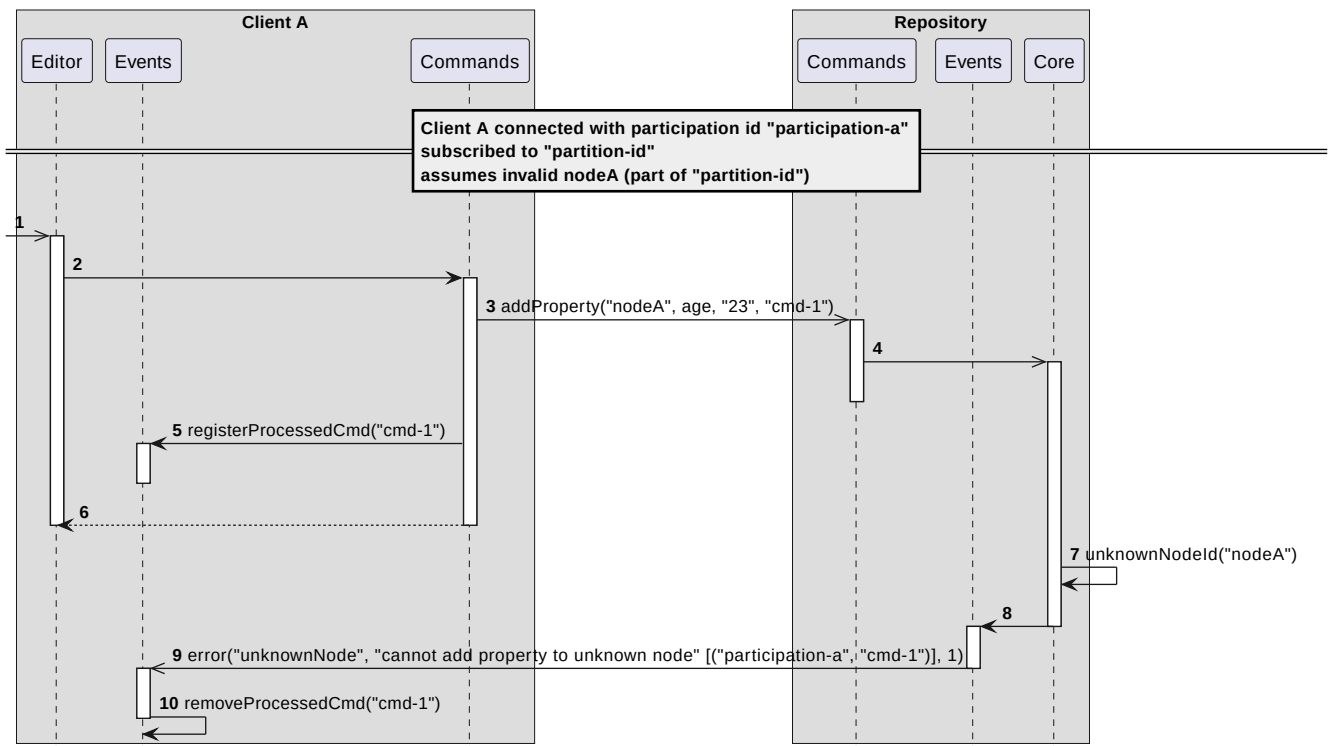
2. Client updates `nodeA` with new property value.
3. Client editor forwards user action into command.
4. Client sends `Add property` command with `nodeA` node id, `age` property (details omitted), `23` value and command id `cmd-1`.
5. Repository registers command for processing.
6. Client registers processed command id `cmd-1`.
7. Repository updates internal representation.
8. Repository creates event for update.
9. Repository emits event `Property added` with `nodeA` node id, `age` property (details omitted), `23` value and sequence number `1`. It includes one `CommandSource` with value `(participation-a, cmd-1)`
10. Client found a matching registered processed command id.
11. Client unregisters processed command id `cmd-1`, and does NOT forward information to editor — the event originates from this client, thus has already been processed.
12. Another client sends `Add property` command with command id `cmd-x` (other parameters omitted).
13. Repository updates internal representation.
14. Repository creates event for update.
15. Repository emits event `Property added` with sequence number `2` (other parameters omitted). It includes one `CommandSource` with value `(other-client, cmd-x)`
16. Client did not find any matching registered processed command id.
17. Client forwards information to editor.
18. Client updates with new property value.

4.5. Client tries to change invalid node id



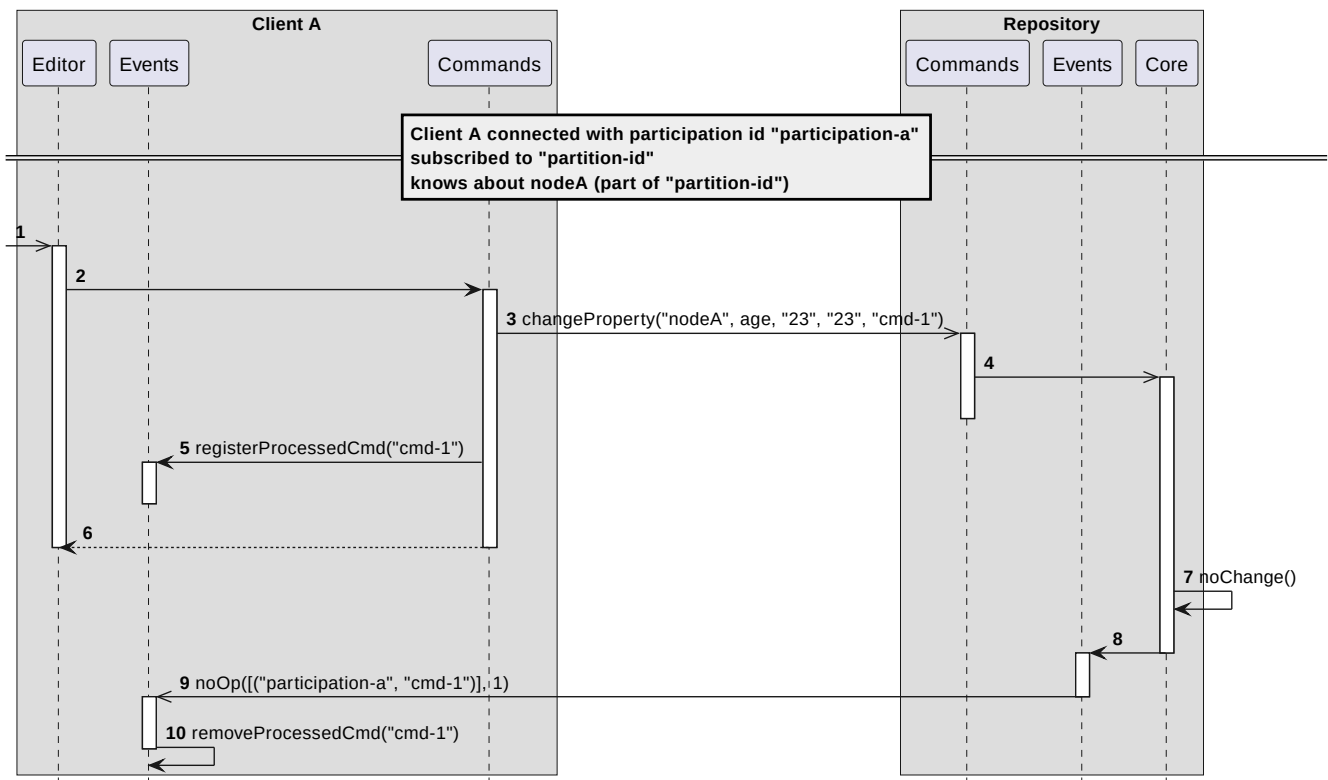
1. User enters new property value on node **node with spaces** that's known to the editor.
2. Client editor forwards user action into command.
3. Client sends **Add property** command with **node with spaces** node id, **age** property (details omitted), **23** value and command id **cmd-1**.
4. Repository checks whether node id **node with spaces** is valid.
5. Client registers processed command id **cmd-1**.
6. Client informs editor of command submission.
7. Repository rejects invalid node id.
8. Repository sends **ErrorEvent** only to this participation with **invalidNodeId** error code, **invalid node id 'node with spaces'** message and sequence number **1**. It includes one **CommandSource** with value (**participation-a**, **cmd-1**).
9. Client removes **cmd-1** from open commands list.
10. Client informs editor of failed command.
11. Client informs user.

4.6. Client tries to change unknown node with failed event



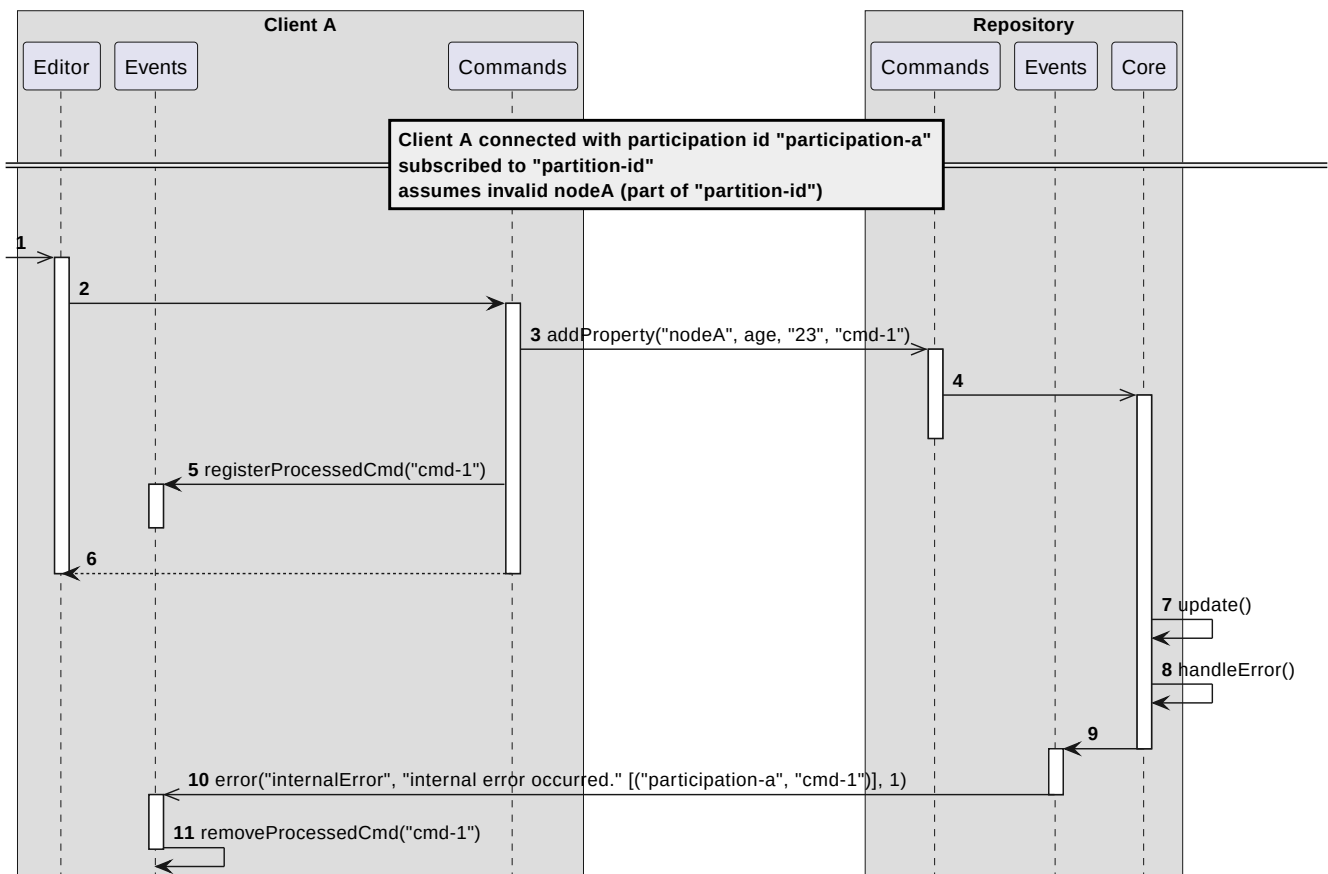
1. User enters new property value on node **nodeA** that's known to the editor.
2. Client editor forwards user action into command.
3. Client sends **Add property** command with **nodeA** node id, **age** property (details omitted), **23** value and command id **cmd-1**.
4. Repository registers command for processing.
5. Client registers processed command id **cmd-1**.
6. Client informs editor of command submission.
7. Repository realizes it does not know a node with id **nodeA**
8. Repository creates event for notification.
9. Repository emits **ErrorEvent** with **unknownNode** error code, **cannot add property to unknown node** message and sequence number **1**. It includes one **CommandSource** with value (**participation-a**, **cmd-1**).
10. Client removes **cmd-1** from open commands list.

4.7. Client with remote update changes property to same value as it already has



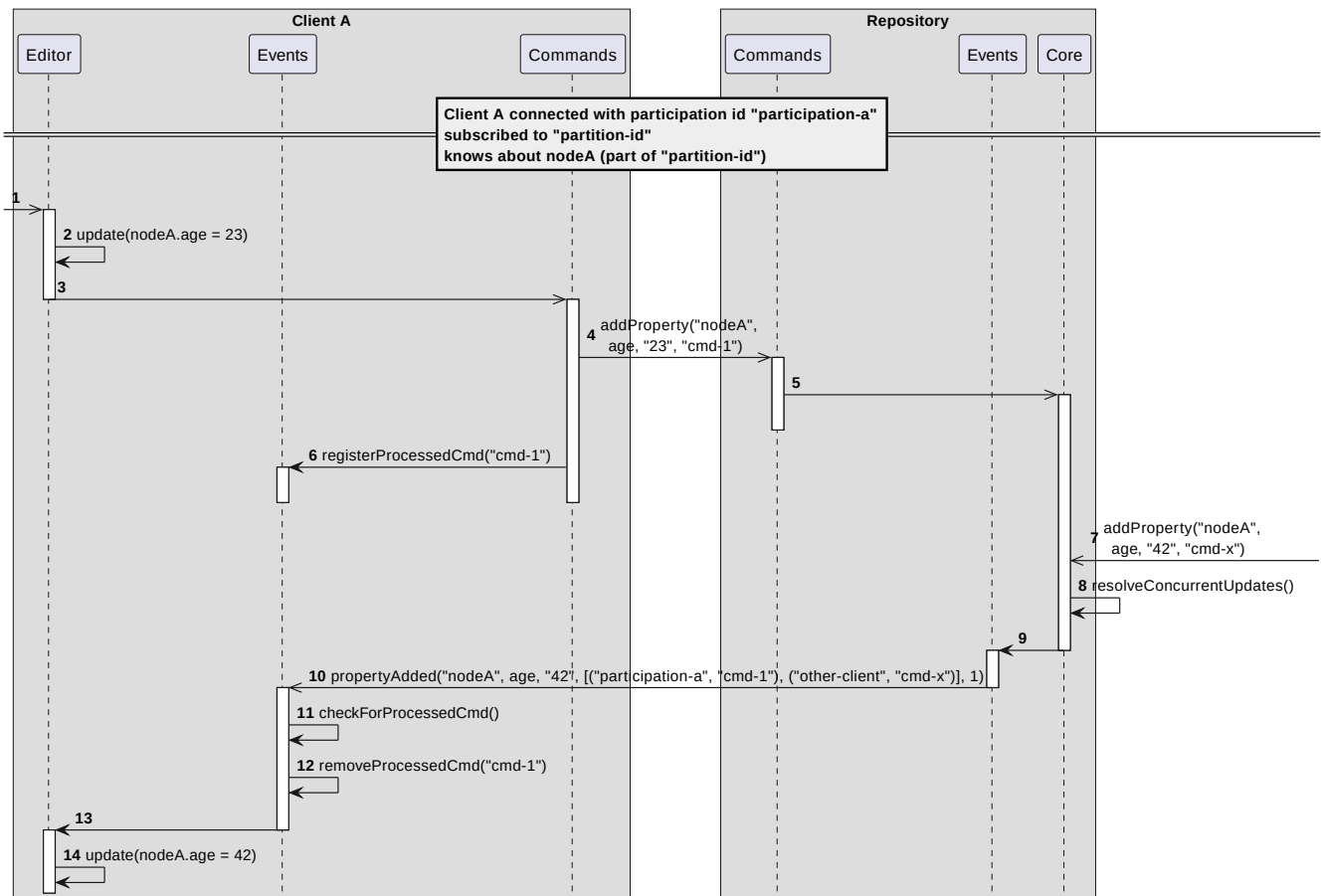
1. User enters changed property value that's the same value as the old value.
2. Client editor forwards user action into command.
3. Client sends **Change property** command with **nodeA** node id, **age** property (details omitted), **23** old value, **23** new value and command id **cmd-1**.
4. Repository registers command for processing.
5. Client registers processed command id **cmd-1**.
6. Client informs editor of command submission.
7. Repository determines no change to internal representation.
8. Repository creates event for notification.
9. Repository emits **No-op Event** event with **CommandSource** with value (**participation-a**, **cmd-1**) and sequence number **1**.
10. Client removes **cmd-1** from open commands list.

4.8. Repository has bug, needs to tell client it couldn't apply its commands



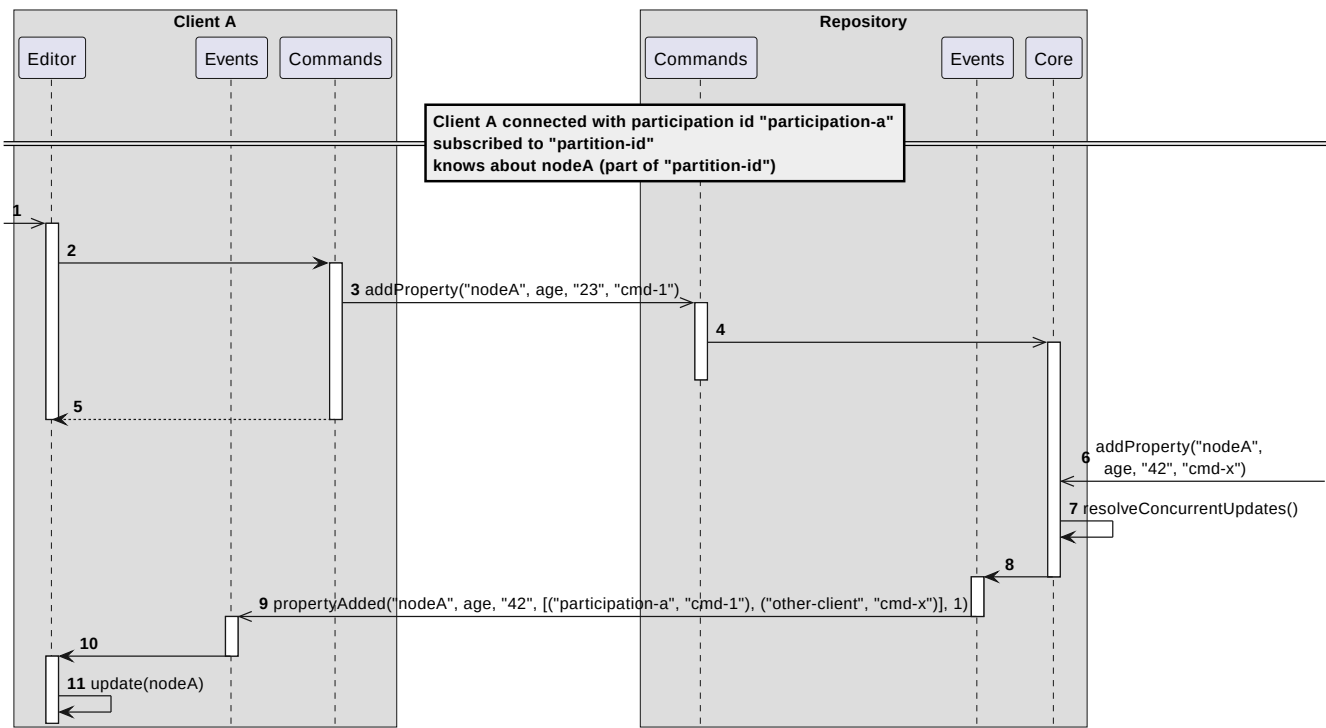
1. User enters new property value on node **nodeA** that's known to the editor.
2. Client editor forwards user action into command.
3. Client sends **Add property** command with **nodeA** node id, **age** property (details omitted), **23** value and command id **cmd-1**.
4. Repository registers command for processing.
5. Client registers processed command id **cmd-1**.
6. Client informs editor of command submission.
7. Repository tries to update internal representation. Some error occurs (e.g. database unavailable).
8. Repository handles the error.
9. Repository creates event for notification.
10. Repository emits **ErrorEvent** with **internalError** error code, **internal error occurred** message and sequence number **1**. It includes one **CommandSource** with value (**participation-a**, **cmd-1**).
11. Client removes **cmd-1** from open commands list.

4.9. Concurrent changes to known node with local update



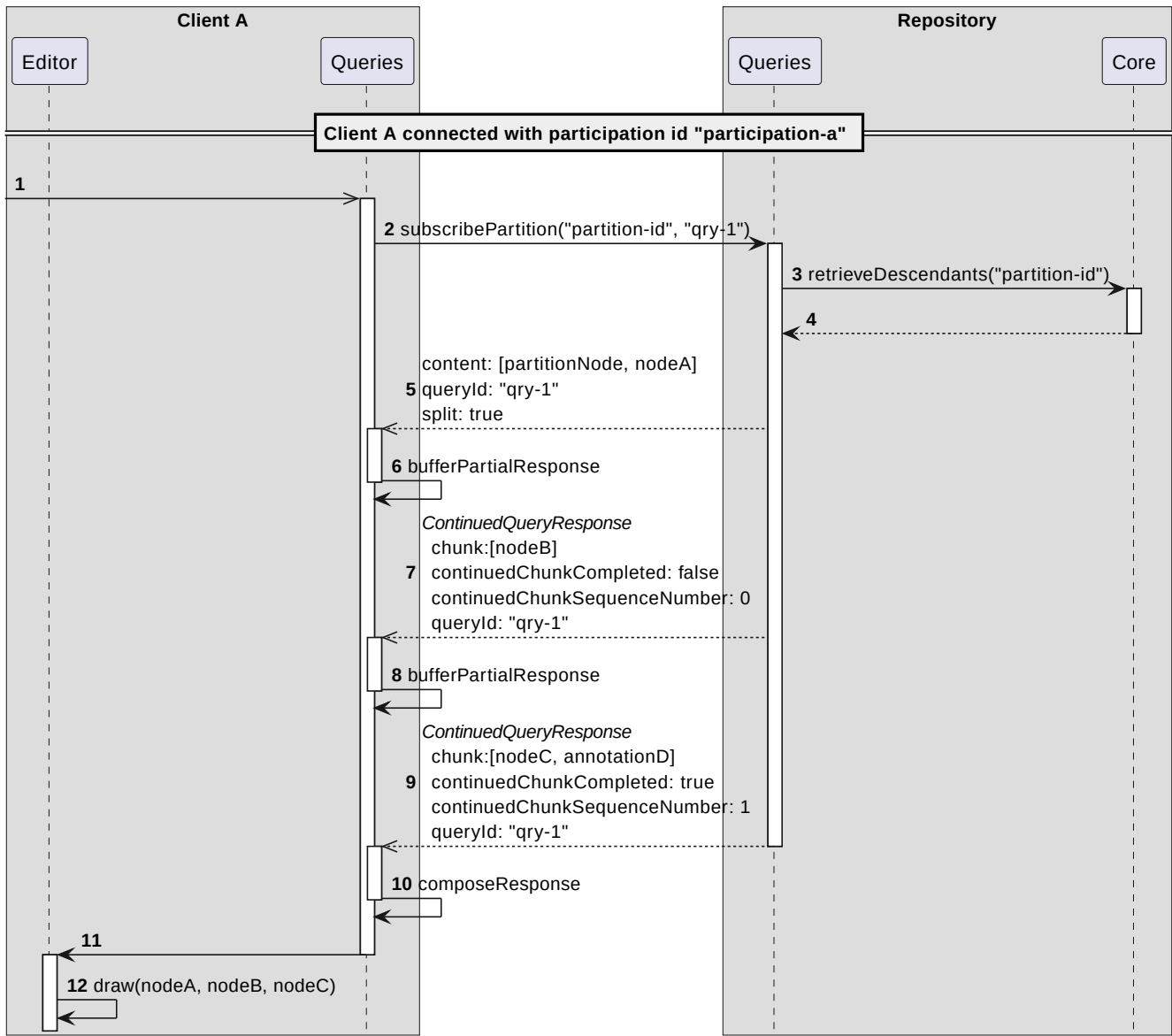
1. User enters new property value.
2. Client updates **nodeA** with new property value.
3. Client editor forwards user action into command.
4. Client sends **Add property** command with **nodeA** node id, **age** property (details omitted), **23** value and command id **cmd-1**.
5. Repository registers command for processing.
6. Client registers processed command id **cmd-1**.
7. Another client sends **Add property** command **nodeA** node id, **age** property (details omitted), **42** value and command id **cmd-x**.
8. Repository resolves concurrent updates t **nodeA**, **age** property with "last write wins" strategy. Repository updates internal representation with value **42**.
9. Repository creates event for update.
10. Repository emits event **Property added** with **nodeA** node id, **age** property (details omitted), **23** value and sequence number **1**. It includes one **CommandSource** with values (**participation-a**, **cmd-1**) and (**other-client**, **cmd-x**).
11. Client found a matching registered processed command id.
12. Client unregisters processed command id **cmd-1**.
13. Not all command sources could be matched against registered processed command ids. Client forwards information to editor.
14. Client updates with new property value.

4.10. Concurrent changes to known node with remote update



1. User enters new property value.
2. Client editor forwards user action into command.
3. Client sends **Add property** command with **nodeA** node id, **age** property (details omitted), **23** value and command id **cmd-1**.
4. Repository registers command for processing.
5. Client informs editor of command submission.
6. Another client sends **Add property** command **nodeA** node id, **age** property (details omitted), **42** value and command id **cmd-x**.
7. Repository resolves concurrent updates to **nodeA**, **age** property with "last write wins" strategy. Repository updates internal representation with value **42**.
8. Repository creates event for update.
9. Repository emits event **Property added** with **nodeA** node id, **age** property (details omitted), **23** value and sequence number **1**. It includes one **CommandSource** with values (**participation-a**, **cmd-1**) and (**other-client**, **cmd-x**).
10. Client receives event and forwards information to editor.
11. Client updates **nodeA** with new property value.

4.11. Split query response



1. Client knows the relevant partition, wants to subscribe.
2. Client sends `Subscribe to partition contents` query with partition node id `partition-id`.
3. Repository retrieves all contents of partition with node id `partition-id`.
4. Repository found contents.
5. Repository responds to query with id `qry-1` with contents of 2 nodes: `partitionNode`, `nodeA` (details omitted). Repository decided the response is too big, and splits it up (`split: true`).
6. Client receives split query result and buffers partial response.
7. Repository continues its response to query with id `qry-1` with another `Continued query response` with chunk of 1 node: `nodeB` (details omitted). This is the first in the sequence of continued chunks (`continuedChunkSequenceNumber: 0`) and there will be another continued chunk (`continuedChunkCompleted: false`).
8. Client receives the next split query result and buffers partial response.
9. Repository continues its response to query with id `qry-1` with another `Continued query response` with chunk of 2 nodes: `nodeC`, `annotationD` (details omitted). This is the second in the sequence of continued chunks (`continuedChunkSequenceNumber: 1`) and there will *not* be another

continued chunk (`continuedChunkCompleted: true`).

10. Client receives the final next split query result and composes the response.
11. Client forwards information to editor.
12. Client draws `nodeA` and `nodeB`. `partitionNode` is not drawn, it represents the canvas.

5. Messages

5.1. Parameter syntax

- Parameters suffixed with `?` CAN be omitted, or CAN have a value of `null`. Both representations express the parameter to be unset.^[46]
- Parameters suffixed with `[]` are ordered lists with zero or more elements.

5.2. Parameter types

AdditionalInfos	Optionally empty ^[47] list of additional infos .
NodeId	A node represented by its id .
MetaPointer	MetaPointer-tuple to describe an M2 element (classifier, feature, etc.). Encoded as meta-pointer .
PropertyValue	Property value encoded as property value .
ResolveInfo	ResolveInfo of a reference as string. Can be <code>null</code> .
Index	Position inside a (list of) multiple link or annotations as integer ≥ 0 . Cannot be <code>null</code> . Use value <code>0</code> for single links .
IndexOffset	Relative position inside a (list of) multiple link or annotations as integer. Cannot be <code>0</code> or <code>null</code> .
Command	Any command .
CommandId	Id of a command. ^[27] Same format as identifiers ; must be unique per participation .
QueryId	Id of a query. Same format as identifiers ; must be unique per participation .
DeltaChunk	Chunk of nodes as described in serialization specification , but without top-level <code>serializationFormatVersion</code> and <code>languages</code> members. Thus, the serialization chunk has only the top-level member <code>nodes</code> . ^{[48][49]}

Nodes in the chunk MAY have references to already [existing nodes](#), and already [existing nodes](#) MAY have references to nodes in the chunk.^[50]

Chunks have different constraints, depending on use case:

- | | |
|----------------|--|
| Single | MUST contain a single <i>anchor node</i> , and all other nodes MUST be descendants of the <i>anchor node</i> . The chunk MUST contain all features of the <i>anchor node</i> and all its descendants, i.e. the <i>anchor node</i> is complete. |
| Multi | MAY contain more than one <i>root node</i> (i.e. independent subtrees), and the chunk MUST contain all descendants of each <i>root node</i> . MUST contain all features of all contained nodes. |
| Shallow | MAY contain more than one <i>root node</i> (i.e. independent subtrees), and each root node might be incomplete. |

DepthLimit

Depth of returned [shallow chunk](#). MUST be an integer in the range of 0 to 2147483647 (i.e. $2^{31} - 1$)^[51], with

- 0 meaning "return only the *root node(s)*",
- 1 meaning "return the *root node(s)* and their direct children/annotations",
- 2 meaning "return the *root node(s)*, their direct children/annotations, and the direct children/annotations of these",
- etc.

SplitFlag

boolean flag whether the chunk is complete within this [splittable message](#).^[36]

ContinuedChunkCompletedFlag

boolean flag whether this [continued chunk](#) completes its [splittable message](#).^[36]

ContinuedChunkSequenceNumber

orders connected [continued chunks](#). Positive integer that starts at 1 and increments by one for each consecutive [continued chunk](#).^{[36][52]}

Event

Any [event](#).

ParticipationId

Id of a [participation](#). Same format as [identifiers](#); must

	be unique for all concurrent participations.
clientId	Id of a client . Same format as identifiers ; must be stable over time for the same client to use reserved free node ids .
RepositoryId	Id of a repository . Same format as identifiers ; must be unique for all repositories in the same context. LionWeb doesn't specify the scope of the context. ^[11]
CommandSource	(ParticipationId , CommandId) tuple ^[5] an event is based on. This allows a client to know when its command has been processed (either successful or not).
EventSequenceNumber	event sequence number of this event.
FreeId	Free node id .
DeltaProtocolVersion	Version of the delta protocol, e.g. 2026.1 .
ErrorCode	Code of an error. Same format as identifiers ; must be unique for reported errors.
ErrorMessage	Human-readable message of an error. Can be any string, including empty string.
boolean	A boolean value, i.e. either true or false .
count	An integer value in the range of 0 to 2147483647 (i.e. $2^{31} - 1$).

5.2.1. New nodes

The term "new node" refers to nodes that MUST NOT yet exist in the repository.^[50]

5.2.2. Existing nodes

The term "existing node" refers to nodes that MUST exist in the repository.

5.2.3. Reused nodes

The term "reused node" refers to nodes that are removed and re-added in the same command/event.^[53] A client SHOULD keep the identity of the reused node in its internal representation.

These *replace with new content commands* / *replace with new content events* are:

- [Replace child](#) / [Child replaced](#)
- [Replace annotation](#) / [Annotation replaced](#)
- Indirectly any command / event nested within [CompositeCommand](#) / [CompositeEvent](#).^[54]

5.2.4. How to address references

We refer to references by the tuple of (*parent*, *reference*, *index*). Rationale: Both *reference* and *resolveInfo* can be *null*, so they aren't suitable for identifying the reference. However, *reference* and *resolveInfo* MUST NOT both be *null*. The *reference* value has preference over the *resolveInfo* value in the following sense: if the *reference* value resolves to a node n_1 and the *resolveInfo* value resolves to a different node n_2 , then the reference (as a whole) MUST resolve to n_1 .

5.3. Message groups

For ease of reference, we label different sets of messages. The sets are non-exclusive, i.e. they overlap.

5.3.1. Create node messages

All messages that create new nodes. They contain a [DeltaChunk](#).

- [Add partition](#) / [Partition added](#)
- [Add child](#) / [Child added](#)
- [Replace child](#) / [Child replaced](#)
- [Add annotation](#) / [Annotation added](#)
- [Replace annotation](#) / [Annotation replaced](#)

5.3.2. Remove node messages

All messages that remove a node from the repository. They contain the id of the removed node, and ids of all removed descendants.

- [Delete partition](#) / [Partition deleted](#)
- [Delete child](#) / [Child deleted](#)
- [Delete annotation](#) / [Annotation deleted](#)
- [Move child from containment in other parent and replace existing child](#) / [Child moved from containment in other parent and replaced existing child](#)
- [Move child from other containment in same parent and replace existing child](#) / [Child moved from other containment in same parent and replaced existing child](#)
- [Move child in same containment in same parent and replace existing child](#) / [Child moved in same containment in same parent and replaced existing child](#)
- [Move annotation from other parent and replace existing annotation](#) / [Annotation moved from other parent and replaced existing annotation](#)
- [Move annotation in same parent and replace existing annotation](#) / [Annotation moved in same parent and replaced existing annotation](#)

5.3.3. Add value messages

All messages that set a value for an unset property or reference. They contain the parent, the feature, and the new value.

- [Add property](#) / [Property added](#)
- [Add reference](#) / [Reference added](#)

5.3.4. Add node messages

All messages that add a new child node. They contain the parent and the new node.

- [Add child](#) / [Child added](#)
- [Add annotation](#) / [Annotation added](#)

5.3.5. Change value messages

All messages that overwrite an existing property or reference with a new one. They contain the parent, the feature, and the old and new value.

- [Change property](#) / [Property changed](#)
- [Change reference](#) / [Reference changed](#)

5.3.6. Replace node messages

All messages that replace an existing node with a new one. They contain the parent, the new node, and the id of the deleted node and ids of all deleted descendants.

- [Replace child](#) / [Child replaced](#)
- [Replace annotation](#) / [Annotation replaced](#)

5.3.7. Delete value messages

All messages that delete an existing property or reference. They contain the parent, the feature, and the old value.

- [Delete property](#) / [Property deleted](#)
- [Delete reference](#) / [Reference deleted](#)

5.3.8. Delete node messages

All messages that delete an existing child node. They contain the parent, the id of the deleted node, and ids of all deleted descendants.

- [Delete child](#) / [Child deleted](#)
- [Delete annotation](#) / [Annotation deleted](#)

5.3.9. Move or replace messages

All messages that move and/or replace a node. They contain the old and new parent, and the moved node.

- Move child from containment in other parent / Child moved from containment in other parent
- Move child from other containment in same parent / Child moved from other containment in same parent
- Move child in same containment in same parent / Child moved in same containment in same parent
- Move child from containment in other parent and replace existing child / Child moved from containment in other parent and replaced existing child
- Move child from other containment in same parent and replace existing child / Child moved from other containment in same parent and replaced existing child
- Move child in same containment in same parent and replace existing child / Child moved in same containment in same parent and replaced existing child
- Move annotation from other parent / Annotation moved from other parent
- Move annotation in same parent / Annotation moved in same parent
- Move annotation from other parent and replace existing annotation / Annotation moved from other parent and replaced existing annotation
- Move annotation in same parent and replace existing annotation / Annotation moved in same parent and replaced existing annotation

5.3.10. Move-only messages

All messages that move a node. They contain the old and new parent, and the moved node.

- Move child from containment in other parent / Child moved from containment in other parent
- Move child from other containment in same parent / Child moved from other containment in same parent
- Move child in same containment in same parent / Child moved in same containment in same parent
- Move annotation from other parent / Annotation moved from other parent
- Move annotation in same parent / Annotation moved in same parent

5.3.11. Move and replace messages

All messages that move and replace a node. They contain the old and new parent, the moved node, and the replaced node.

- Move child from containment in other parent and replace existing child / Child moved from containment in other parent and replaced existing child
- Move child from other containment in same parent and replace existing child / Child moved

from other containment in same parent and replaced existing child

- [Move child in same containment in same parent and replace existing child](#) / [Child moved in same containment in same parent and replaced existing child](#)
- [Move annotation from other parent and replace existing annotation](#) / [Annotation moved from other parent and replaced existing annotation](#)
- [Move annotation in same parent and replace existing annotation](#) / [Annotation moved in same parent and replaced existing annotation](#)

5.3.12. Move or replace child messages

All messages that move and/or replace a child node in a containment (*not* an annotation). They contain the old and new parent, the moved node, and the old and new containment.

- [Move child from containment in other parent](#) / [Child moved from containment in other parent](#)
- [Move child from other containment in same parent](#) / [Child moved from other containment in same parent](#)
- [Move child in same containment in same parent](#) / [Child moved in same containment in same parent](#)
- [Move child from containment in other parent and replace existing child](#) / [Child moved from containment in other parent and replaced existing child](#)
- [Move child from other containment in same parent and replace existing child](#) / [Child moved from other containment in same parent and replaced existing child](#)
- [Move child in same containment in same parent and replace existing child](#) / [Child moved in same containment in same parent and replaced existing child](#)

5.3.13. Move or replace annotation messages

All messages that move and/or replace an annotation node (*not* a child node in a containment). They contain the old and new parent, and the moved node.

- [Move annotation from other parent](#) / [Annotation moved from other parent](#)
- [Move annotation in same parent](#) / [Annotation moved in same parent](#)
- [Move annotation from other parent and replace existing annotation](#) / [Annotation moved from other parent and replaced existing annotation](#)
- [Move annotation in same parent and replace existing annotation](#) / [Annotation moved in same parent and replaced existing annotation](#)

5.3.14. Move or replace from other parent

All messages that move and/or replace a node from another parent. They contain the old parent, the new parent, and the moved node.

- [Move child from containment in other parent](#) / [Child moved from containment in other parent](#)
- [Move child from containment in other parent and replace existing child](#) / [Child moved from](#)

containment in other parent and replaced existing child

- [Move annotation from other parent / Annotation moved from other parent](#)
- [Move annotation from other parent and replace existing annotation / Annotation moved from other parent and replaced existing annotation](#)

5.3.15. Move or replace in same parent

All messages that move and/or replace a node within the same parent. They contain the parent, and the moved node.

- [Move child from other containment in same parent / Child moved from other containment in same parent](#)
- [Move child in same containment in same parent / Child moved in same containment in same parent](#)
- [Move child from other containment in same parent and replace existing child / Child moved from other containment in same parent and replaced existing child](#)
- [Move child in same containment in same parent and replace existing child / Child moved in same containment in same parent and replaced existing child](#)
- [Move annotation in same parent / Annotation moved in same parent](#)
- [Move annotation in same parent and replace existing annotation / Annotation moved in same parent and replaced existing annotation](#)

5.3.16. Move or replace from other containment

All messages that move and/or replace a node from another containment within the same parent. They contain the parent, the old containment, the new containment, and the moved node.

- [Move child from other containment in same parent / Child moved from other containment in same parent](#)
- [Move child from other containment in same parent and replace existing child / Child moved from other containment in same parent and replaced existing child](#)

5.3.17. Move or replace in same containment

All messages that move and/or replace a node within the same containment. They contain the parent, the containment, and the moved node.

- [Move child in same containment in same parent / Child moved in same containment in same parent](#)
- [Move child in same containment in same parent and replace existing child / Child moved in same containment in same parent and replaced existing child](#)

5.4. Errors

Commonly used errors, referred to by their *technical name* as [ErrorCode](#).

Additional infos of errors are optional. A repository SHOULD provide them, if possible. A client MUST NOT require them.

5.4.1. Error additional info types

Some types can only occur in additional infos of errors.

Parameter

Technical name of the parameter an error refers to.

Subscription mode

The technical name of the current subscription mode, i.e. either [Inform about partition changes](#) or [Subscribe to partition changes](#).

Serialized MetaPointer

string serialization of JSON representation of a [MetaPointer](#).

5.4.2. Generic errors

5.4.2.1. Invalid participation

The [participation](#) the [client](#) uses to communicate with the repository is not valid.

Possible reasons:

- The participation never has been valid.^[55]
- The client [signed off](#).
- The client [disconnected](#).

Technical name

`invalidParticipation`

5.4.2.2. Message syntax incorrect

The delta message does not adhere to the syntax defined by this specification.

Technical name

`messageSyntaxIncorrect`

5.4.2.3. Message kind unknown

The repository does not recognize the message.

See also [Custom messages](#).

Technical name

`messageKindUnknown`

5.4.2.4. Missing required message parameter

A [query response](#) or [command](#) is missing a required parameter.

Technical name

`missingRequiredMessageParameter`

Additional infos

parameter [Parameter](#) Name of the missing parameter.

5.4.2.5. Unknown node

Client tries to operate on a node unknown to the repository.

See also [Move errors](#).

Technical name

`unknownNode`

Additional infos

node [NodeId](#) of the unknown node.

parameter [Parameter](#) Name of the parameter containing `node`.

5.4.2.6. The given parent of a node is not its parent

Client provided an existing node **P** as parent and an existing node **C** as a child of **P**, but **P** does not have **C** as a child (annotation or containment).

NOTE

This error differs from [Parent / containment mismatch](#): This error relates to two already existing nodes, whereas [Parent / containment mismatch](#) relates to two nodes in a [DeltaChunk](#).

Technical name

`parentMismatch`

Additional infos

parent [NodeId](#) of the provided parent.

child [NodeId](#) of the provided child.

actualParent [NodeId](#) of the actual parent of `child`.

parentParameter [Parameter](#) Name of the parameter containing `parent`.

childParameter [Parameter](#) Name of the parameter containing `child`.

5.4.2.7. Index out of range

Client tries to operate on an index beyond the number of existing elements.

See also [Move errors](#).

Technical name

`indexOutOfRange`

Additional infos

parent [NodeId](#) of the parent containing the feature with unknown index.

feature [Serialized MetaPointer](#) of the feature with unknown index.

size [Index](#) The actual size of the feature.

index [Index](#) Unknown index.

parameter [Parameter](#) Name of the parameter containing `index`.

5.4.3. Chunk errors

5.4.3.1. Invalid anchor node

A [single chunk](#) doesn't have exactly one [anchor node](#), or the anchor node's id or parent is invalid.

Technical name

`invalidAnchorNode`

5.4.3.2. Duplicate node id

Chunk contains duplicate node id

Technical name

`duplicateNodeId`

Additional infos

duplicate [NodeId](#) of the already duplicate node.

5.4.3.3. Parent / containment mismatch

Chunk contains a parent node **P** has a child (containment or annotation) node **C**, but **C** doesn't mention **P** as parent.

Technical name

`parentContainmentMismatch`

Additional infos

parent [NodeId](#) of the parent node.

child [NodeId](#) of the child node.

parentId [NodeId](#) of the parent mentioned in the child.

5.4.3.4. Circular containment

Chunk contains a parent node **P** contains a child (containment or annotation) node **C**, that directly or indirectly contains **P**. Includes the edge case **P** contains **P**.

Additional infos

cycle **NodeId**[] Comma-separated list of all node ids that are part of the containment cycle. MUST NOT contain any whitespace.

Technical name

circularContainment

5.4.4. Create node errors

Reported by all **Create node messages** commands.

5.4.4.1. Node already exists

Client tries to create a node with an id that already exists.

Technical name

nodeAlreadyExists

Additional infos

duplicate **NodeId** of the already existing node.

parameter **Parameter** Name of the parameter containing **duplicate**.

5.4.5. Move errors

Reported by all **Move or replace messages** commands.

5.4.5.1. Old and new parent identical

Technical name

oldAndNewParentIdentical

Additional infos

parent **NodeId** of both the old and the new parent.

5.4.5.2. Old and new containment identical

Technical name

oldAndNewContainmentIdentical

Additional infos

containment **Serialized MetaPointer** of the both the old and the new containment.

5.4.5.3. Old and new index identical

Technical name

`oldAndNewIndexIdentical`

Additional infos

index `Index` of both the old and new index.

5.4.5.4. Index offset is invalid

Client provided an index offset which is either 0, or an integer such that a given "old" index + the index offset is not a valid index (in the list that's the subject of the action triggering this error).

Technical name

`invalidIndexOffset`

Additional infos

parent `NodeId` of the parent.

feature `Serialized MetaPointer?` of given feature. Empty for annotations.

offset `IndexOffset` given offset.

min `IndexOffset` Minimal valid offset.

max `IndexOffset` Maximal valid offset.

parameter `Parameter` Name of the parameter containing `offset`.

5.4.6. Annotation errors

5.4.6.1. Annotation at index is not the given annotation

Client provided an annotation node and an index, but the node is not at the given index.

Technical name

`indexAnnotationMismatch`

Additional infos

parent `NodeId` of the parent.

index `Index` given index.

annotation `NodeId` of the given annotation.

actual `NodeId` of the actual annotation at the given index.

parameter `Parameter` Name of the parameter containing `index`.

5.4.7. Containment errors

5.4.7.1. Unknown containment

Client tries to change or delete a containment not present in the parent node.

Technical name

`unknownContainment`

Additional infos

parent [NodeId](#) of the parent missing the requested containment.

containment [Serialized MetaPointer](#) of the requested [Containment](#).

5.4.7.2. Child at index is not the given node

Client provided a child node and an index, but the node is not at the given index.

Technical name

`indexChildMismatch`

Additional infos

parent [NodeId](#) of the parent.

containment [Serialized MetaPointer](#) of given containment.

index [Index](#) given index.

child [NodeId](#) of the given child.

actual [NodeId](#) of the actual child at the given containment and index.

parameter [Parameter](#) Name of the parameter containing `index`.

5.4.8. Reference errors

5.4.8.1. Unknown reference

Client tries to change or delete a reference not present in the parent node.

Technical name

`unknownReference`

Additional infos

parent [NodeId](#) of the parent missing the requested reference.

reference [Serialized MetaPointer](#) of the requested [Reference](#).

5.4.8.2. Reference already set

A [Reference](#) has already been set to a value, and cannot be set new with this command. Use [Change](#)

[reference](#) command instead.

Technical name

`referenceAlreadySet`

Additional infos

parent	NodeId of the parent containing the already set reference.
reference	Serialized MetaPointer of the already set reference.
target	NodeId ? of the current target of the reference, if any.
resolveInfo	ResolveInfo ? of the current target of the reference, if any.

5.4.8.3. Reference not set

A [Reference](#) has no value, and cannot be changed with this command. Use [Add reference](#) command instead.

Technical name

`referenceNotSet`

Additional infos

parent	NodeId of the parent containing the unset reference.
reference	Serialized MetaPointer of the unset reference.

5.4.8.4. Reference at index is not the given reference

Client provided a target/resolveInfo and an index, but the target/resolveInfo is not at the given index.

Technical name

`indexReferenceMismatch`

Additional infos

parent	NodeId of the parent.
reference	Serialized MetaPointer of given reference.
index	Index given index.
target	NodeId ? of the given target, if any.
resolveInfo	ResolveInfo ? of the given target, if any.
actualTarget	NodeId ? of the actual target at the given reference and index, if any.
actualResolveInfo	ResolveInfo ? of the actual target at the given reference and index, if any.
parameter	Parameter Name of the parameter containing <code>index</code> .

5.4.8.5. Undefined reference target

Client tries to set a reference with neither `target` nor `resolveInfo`

Technical name

`undefinedReferenceTarget`

Additional infos

- parent** `NodeId` of the parent.
- reference** `Serialized MetaPointer` of given reference.
- parameter** `Parameter` Name of the parameter containing `reference`.

5.4.9. Property errors

5.4.9.1. Unknown property

Client tries to change or delete a property not present in the parent node.

Technical name

`unknownProperty`

Additional infos

- parent** `NodeId` of the parent missing the requested property.
- property** `Serialized MetaPointer` of the requested `Property`.

5.4.9.2. Property already set

A `Property` has already been set to a value, and cannot be set new with this command. Use `Change property` command instead.

Technical name

`propertyAlreadySet`

Additional infos

- parent** `NodeId` of the parent containing the already set property.
- property** `Serialized MetaPointer` of the already set property.
- value** `PropertyValue` Current value of the property.

5.4.9.3. Property not set

A `Property` has no value, and cannot be changed with this command. Use `Add property` command instead.

Technical name

`propertyNotSet`

Additional infos

parent [NodeId](#) of the parent containing the unset property.

property [Serialized MetaPointer](#) of the unset property.

5.4.10. Subscription errors

5.4.10.1. Mixed Inform about / Subscribe to Changing Partitions

Client already [subscribed to changing partitions](#), and now requests to be [informed about changing partitions](#) — or vice versa.

Technical name

[mixedInformAboutAndSubscribeToChangingPartitions](#)

Additional infos

currentMode [Subscription mode](#) The technical name of the current subscription mode.

creation [boolean](#) Whether the participation is currently subscribed to newly created partitions.

deletion [boolean](#) Whether the participation is currently subscribed to deleted partitions.

5.4.10.2. Already subscribed to partition

Participation is already subscribed to the requested partition.

Technical name

[alreadySubscribedToPartition](#)

Additional infos

partition [NodeId](#) of the already subscribed partition.

5.4.10.3. Not subscribed to partition

Participation is not subscribed to the requested partition.

Technical name

[notSubscribedToPartition](#)

Additional infos

partition [NodeId](#) of the not subscribed partition.

5.4.10.4. Invalid depth limit

The provided depth limit is negative or beyond the valid depth.

Technical name

[invalidDepthLimit](#)

Additional infos

invalidDepthLimit [depth limit](#) requested by the client.

5.4.10.5. Unsupported protocol version

Technical name

unsupportedProtocolVersion

Additional infos

protocolVersion [DeltaProtocolVersion](#) of the unsupported protocol version.

5.4.10.6. Unknown repository

The requested repository is not known.

Technical name

unknownRepository

Additional infos

repository [RepositoryId](#) of the unknown repository.

5.4.11. Splittable message errors

Reported by all [continued chunk](#) messages.

5.4.11.1. No active split message

Continued chunk received without any active split message, or with a higher [ContinuedChunkSequenceNumber](#) than a previous one for which the [ContinuedChunkCompletedFlag](#) was **true**.

Technical name

noActiveSplitMessage

5.4.11.2. Duplicate continued chunk

Received continued chunk with [ContinuedChunkSequenceNumber](#) that was already seen.

Technical name

duplicateContinuedChunk

Additional infos

duplicateSequenceNumber [ContinuedChunkSequenceNumber](#) The already seen continued chunk sequence number.

5.4.11.3. Unknown splittable message in continued chunk

Continued chunk refers to a non-existent query/command/event.

Technical name

unknownSplittableMessageInContinuedChunk

Additional infos

unknownSplitMessage The unknown query/command/event id.

5.4.11.4. Invalid splittable message in continued chunk

Continued chunk refers to a query/command/event that's not splittable or doesn't have its [SplitFlag](#) set to **true**.

Technical name

invalidSplittableMessageInContinuedChunk

Additional infos

splitMessage	The referenced query/command/event id.
invalidContinuedChunkSequenceNumber	ContinuedChunkSequenceNumber The continued message referring to an invalid continued chunk sequence number.

5.4.11.5. Interrupted continued chunk

Received message on this [channel](#) before the sequence of continued chunks had finished.

Technical name

interruptedContinuedChunk

Additional infos

unfinishedSplittableMessage	The unfinished query/command/event id.
unexpectedMessage	The query/command/event id of the message unexpected because of the still ongoing continued message..

5.4.11.6. Composite command contains splittable

A [CompositeCommand](#) contains [continued chunks](#) or declares a [splittable message](#) as split.^[39]

Technical name

compositeCommandContainsSplittable

Additional infos

commandId	CommandId of the splittable or continued command.
------------------	---

5.5. Custom messages

We support custom messages, i.e. messages that match the general structure of delta messages, but with custom content.^[56]

Technical name

The technical name of all custom messages and custom errors MUST adhere to the same format as [identifiers](#), MUST start with **Custom_**, and MUST have at least 8 characters.^[57]

5.5.1. Custom queries

A client CAN send custom query messages. If the repository understands the custom query, it replies with a matching response; otherwise, the repository MUST send back an [Error response](#) with [Message kind unknown](#).

Custom queries MUST have at least the following parameters:

Required request parameters

- queryId** [QueryId](#) Id of this query.
- additionalInfos** [AdditionalInfos](#) Optionally empty list of [additional infos](#).

Required response parameters

- queryId** [QueryId](#) Id of this query.
- additionalInfos** [AdditionalInfos](#) Optionally empty list of [additional infos](#).

Errors

- [Invalid participation](#)
- [Message kind unknown](#)

5.5.2. Custom commands

A client CAN send custom command messages. If the repository understands the custom command, it replies with a matching event; otherwise, the repository MUST send back an [ErrorEvent](#) with [Message kind unknown](#).

Custom commands MUST have at least the following parameters:

Required parameters

- commandId** [CommandId](#) Id of this command.
- additionalInfos** [AdditionalInfos](#) Optionally empty list of [additional infos](#).

Errors

- [Invalid participation](#)
- [Message kind unknown](#)

5.5.3. Custom events

A repository CAN send custom event messages. If the client understands the custom event, it handles the event accordingly; otherwise, the client SHOULD ignore the message.

Custom events MUST have at least the following parameters:

Parameters

- originCommands** [CommandSource](#)[] List of ([participationId](#), [commandId](#)) pairs this event is based on.

sequenceNumber [EventSequenceNumber](#) Event sequence number of this event.

additionalInfos [AdditionalInfos](#) Optionally empty list of [additional infos](#).

5.6. Queries

5.6.1. Special responses

5.6.1.1. Error response

Any query might return with error parameters instead of its declared response parameters.^[58]

See also [ErrorEvent](#).

Technical name

[ErrorResponse](#)

Response parameters

errorCode [ErrorCode](#) Code of the reported error.

message [ErrorMessage](#) Human-readable message of the error.

queryId [QueryId](#) Id of the related query.

additionalInfos [AdditionalInfos](#) Optionally empty list of [additional infos](#).

5.6.1.2. Continued query response

A [continued chunk](#) for a [splittable](#) response of one of the following queries:^[59]

- [Subscribe to partition contents](#) response [contents](#)
- [List partitions](#) response [partitions](#)
- [List and subscribe partitions](#) response [partitions](#)

We identify the splittable response by its [queryId](#).

Technical name

[ContinuedQueryResponse](#)

Response parameters

chunk [DeltaChunk](#) The continued chunk. The combination of all continued chunks MUST adhere to the same constraints as [queryId](#)'s chunk.

continuedChunkCompleted [ContinuedChunkCompletedFlag](#) Whether this message completes [queryId](#).

continuedChunkSequenceNumber [ContinuedChunkSequenceNumber](#) Number of this message in the sequence of all continued query responses relating to the same [queryId](#).

queryId	QueryId Id of the related splittable query.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Errors

- [No active split message](#)
- [Duplicate continued chunk](#)
- [Unknown splittable message in continued chunk](#)
- [Invalid splittable message in continued chunk](#)
- [Interrupted continued chunk](#)
- [Invalid participation](#)

5.6.2. Subscription

5.6.2.1. Subscribe to partition changes

This client wants to receive [Partition added](#) events on newly created (if [creation](#) is true) or [Partition deleted](#) events on deleted (if [deletion](#) is true) partitions. The client automatically subscribes to newly created partitions (if [creation](#) is true), and receives the full contents of partitions.

Subsequent calls to this query update the client's subscription. For example, let's assume the client first issued this query with [creation](#) = [true](#) and [deletion](#) = [false](#). Later the same client issues this query with [creation](#) = [true](#) and [deletion](#) = [true](#). Afterward, the client will receive [Partition deleted](#) events.^[25]

See also [subscription](#) and [Inform about partition changes](#).

Technical name

[SubscribeToChangingPartitions](#)

Request parameters

creation	boolean Whether this client wants to receive events on newly created partitions (true), or not (false).
deletion	boolean Whether this client wants to receive events on deleted partitions (true), or not (false).
queryId	QueryId Id of this query.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Response parameters

queryId	QueryId Id of this query.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Errors

- [Invalid participation](#)
- [Mixed Inform about / Subscribe to Changing Partitions](#)

5.6.2.2. Inform about partition changes

This client wants to receive [Partition added](#) events on newly created (if `creation` is `true`) or [Partition deleted](#) events on deleted (if `deletion` is `true`) partitions. The client receives partitions contents up to `depthLimit`. The client *does not* subscribe to newly created partitions.^[36]

Subsequent calls to this query update the client's subscription. For example, let's assume the client first issued this query with `creation = true` and `deletion = false`. Later the same client issues this query with `creation = true` and `deletion = true`. Afterward, the client will receive [Partition deleted](#) events.^[25]

See also [subscription](#) and [Subscribe to partition changes](#).

Technical name

`InformAboutChangingPartitions`

Request parameters

creation	boolean Whether this client wants to receive events on newly created partitions (<code>true</code>), or not (<code>false</code>).
deletion	boolean Whether this client wants to receive events on deleted partitions (<code>true</code>), or not (<code>false</code>).
depthLimit	depth limit Depth of <code>newPartition</code> in received Partition added events.
queryId	QueryId Id of this query.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Response parameters

queryId	QueryId Id of this query.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Errors

- [Invalid participation](#)
- [Mixed Inform about / Subscribe to Changing Partitions](#)
- [Invalid depth limit](#)

5.6.2.3. Subscribe to partition contents

This client wants to receive events on any changes to `partition` or any of its descendants.^{[60][24]}

See also [splittable message](#) and [subscription](#).

Technical name

`SubscribeToPartitionContents`

Request parameters

partition	NodeId Node id of the partition this client wants to receive events of.
queryId	QueryId Id of this query.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Response parameters

contents	single chunk Complete contents of partition , i.e. the node with id partition and all of its descendants (including annotation instances).
split?	SplitFlag Whether contents is complete within this message.
queryId	QueryId Id of this query.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Errors

- [Already subscribed to partition](#) for [partition](#).
- [Unknown node](#) for [partition](#).
- [Invalid participation](#)

5.6.2.4. Unsubscribe from partition contents

This client does not want to receive events on any changes to [partition](#) or any of its descendants anymore.^[60]

See also and [subscription](#).

Technical name

[UnsubscribeFromPartitionContents](#)

Request parameters

partition	NodeId Node id of the partition this client wants to stop receiving events of.
queryId	QueryId Id of this query.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Response parameters

queryId	QueryId Id of this query.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Errors

- [Not subscribed to partition](#) for [partition](#).
- [Unknown node](#) for [partition](#).
- [Invalid participation](#)

5.6.3. Participation

Even though participation handling does not relate to repository contents, we consider them queries.^[4]

5.6.3.1. Sign On (start a participation)

The [client](#) starts a [participation](#) for a specific [repository](#).

NOTE

signing on is not the same as logging in: The latter includes some authentication/authorization check, which is out of scope of this specification.

Technical name

SignOn

Request parameters

deltaProtocolVersion	DeltaProtocolVersion Version of the delta protocol this client wishes to communicate in.
clientId	clientId Id of the client signing on. ^[1]
repositoryId	RepositoryId Id of the repository to which the client wants to establish a participation.
queryId	QueryId Id of this query.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Response parameters

participationId	ParticipationId the repository assigns to this participation.
queryId	QueryId Id of this query.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Errors

- [Unsupported protocol version](#) for **deltaProtocolVersion**
- [Unknown repository](#) for **repositoryId**

5.6.3.2. Sign Off (end a participation)

The [client](#) ends a [participation](#).^[61]

NOTE

signing off is not the same as logging off: The latter includes some authentication/authorization invalidation, which is out of scope of this specification.

Technical name

SignOff

Request parameters

queryId	QueryId Id of this query.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Response parameters

queryId	QueryId Id of this query.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Errors

- [Invalid participation](#)

5.6.3.3. Reconnect (resume an existing participation)

The [client](#) has been technically disconnected, but still knows its [participation id](#). Then the client can ask to reconnect to the repository.^[20]

Technical name

Reconnect

Request parameters

deltaProtocolVersion	DeltaProtocolVersion MUST be the same id as used previously for the same participationId . ^[62]
clientId	clientId Id of the reconnecting client. MUST be the same id as used previously for the same participationId . ^[62]
repositoryId	RepositoryId Id of the repository to which the client wants to re-establish a participation. MUST be the same id as used previously for the same participationId . ^[62]
participationId	ParticipationId The previously used participation id .
lastReceivedSequenceNumber	EventSequenceNumber Last event sequence number received by the <i>client</i> .
queryId	QueryId Id of this query.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Response parameters

lastSentSequenceNumber	EventSequenceNumber Last event sequence number sent by the <i>repository</i> .
queryId	QueryId Id of this query.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Errors

- [Invalid participation](#) If the participation is not valid, or the participation doesn't match with [deltaProtocolVersion](#) and/or [clientId](#) and/or [repositoryId](#).^[63]

- [Unsupported protocol version](#) for `deltaProtocolVersion`
- [Unknown repository](#) for `repositoryId`

5.6.4. Miscellaneous

5.6.4.1. Get available ids

Request `count` number of unused `valid ids`.

Same functionality as [bulk API ids command](#).

We don't assume leases, i.e. ids handed out to one client are "owned" by that client forever. We identify the client by its `clientId`.

Technical name

`GetAvailableIds`

Request parameters

count	count Number of ids requested.
queryId	QueryId Id of this query.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Response parameters

ids	FreeId[] List of ids guaranteed to be free. The repository MUST return between one (inclusive) and <code>count</code> (inclusive) ids. It MAY return less than <code>count</code> ids.
queryId	QueryId Id of this query.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Errors

- [Invalid participation](#)

5.6.4.2. List partitions

Lists all partitions^[64] accessible in the repository.^[65]

Same functionality as [bulk API listPartitions command](#).

See also [splittable message](#) and [List and subscribe partitions](#).

Technical name

`ListPartitions`

Request parameters

depthLimit	depth limit Depth of <code>partitions</code> .
queryId	QueryId Id of this query.

additionalInfos [AdditionalInfos](#) Optionally empty list of [additional infos](#).

Response parameters

partitions [shallow chunk](#) All accessible [Partitions](#) in the Repository up to [depthLimit](#).

split? [SplitFlag](#) Whether [partitions](#) is complete within this message.

queryId [QueryId](#) Id of this query.

additionalInfos [AdditionalInfos](#) Optionally empty list of [additional infos](#).

Errors

- [Invalid participation](#)
- [Invalid depth limit](#)

5.6.4.3. List and subscribe partitions

Lists all partitions^[64] accessible in the repository, and subscribes the client to all of them.^[65] The client receives the full contents of all listed partitions.^[36]

See also [splittable message](#), [subscription](#), and [List partitions](#).

Technical name

[ListAndSubscribePartitions](#)

Request parameters

queryId [QueryId](#) Id of this query.

additionalInfos [AdditionalInfos](#) Optionally empty list of [additional infos](#).

Response parameters

partitions [multi chunk](#) All accessible [Partitions](#) in the Repository.

split? [SplitFlag](#) Whether [partitions](#) is complete within this message.

queryId [QueryId](#) Id of this query.

additionalInfos [AdditionalInfos](#) Optionally empty list of [additional infos](#).

Errors

- [Invalid participation](#)

5.7. Command — event pairs overview

Colors	
added	
deleted	
moved	

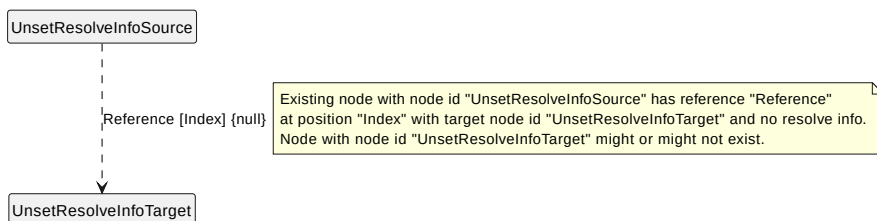
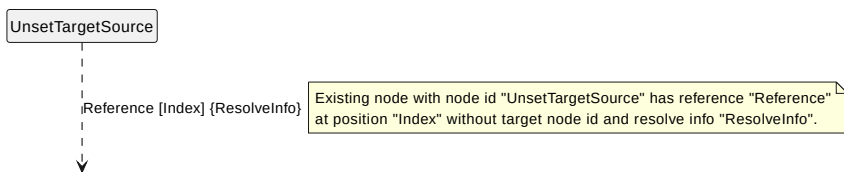
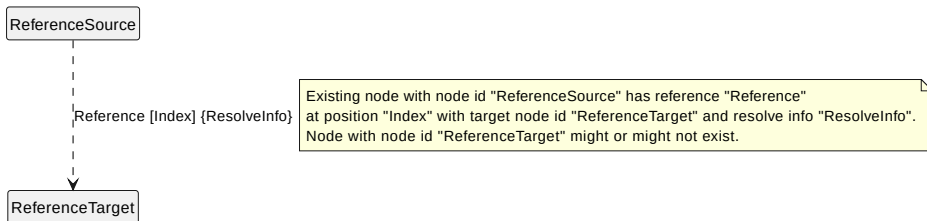
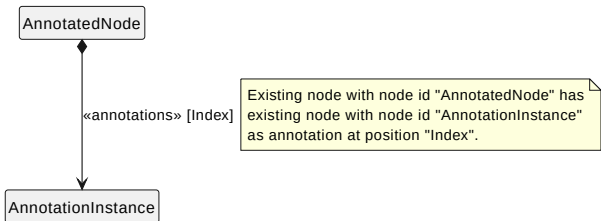
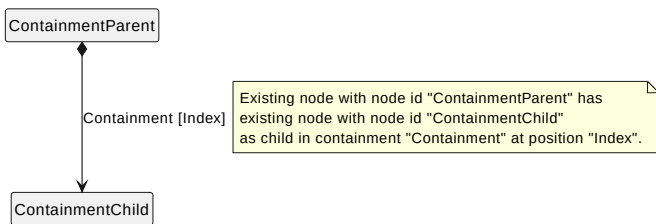
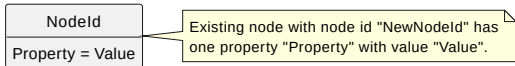
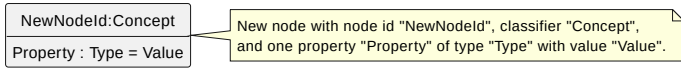
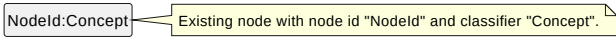
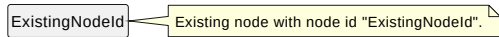


Diagram legend

5.7.1. Partitions

Command / Event / Description	Before	After
Add partition Partition added Add <code>newPartition</code> node as new partition.	<i>nothing</i>	«Partition» newPartition:Person name: String = "John" age: Integer = 23 alive: Boolean = true
Delete partition Partition deleted Delete <code>deletedPartition</code> partition.	<pre> graph TD A["«Partition» deletedPartition"] --> B[firstLevelChild] B --> C[secondLevelChild] </pre>	<i>nothing</i>

5.7.2. Nodes

Command / Event / Description	Before	After
Change classifier Classifier changed Change classifier of <code>node</code> from <code>oldClassifier</code> to <code>newClassifier</code>.	node:oldClassifier	node:newClassifier

5.7.3. Properties

Command / Event / Description	Before	After
Add property Property added Add <code>newValue</code> as value of property <code>property</code> to <code>node</code>.	node	node property = newValue

Command / Event / Description	Before	After
Delete property Property deleted Delete existing property with oldValue from node .	node property = oldValue	node
Change property Property changed Change existing property of node from oldValue to newValue .	node property = oldValue	node property = newValue

5.7.4. Children

Command / Event / Description	Before	After
Add child Child added Add new node newChild to parent in containment at index .		
Delete child Child deleted Delete existing node deletedChild from parent's containment at index .		
Replace child Child replaced Replace existing node replacedChild inside parent's containment at index with new node newChild . Delete replacedChild .		

Command / Event / Description	Before	After
Move child from containment in other parent Child moved from containment in other parent Move existing node movedChild (currently inside oldParent's oldContainment at oldIndex) inside newParent's newContainment at newIndex.		
Move child from other containment in same parent Child moved from other containment in same parent Move existing node movedChild (currently inside parent's oldContainment at oldIndex) inside parent's newContainment at newIndex.		

Command / Event / Description	Before	After
Move child in same containment in same parent Child moved in same containment in same parent Move existing node movedChild (currently inside parent's containment at oldIndex) inside parent's containment to (oldIndex + indexOffset). (Showing diagram for indexOffset positive.)		
Move child from containment in other parent and replace existing child Child moved from containment in other parent and replaced existing child Move existing node movedChild (currently inside oldParent's oldContainment at oldIndex) inside newParent's newContainment at newIndex, replacing replacedChild that was previously inside newParent's newContainment at newIndex.		

Command / Event / Description	Before	After
Move child from other containment in same parent and replace existing child Child moved from other containment in same parent and replaced existing child Move existing node movedChild (currently inside parent's oldContainment at oldIndex) inside parent's newContainment at newIndex, replacing replacedChild that was previously inside parent's newContainment at newIndex.		
Move child in same containment in same parent and replace existing child Child moved in same containment in same parent and replaced existing child Move existing node movedChild (currently inside parent's containment at oldIndex) inside parent's containment, replacing replacedChild that was previously inside parent's containment at (oldIndex + indexOffset). (Showing diagram for indexOffset negative.)		

5.7.5. Annotations

Command / Event / Description	Before	After
Add annotation Annotation added Add new node newAnnotation to parent 's annotations at index .		
Delete annotation Annotation deleted Delete existing node deletedAnnotation from parent 's annotations at index .		
Replace annotation Annotation replaced Replace existing node replacedAnnotation inside parent 's annotations at index with new node newAnnotation . Delete replacedAnnotation .		
Move annotation from other parent Annotation moved from other parent Move existing node movedAnnotation (currently inside oldParent 's annotations at oldIndex) inside newParent 's annotations at newIndex .		

Command / Event / Description	Before	After
Move annotation in same parent Annotation moved in same parent Move existing node movedAnnotation (currently inside parent's annotations at oldIndex) inside parent's annotations to (oldIndex + indexOffset). (Showing diagram for indexOffset positive.)		
Move annotation from other parent and replace existing annotation Annotation moved from other parent and replaced existing annotation Move existing node movedAnnotation (currently inside oldParent's annotations at oldIndex) inside newParent's annotations at newIndex, replacing replacedAnnotation that was previously inside newParent's annotations at newIndex.		

Command / Event / Description	Before	After
Move annotation in same parent and replace existing annotation Annotation moved in same parent and replaced existing annotation Move existing node movedAnnotation (currently inside parent's annotations at oldIndex) inside parent's annotations at newIndex, replacing replacedAnnotation that was previously inside parent's annotations at (oldIndex + indexOffset). (Showing diagram for indexOffset negative.)		

5.7.6. References

Command / Event / Description	Before	After
Add reference Reference added Add newTarget / newResolveInfo to parent's reference at index.		
Delete reference Reference deleted Delete existing deletedTarget/deletedResolveInfo from parent's reference at index.		

Command / Event / Description	Before	After
Change reference Reference changed Replace existing <code>replacedTarget/replacedResolveInfo</code> inside parent's reference at <code>index</code> with <code>newTarget/newResolveInfo</code>.		

5.8. Commands

5.8.1. Continued Command

A [continued chunk](#) for one of the following [splittable](#) commands.^[59]

- [Add partition](#) `newPartition`
- [Add child](#) `newChild`
- [Replace child](#) `newChild`
- [Add annotation](#) `parent`
- [Replace annotation](#) `newAnnotation`

We identify the splittable command by its `commandId`.

Technical name

`ContinuedCommand`

Parameters

chunk

[DeltaChunk](#) The continued chunk. The combination of all continued chunks MUST adhere to the same constraints as `commandId`'s chunk.

continuedChunkCompleted

[ContinuedChunkCompletedFlag](#) Whether this message completes `commandId`.

continuedChunkSequenceNumber

[ContinuedChunkSequenceNumber](#) Number of this message in the sequence of all continued commands relating to the same `commandId`.

commandId	CommandId Id of the related splittable command.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Errors

- [No active split message](#)
- [Duplicate continued chunk](#)
- [Unknown splittable message in continued chunk](#)
- [Invalid splittable message in continued chunk](#)
- [Interrupted continued chunk](#)
- [Invalid participation](#)

5.8.2. Partitions

5.8.2.1. Add partition

Add **newPartition** node as [new](#) partition.

The client implicitly subscribes to any changes to **newPartition** or any of its descendants.^[66]

See also [splittable message](#) and [subscription](#).

Before	After
<i>nothing</i>	«Partition» newPartition:Person name: String = "John" age: Integer = 23 alive: Boolean = true

Technical name

AddPartition

Parameters

newPartition [single chunk](#) Subtree to form a new partition. The *anchor node* MUST NOT contain a *parent* (as this is a partition, thus cannot have a parent). All nodes in that subtree MUST be [new nodes](#).

NOTE

This parameter differs from the [corresponding event parameter](#): The command's parameter contains a whole subtree, whereas the event's parameter may contain only the top of the subtree.^[36]

split? [SplitFlag](#) Whether **newPartition** is complete within this message.

- commandId** [CommandId](#) Id of this command.
- additionalInfos** [AdditionalInfos](#) Optionally empty list of [additional infos](#).

Errors

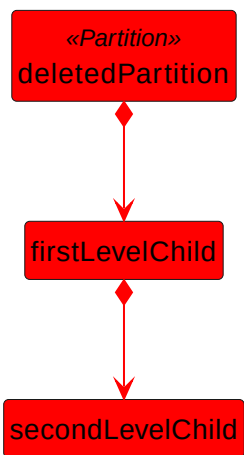
- [Invalid anchor node](#) for **newPartition**
- [Node already exists](#) for **newPartition**
- [Duplicate node id](#) for **newPartition**
- [Parent / containment mismatch](#) for **newPartition**
- [Circular containment](#) for **newPartition**
- [Invalid participation](#)

Related event

[Partition added](#)

5.8.2.2. Delete partition

Delete [existing](#) partition **deletedPartition**, and all its descendants (including annotation instances). Does NOT change references to any of the deleted nodes.^[67]

Before	After
 <pre> graph TD A["«Partition» deletedPartition"] --> B[firstLevelChild] B --> C[secondLevelChild] </pre>	<i>nothing</i>

Technical name

DeletePartition

Parameters

- deletedPartition** [NodeId](#) of the partition to delete.
- commandId** [CommandId](#) Id of this command.
- additionalInfos** [AdditionalInfos](#) Optionally empty list of [additional infos](#).

Errors

- [Unknown node](#) for **deletedPartition**
- [Invalid participation](#)

Related event

[Partition deleted](#)

5.8.3. Nodes

5.8.3.1. Change classifier

Change classifier of [existing node](#) `node` to `newClassifier`.

Before	After
node:<code>oldClassifier</code>	node:<code>newClassifier</code>

Technical name

`ChangeClassifier`

Parameters

- node** [NodeId](#) of the node we want to change the classifier.
- newClassifier** [MetaPointer](#) New [Classifier](#) of `node`.
- commandId** [CommandId](#) Id of this command.
- additionalInfos** [AdditionalInfos](#) Optionally empty list of [additional infos](#).

Errors

- [Unknown node](#) for `node`
- [Invalid participation](#)

Related event

[Classifier changed](#)

5.8.4. Properties

5.8.4.1. Add property

Add `newValue` as value of property `property` to `node`.

Before	After
node	node <code>property = newValue</code>

Technical name

`AddProperty`

Parameters

- node** [NodeId](#) of the node we want to add a property to.
- property** [MetaPointer Property](#) we want to add to **node**.
- newValue** [PropertyValue](#) Value we want to set **property** in **node** to.
- commandId** [CommandId](#) Id of this command.
- additionalInfos** [AdditionalInfos](#) Optionally empty list of [additional infos](#).

Errors

- [Unknown node](#) for **node**
- [Unknown property](#) for **property** in **node**
- [Property already set](#) for **property** in **node**
- [Invalid participation](#)

Related event

[Property added](#)

5.8.4.2. Delete property

Delete existing **property** from **node**.

Before	After
node property = oldValue	node

Technical name

[DeleteProperty](#)

Parameters

- node** [NodeId](#) of the node we want to delete a property from.
- property** [MetaPointer Property](#) we want to delete from **node**.
- commandId** [CommandId](#) Id of this command.
- additionalInfos** [AdditionalInfos](#) Optionally empty list of [additional infos](#).

Errors

- [Unknown node](#) for **node**
- [Unknown property](#) for **property** in **node**
- [Property not set](#) for **property** in **node**
- [Invalid participation](#)

Related event

Property deleted

5.8.4.3. Change property

Change existing **property** of **node** to **newValue**.

Before	After
node property = oldValue	node property = newValue

Technical name

ChangeProperty

Parameters

- node** [NodeId](#) of the node we want to change a property of.
- property** [MetaPointer Property](#) we want to change in **node**.
- newValue** [PropertyValue](#) Value we want to change **property** in **node** to.
- commandId** [CommandId](#) Id of this command.
- additionalInfos** [AdditionalInfos](#) Optionally empty list of [additional infos](#).

Errors

- [Unknown node](#) for **node**
- [Unknown property](#) for **property** in **node**
- [Property not set](#) for **property** in **node**
- [Invalid participation](#)

Related event

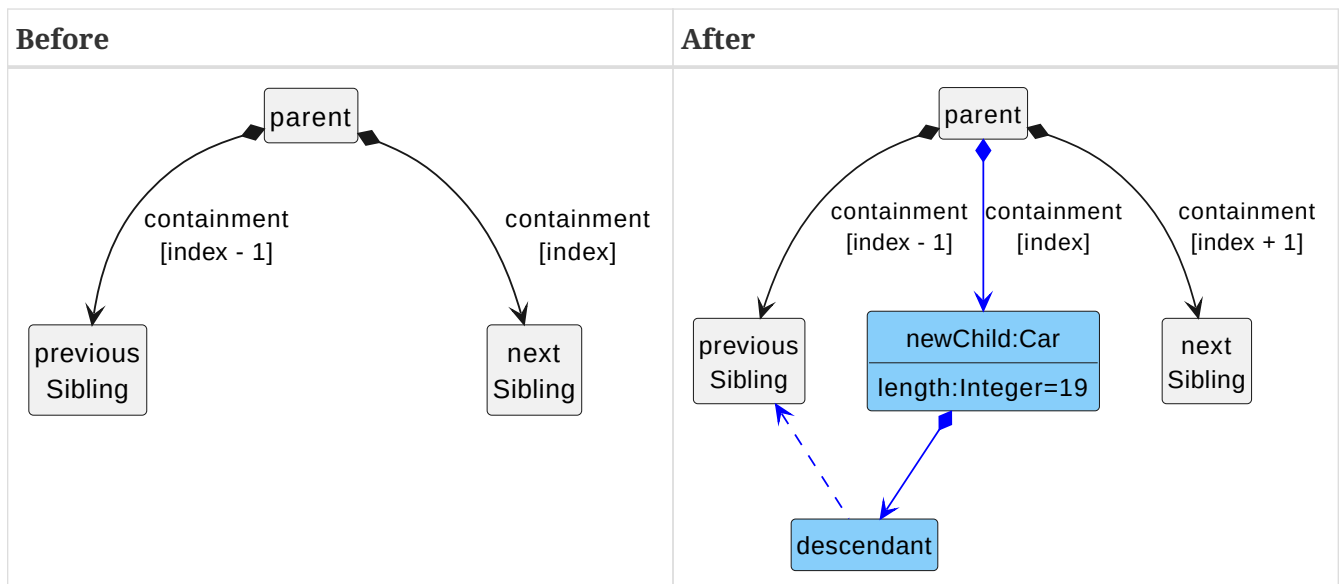
[Property changed](#)

5.8.5. Children

5.8.5.1. Add child

Add [new node](#) **newChild** to **parent** in **containment** at **index**.

See also [splittable message](#).



Technical name

AddChild

Parameters

parent	NodeId of the node we want to add a new child to.
newChild	single chunk Subtree to form the new child. The <i>anchor node</i> MUST contain the parent as <i>parent</i> . ^[68] All nodes in that subtree MUST be new nodes .
containment	MetaPointer Containment we want to add newChild to in parent .
index	Index where we want to add newChild at in containment of parent .
split?	SplitFlag Whether newChild is complete within this message.
commandId	CommandId Id of this command.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Errors

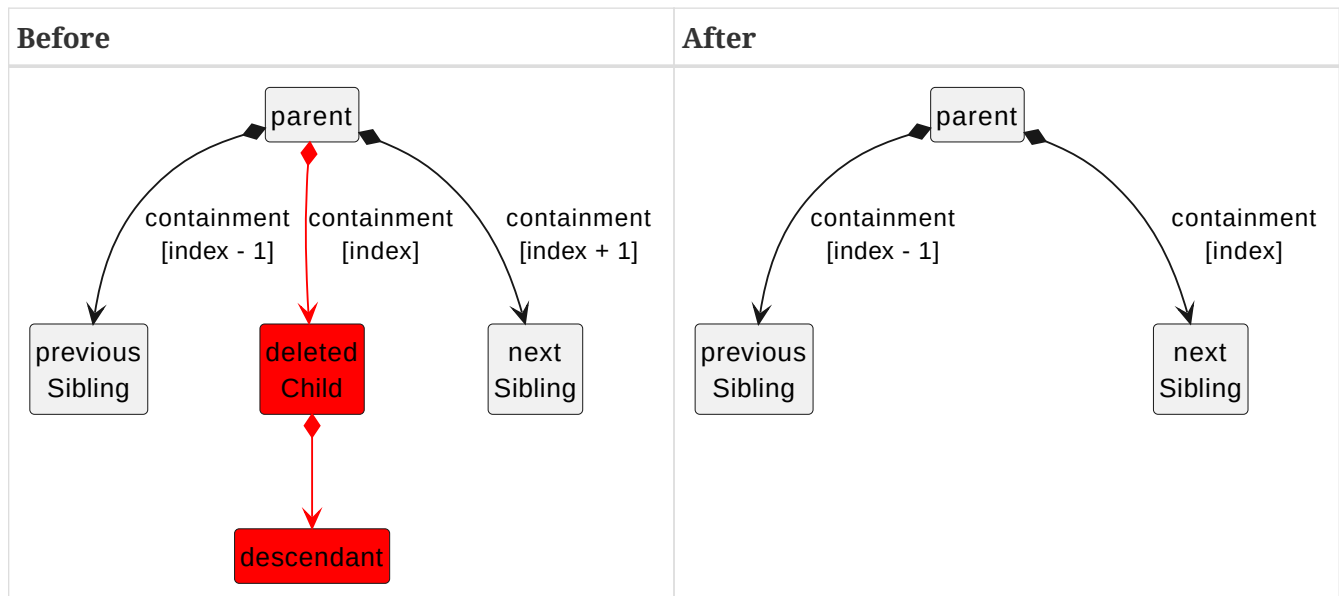
- [Unknown node](#) for [parent](#)
- [Invalid anchor node](#) for [newChild](#)
- [Node already exists](#) for [newChild](#)
- [Duplicate node id](#) for [newChild](#)
- [Parent / containment mismatch](#) for [newChild](#)
- [Circular containment](#) for [newChild](#)
- [Index out of range](#) for [index](#) (is beyond the number of [containment](#) entries in [parent](#))
- [Invalid participation](#)

Related event

[Child added](#)

5.8.5.2. Delete child

Delete **existing node** `deletedChild`^[69] from **parent's** **containment** at **index**, and all its descendants (including annotation instances). Does NOT change references to any of the deleted nodes.^[67]



Technical name

DeleteChild

Parameters

parent	NodeId of the node we want to delete a child from.
containment	MetaPointer Containment we want to delete deletedChild from in parent .
index	Index where we want to delete deletedChild from in containment of parent .
deletedChild	NodeId of the node we want to delete in containment of parent at index .
commandId	CommandId Id of this command.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Errors

- **Unknown node** for **parent**
- **Unknown node** for **deletedChild**
- **Unknown containment** for **containment** in **parent**
- **The given parent of a node is not its parent** for **parent** and **deletedChild**
- **Index out of range** for **index** (is beyond (the number of children in **containment** of **parent**) - 1)
- **Child at index is not the given node** for **containment/index** and **deletedChild**
- **Invalid participation**

Related event

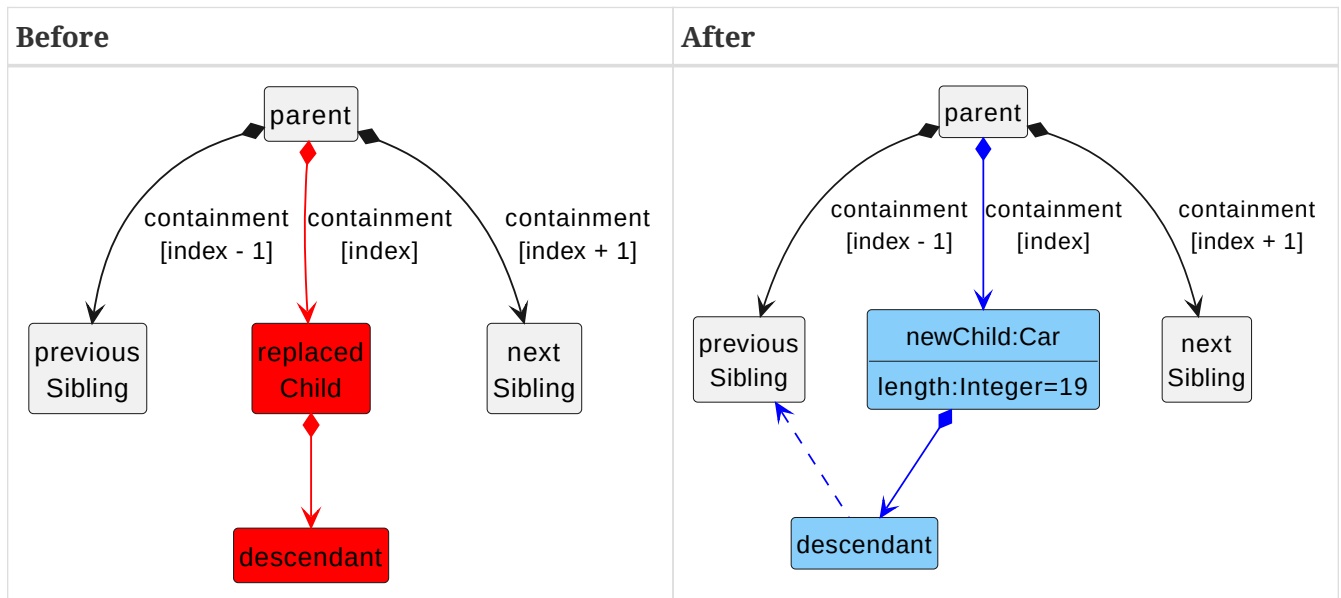
Child deleted

5.8.5.3. Replace child

Delete **current child** `replacedChild`^[69] inside **parent's** `containment` at `index`, and all its descendants (including annotation instances). Does NOT change references to any of the deleted nodes.^[67]

Replace **existing node** `replacedChild` inside **parent's** `containment` at `index` with **new** or **reused** node `newChild`.

See also [splittable message](#).



Technical name

ReplaceChild

Parameters

newChild	single chunk Subtree to form the new child that replaces replacedChild . The <i>anchor node</i> MUST contain the parent as <i>parent</i> . ^[68] All nodes in that subtree MUST either be new nodes or reused nodes .
parent	NodeId of the node we want to replace a child in.
containment	MetaPointer Containment we want to replace replacedChild in parent .
index	Index where we want to replace replacedChild at in <code>containment</code> of parent .
replacedChild	NodeId of the node we want to replace in <code>containment</code> of parent at <code>index</code> .
split?	SplitFlag Whether newChild is complete within this message.
commandId	CommandId Id of this command.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Errors

- **Unknown node** for **parent**
- **Unknown node** for **replacedChild**

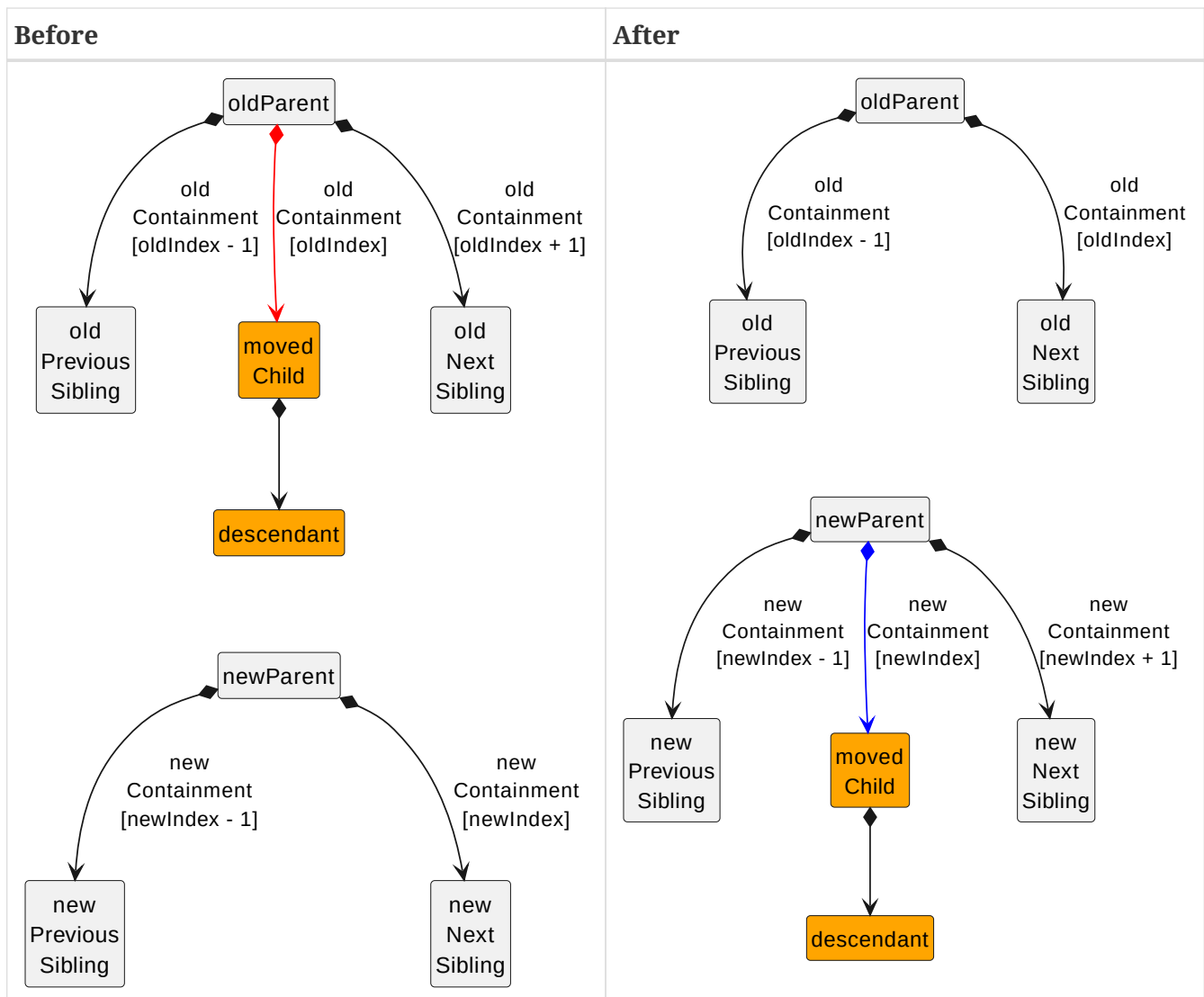
- Unknown containment for containment in parent
- The given parent of a node is not its parent for parent and replacedChild
- Invalid anchor node for newChild
- Node already exists for newChild
- Duplicate node id for newChild
- Parent / containment mismatch for newChild
- Circular containment for newChild
- Index out of range for index (is beyond (the number of children in containment of parent) - 1)
- Child at index is not the given node for containment/index and replacedChild
- Invalid participation

Related event

Child replaced

5.8.5.4. Move child from containment in other parent

Move existing node movedChild from oldContainment at oldIndex in oldParent to newParent's newContainment at newIndex.



Technical name

MoveChildFromContainmentInOtherParent^[70]

Parameters

newParent	NodeId of the node we want to move a child to.
newContainment	MetaPointer Containment we want to move movedChild to in newParent .
newIndex	Index where we want to move movedChild at in newContainment of newParent .
oldParent	NodeId of the node we want to move an existing child from.
oldContainment	MetaPointer Containment we want to move movedChild from in oldParent .
oldIndex	Index where we want to move movedChild from in oldContainment of oldParent .
movedChild	NodeId of the node we want to move from oldParent to newParent .
commandId	CommandId Id of this command.

additionalInfos **AdditionalInfos** Optionally empty list of **additional infos**.

Errors

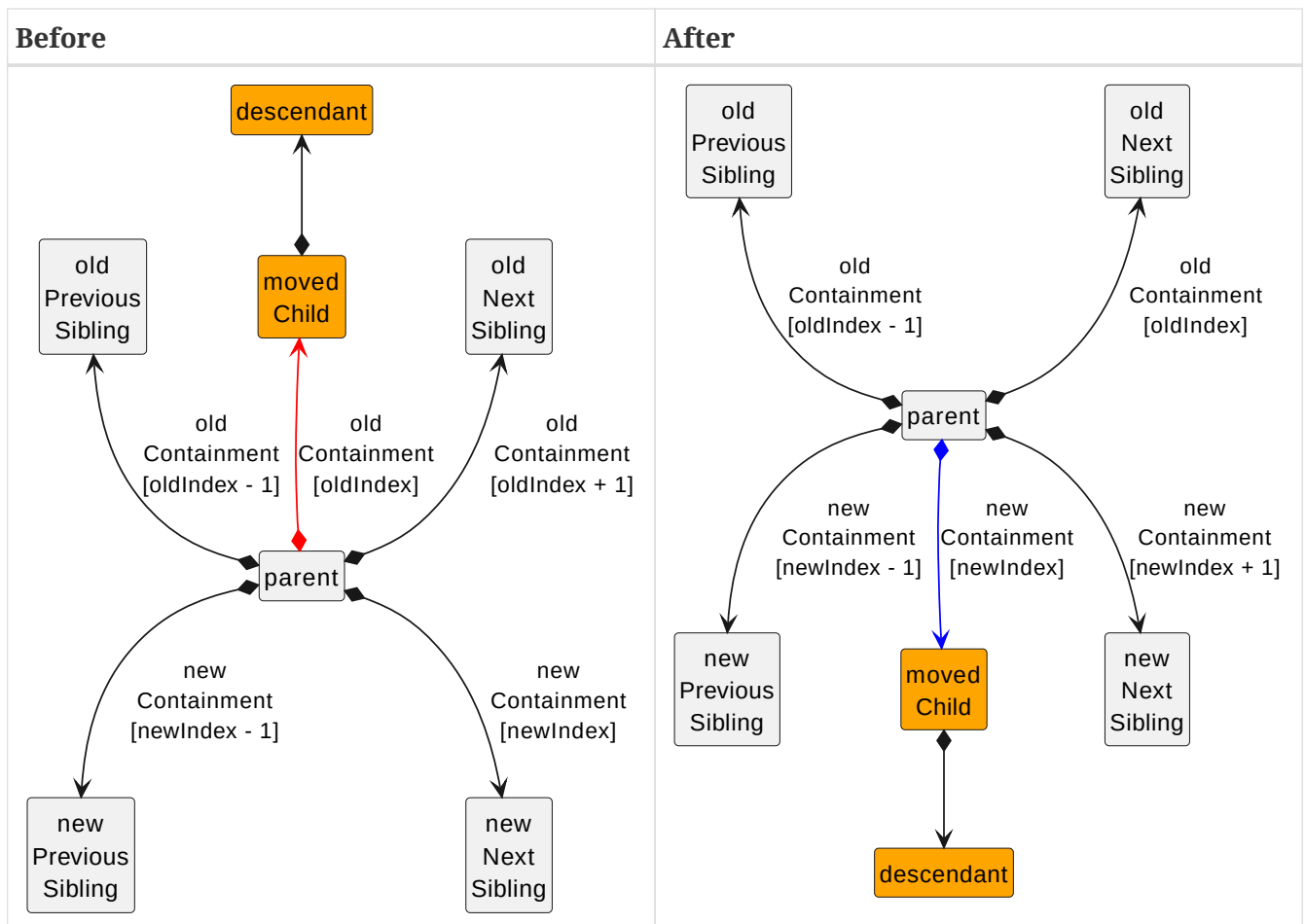
- **Unknown node** for **oldParent**
- **Unknown node** for **newParent**
- **Unknown node** for **movedChild**
- **Unknown containment** for **oldContainment** in **oldParent**
- **Unknown containment** for **newContainment** in **newParent**
- **The given parent of a node is not its parent** for **oldParent** and **movedChild**
- **Index out of range** for **newIndex** (is beyond the number of **newContainment** entries in **newParent**)
- **Index out of range** for **oldIndex** (is beyond the number of **oldContainment** entries in **oldParent**)
- **Child at index is not the given node** for **oldContainment/oldIndex** and **movedChild**
- **Old and new parent identical** for **oldParent** and **newParent**
- **Old and new containment identical** for **oldContainment** and **newContainment**
- **Old and new index identical** for **oldIndex** and **newIndex**
- **Invalid participation**

Related event

Child moved from containment in other parent

5.8.5.5. Move child from other containment in same parent

Move **existing node** **movedChild** from **oldContainment** at **oldIndex** to **newContainment** at **newIndex**. Both containments are inside **parent**.



Technical name

MoveChildFromOtherContainmentInSameParent

Parameters

parent	NodeId of the node we want to move children in.
newContainment	MetaPointer Containment we want to move movedChild to in parent .
newIndex	Index where we want to move movedChild at in newContainment of parent .
oldContainment	MetaPointer Containment we want to move movedChild from in parent .
oldIndex	Index where we want to move movedChild from in oldContainment of parent .
movedChild	NodeId of the node we want to move from oldContainment to newContainment .
commandId	CommandId Id of this command.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Errors

- **Unknown node** for **parent**
- **Unknown node** for **movedChild**

- Unknown containment for `oldContainment` in `parent`
- Unknown containment for `newContainment` in `parent`
- The given parent of a node is not its parent for `parent` and `movedChild`
- Index out of range for `newIndex` (is beyond the number of `newContainment` entries in `parent`)
- Index out of range for `oldIndex` (is beyond the number of `oldContainment` entries in `parent`)
- Child at index is not the given node for `oldContainment/oldIndex` and `movedChild`
- Old and new containment identical for `oldContainment` and `newContainment`
- Old and new index identical for `oldIndex` and `newIndex`
- Invalid participation

Related event

Child moved from other containment in same parent

5.8.5.6. Move child in same containment in same parent

Move existing node `movedChild` that's initially at index `oldIndex` within the `containment` inside `parent` by `indexOffset` positions to index (`oldIndex + indexOffset`).

This conforms to the action of moving a node in one specific list — see that section for more information, diagrams, and examples.

Technical name

`MoveChildInSameContainmentInSameParent`^[71]

Parameters

parent	<code>NodeId</code> of the node we want to move a child in.
containment	<code>MetaPointer Containment</code> we want to move <code>movedChild</code> in <code>parent</code> .
oldIndex	<code>Index</code> where we want to move <code>movedChild</code> from in <code>containment</code> of <code>parent</code> .
indexOffset	<code>IndexOffset</code> how much we want to move <code>movedChild</code> in <code>containment</code> of <code>parent</code> .
movedChild	<code>NodeId</code> of the node we want to move in <code>containment</code> .
commandId	<code>CommandId</code> Id of this command.
additionalInfos	<code>AdditionalInfos</code> Optionally empty list of <code>additional infos</code> .

Errors

- Unknown node for `parent`
- Unknown node for `movedChild`
- Unknown containment for `containment` in `parent`
- The given parent of a node is not its parent for `parent` and `movedChild`

- **Index out of range** for `oldIndex` is not an integer in the range $[0..(\text{the number of containment entries in } \text{containment of } \text{parent} - 1)]$
- **Index offset is invalid** for `indexOffset` if `indexOffset` equals 0, or if $(\text{oldIndex} + \text{indexOffset})$ is not in the range $[0..(\text{the number of containment entries in } \text{containment of } \text{parent} - 1)]$
- **Child at index is not the given node** for `containment/oldIndex` and `movedChild`
- **Invalid participation**

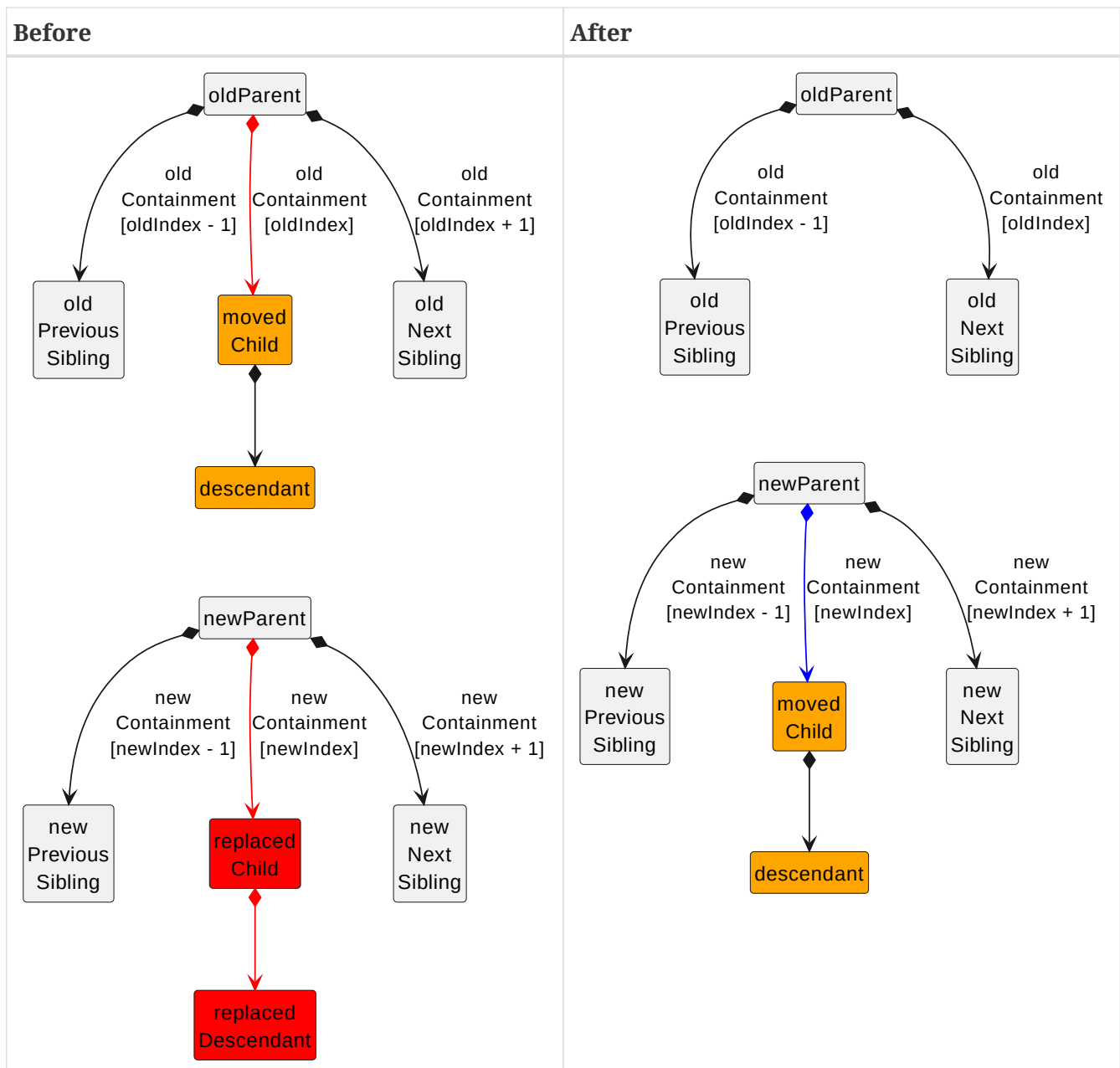
Related event

Child moved in same containment in same parent

5.8.5.7. Move child from containment in other parent and replace existing child

Move **existing node** `movedChild` from `oldIndex` inside `oldParent's oldContainment` to `newParent's newContainment` at `newIndex`.

Delete **current child** `replacedChild`^[69] inside `newParent's newContainment` at `newIndex`, and all its descendants (including annotation instances). Does NOT change references to any of the deleted nodes.^[67]



Technical name

MoveAndReplaceChildFromContainmentInOtherParent^[70]

Parameters

- newParent** [NodeId](#) of the node we want to move a child to.
- newContainment** [MetaPointer Containment](#) we want to move **movedChild** to in **newParent**.
- newIndex** [Index](#) where we want to move **movedChild** to in **newContainment** of **newParent**.
- oldParent** [NodeId](#) of the node we want to move a child from.
- oldContainment** [MetaPointer Containment](#) we want to move **movedChild** from in **oldParent**.
- oldIndex** [Index](#) where we want to move **movedChild** from in **oldContainment** of **oldParent**.

replacedChild	NodeId of the node we want to replace at newIndex of newContainment in newParent .
movedChild	NodeId of the node we want to move from oldContainment in oldParent to newContainment in newParent .
commandId	CommandId Id of this command.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Errors

- [Unknown node](#) for [newParent](#)
- [Unknown node](#) for [oldParent](#)
- [Unknown node](#) for [movedChild](#)
- [Unknown node](#) for [replacedChild](#)
- [Unknown containment](#) for [oldContainment](#) in [oldParent](#)
- [Unknown containment](#) for [newContainment](#) in [newParent](#)
- The given parent of a node is not its parent for [oldParent](#) and [movedChild](#)
- The given parent of a node is not its parent for [newParent](#) and [replacedChild](#)
- [Index out of range](#) for [oldIndex](#) (is beyond the number of [oldIndex](#) entries in [oldParent](#))
- [Index out of range](#) for [newIndex](#) (is beyond the number of [newContainment](#) entries in [newParent](#))
- [Child at index is not the given node](#) for [oldContainment/oldIndex](#) and [movedChild](#)
- [Child at index is not the given node](#) for [newContainment/newIndex](#) and [replacedChild](#)
- [Old and new parent identical](#) for [oldParent](#) and [newParent](#)
- [Old and new containment identical](#) for [oldContainment](#) and [newContainment](#)
- [Old and new index identical](#) for [oldIndex](#) and [newIndex](#)
- [Invalid participation](#)

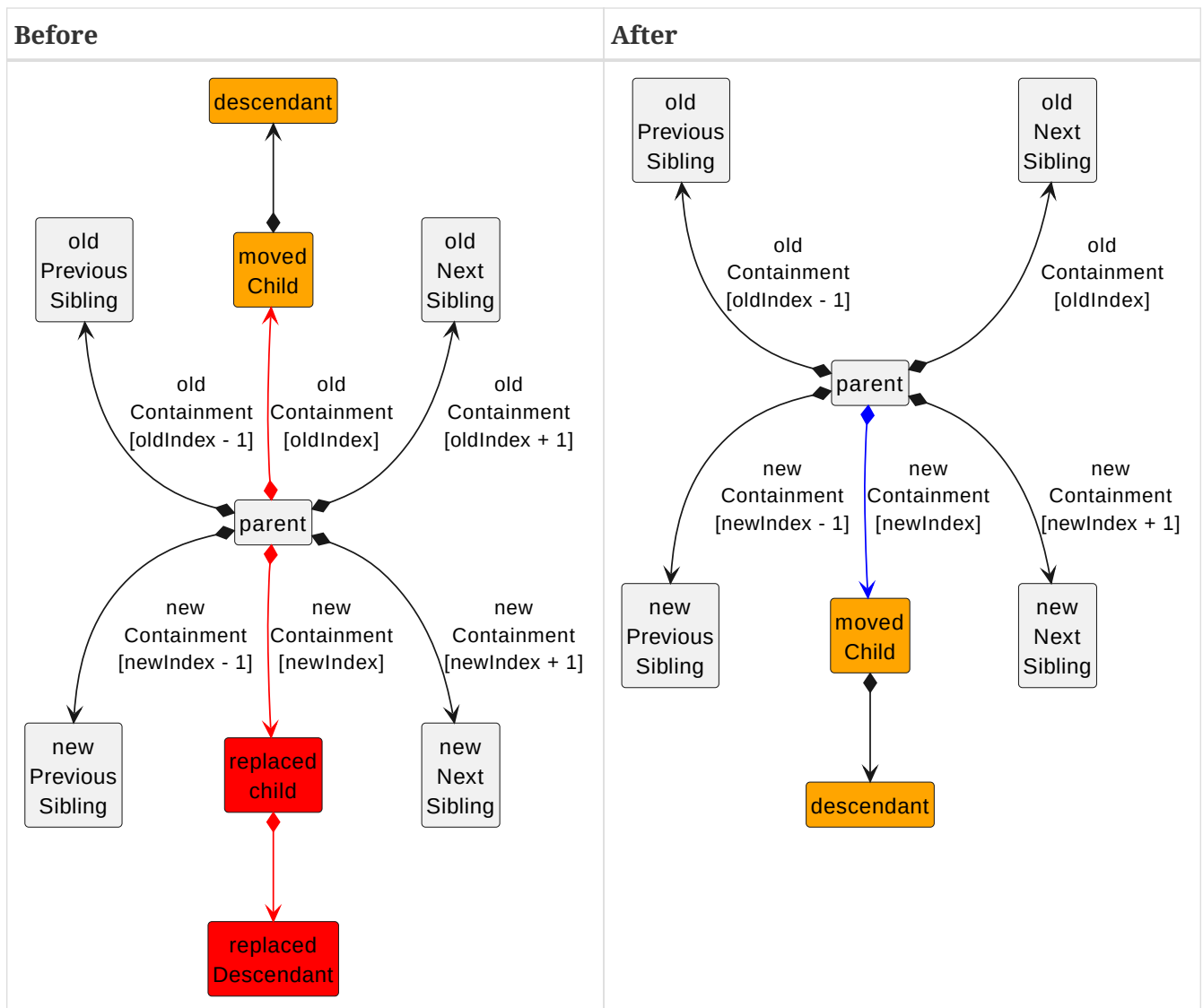
Related event

[Child moved from containment in other parent and replaced existing child](#)

5.8.5.8. Move child from other containment in same parent and replace existing child

Move [existing node](#) [movedChild](#) from [oldContainment](#) at [oldIndex](#) to [newContainment](#) at [newIndex](#). Both containments are in the same [parent](#).

Delete [current child](#) [replacedChild](#)^[69] inside [parent's newContainment](#) at [newIndex](#), and all its descendants (including annotation instances). Does NOT change references to any of the deleted nodes.^[67]



Technical name

MoveAndReplaceChildFromOtherContainmentInSameParent

Parameters

parent	NodeId of the node we want to move children in.
newContainment	MetaPointer Containment we want to move movedChild to in parent .
newIndex	Index where we want to move movedChild to in newContainment of parent .
oldContainment	MetaPointer Containment we want to move movedChild from in parent .
oldIndex	Index where we want to move movedChild from in oldContainment of parent .
replacedChild	NodeId of the node we want to replace at newIndex of newContainment in parent .
movedChild	NodeId of the node we want to move from oldContainment to newContainment in parent .
commandId	CommandId Id of this command.

additionalInfos **AdditionalInfos** Optionally empty list of **additional infos**.

Errors

- **Unknown node** for **parent**
- **Unknown node** for **movedChild**
- **Unknown node** for **replacedChild**
- **Unknown containment** for **oldContainment** in **parent**
- **Unknown containment** for **newContainment** in **parent**
- **The given parent of a node is not its parent** for **parent** and **movedChild**
- **The given parent of a node is not its parent** for **parent** and **replacedChild**
- **Index out of range** for **newIndex** (is beyond the number of **newContainment** entries in **movedChild's** parent)
- **Index out of range** for **oldIndex** (is beyond the number of **oldContainment** entries in **replacedChild's** parent)
- **Child at index is not the given node** for **oldContainment/oldIndex** and **movedChild**
- **Child at index is not the given node** for **newContainment/newIndex** and **replacedChild**
- **Old and new containment identical** for **oldContainment** and **newContainment**
- **Old and new index identical** for **oldIndex** and **newIndex**
- **Invalid participation**

Related event

Child moved from other containment in same parent and replaced existing child

5.8.5.9. Move child in same containment in same parent and replace existing child

Move **existing node** **movedChild** that's initially at index **oldIndex** within the **containment** inside **parent** to the target index (**oldIndex** + **indexOffset**), replacing **current child** **replacedChild**^[69] at the target index.^[72] **replacedChild** subsequently gets deleted, including all its descendants (including annotation instances). This does NOT change references to any of the deleted nodes.^[67]

This conforms to **the action of moving a node and replacing another one in one specific list** — see that section for more information, diagrams, and examples.

Technical name

MoveAndReplaceChildInSameContainmentInSameParent^[71]

Parameters

parent	NodeId of the node we want to move a child in.
containment	MetaPointer Containment we want to move movedChild and replace replacedChild in parent .
oldIndex	Index where we want to move movedChild from in containment of parent .

indexOffset	IndexOffset how much we want to move movedChild in containment of parent .
movedChild	NodeId of the node we want to move in containment in parent .
replacedChild	NodeId of the node we want to replace in containment in parent .
commandId	CommandId Id of this command.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Errors

- [Unknown node](#) for [parent](#)
- [Unknown node](#) for [movedChild](#)
- [Unknown node](#) for [replacedChild](#)
- [Unknown containment](#) for [containment](#) in [parent](#)
- The given [parent](#) of a node is not its [parent](#) for [parent](#) and [movedChild](#)
- The given [parent](#) of a node is not its [parent](#) for [parent](#) and [replacedChild](#)
- [Index out of range](#) for [oldIndex](#) is not an integer in the range [0..(the number of containment entries in [containment](#) of [parent](#) - 1)]
- [Index offset is invalid](#) for [indexOffset](#) if [indexOffset](#) equals 0, or if ([oldIndex](#) + [indexOffset](#)) is not in the range [0..(the number of containment entries in [containment](#) of [parent](#) - 1)]
- [Child at index is not the given node](#) for [oldIndex](#) and [movedChild](#) if [movedChild](#) is not at index [oldIndex](#) of the containment entries in [containment](#) of [parent](#)
- [Child at index is not the given node](#) for [containment/\(oldIndex + indexOffset\)](#) and [replacedChild](#) if [replacedChild](#) is not at index ([oldIndex](#) + [indexOffset](#)) of the containment entries in [containment](#) of [parent](#)
- [Invalid participation](#)

Related event

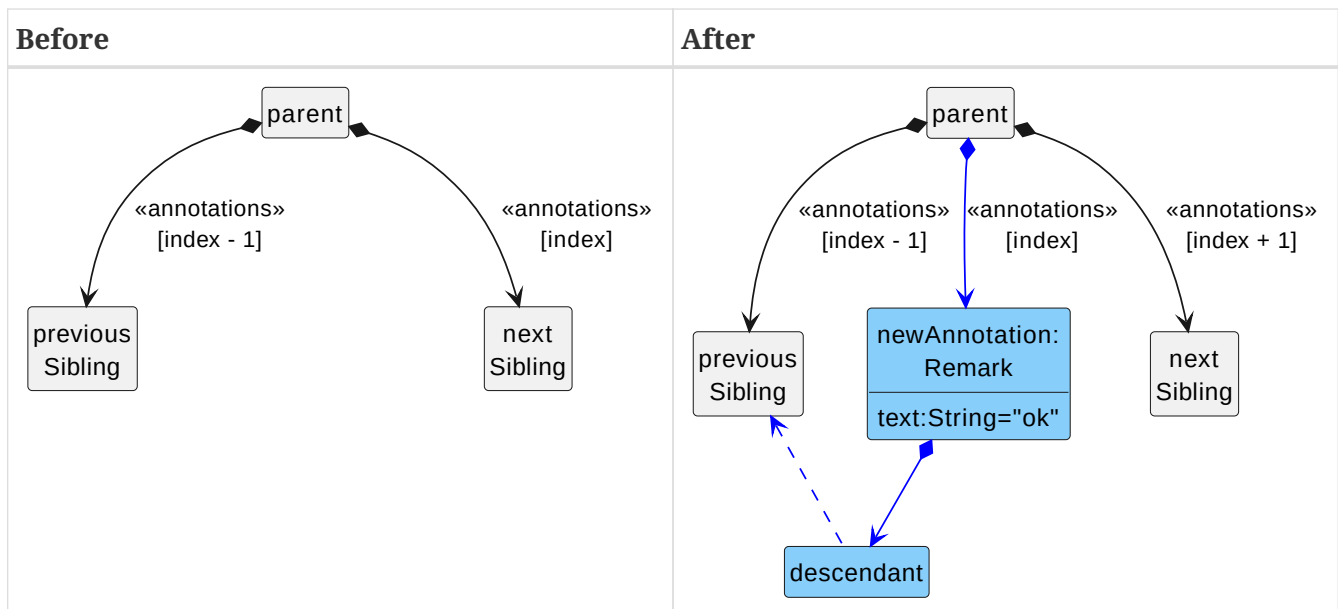
[Child moved in same containment in same parent and replaced existing child](#)

5.8.6. Annotations

5.8.6.1. Add annotation

Add [new node](#) [newAnnotation](#) to [parent](#)'s annotations at [index](#).

See also [splittable message](#).



Technical name

AddAnnotation

Parameters

parent	NodeId of the node we want to add a new annotation to.
newAnnotation	single chunk Subtree to form a new annotation. The <i>anchor node</i> MUST contain the parent as <i>parent</i> . ^[68] All nodes in that subtree MUST be new nodes .
index	Index where we want to add newAnnotation at annotations of parent .
split?	SplitFlag Whether newAnnotation is complete within this message.
commandId	CommandId Id of this command.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Errors

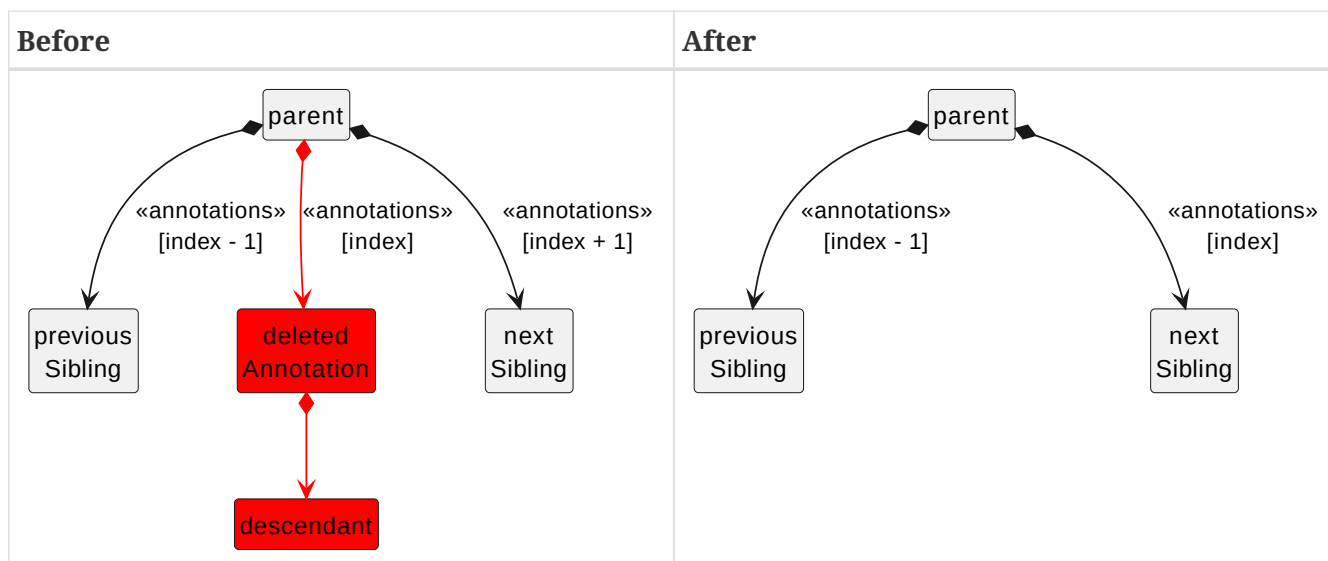
- [Unknown node](#) for [parent](#)
- [Invalid anchor node](#) for [newAnnotation](#)
- [Node already exists](#) for [newAnnotation](#)
- [Duplicate node id](#) for [newAnnotation](#)
- [Parent / containment mismatch](#) for [newAnnotation](#)
- [Circular containment](#) for [newAnnotation](#)
- [Index out of range](#) for [index](#) (is beyond the number of annotations of [parent](#))
- [Invalid participation](#)

Related event

[Annotation added](#)

5.8.6.2. Delete annotation

Delete **existing node** `deletedAnnotation`^[69] from `parent`'s annotations at `index`, and all its descendants (including annotation instances). Does NOT change references to any of the deleted nodes.^[67]



Technical name

`DeleteAnnotation`

Parameters

parent	<code>NodeId</code> of the node we want to delete an annotation from.
index	<code>Index</code> where we want to delete <code>deletedAnnotation</code> from in annotations of <code>parent</code> .
deletedAnnotation	<code>NodeId</code> of the node we want to delete in annotations of <code>parent</code> at <code>index</code> .
commandId	<code>CommandId</code> Id of this command.
additionalInfos	<code>AdditionalInfos</code> Optionally empty list of <code>additional infos</code> .

Errors

- `Unknown node` for `parent`
- `Unknown node` for `deletedAnnotation`
- `The given parent of a node is not its parent` for `parent` and `deletedAnnotation`
- `Index out of range` for `index` (is beyond (the number of annotations of `parent`) - 1)
- `Annotation at index is not the given annotation` for `index` and `deletedAnnotation`
- `Invalid participation`

Related event

`Annotation deleted`

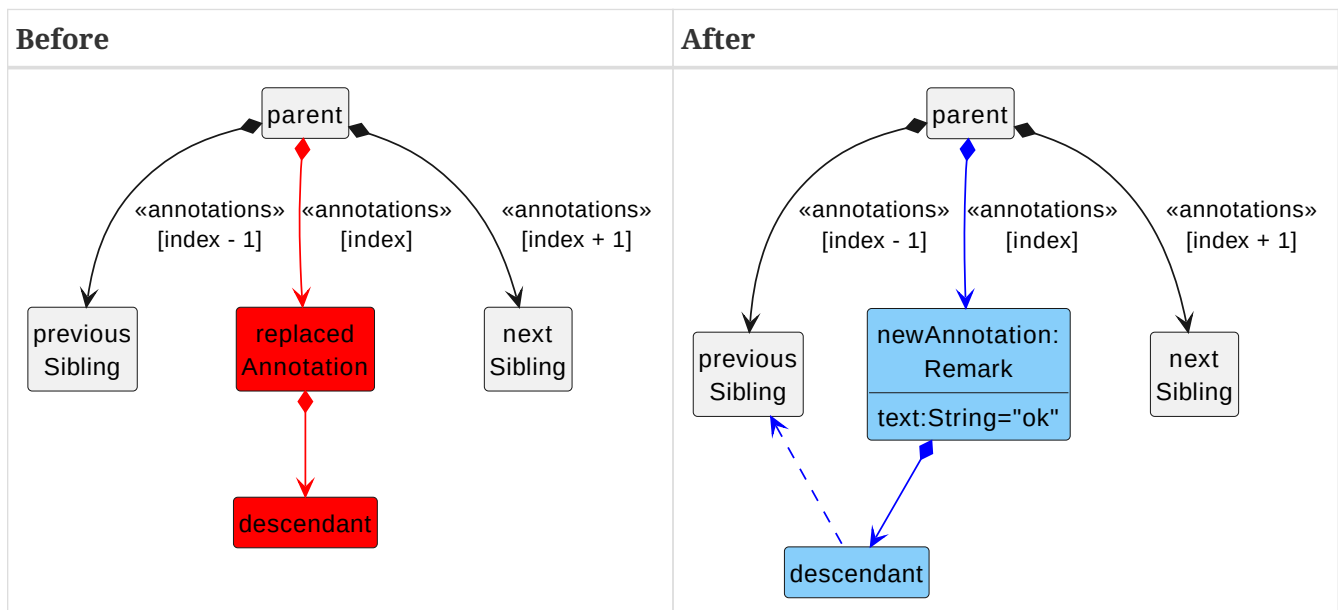
5.8.6.3. Replace annotation

Delete **current node** `replacedAnnotation`^[69] at `parent`'s annotations at `index`, and all its descendants

(including annotation instances). Does NOT change references to any of the deleted nodes.^[67]

Replace **existing node** **replacedAnnotation** inside **parent**'s annotations at **index** with **new** or **reused** node **newAnnotation**.

See also [splittable message](#).



Technical name

ReplaceAnnotation

Parameters

newAnnotation [single chunk](#) Subtree to form a new annotation that replaces **replacedAnnotation**. The *anchor node* MUST contain the **parent** as *parent*.^[68] All nodes in that subtree MUST either be **new nodes** or **reused nodes**.

parent [NodeId](#) of the node we want to replace an annotation in.

index [Index](#) where we want to replace **replacedAnnotation** at in annotations of **parent**.

replacedAnnotation [NodeId](#) of the node we want to replace in annotations of **parent** at **index**.

split? [SplitFlag](#) Whether **newAnnotation** is complete within this message.

commandId [CommandId](#) Id of this command.

additionalInfos [AdditionalInfos](#) Optionally empty list of [additional infos](#).

Errors

- [Unknown node](#) for **parent**
- [Unknown node](#) for **replacedAnnotation**
- The given parent of a node is not its parent for **parent** and **replacedAnnotation**

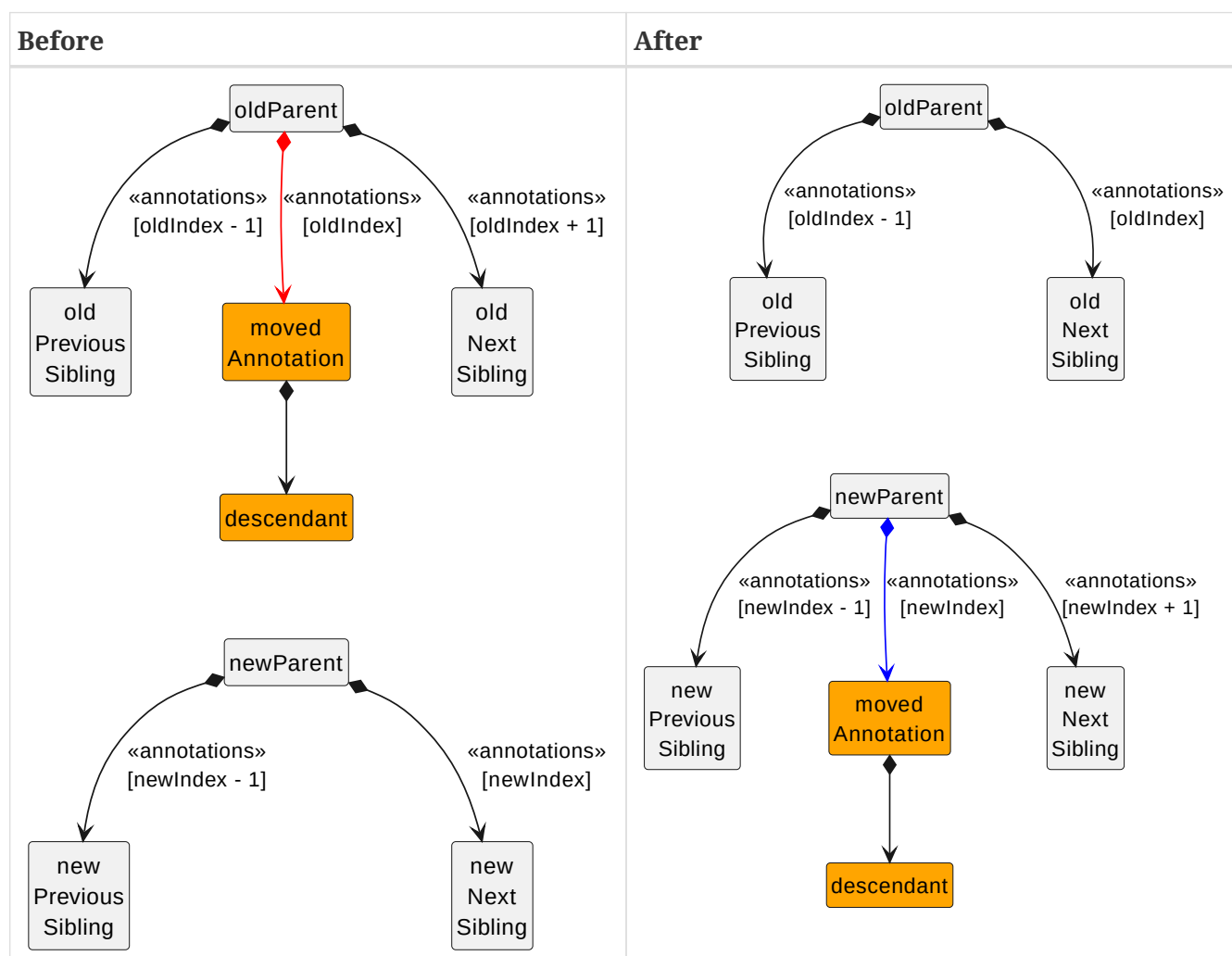
- Invalid anchor node for newAnnotation
- Node already exists for newAnnotation
- Duplicate node id for newAnnotation
- Parent / containment mismatch for newAnnotation
- Circular containment for newAnnotation
- Index out of range for index (is beyond (the number of annotations of parent) - 1)
- Annotation at index is not the given annotation for index and replacedAnnotation
- Invalid participation

Related event

Annotation replaced

5.8.6.4. Move annotation from other parent

Move existing node movedAnnotation from oldIndex in oldParent's annotations to newParent's annotations at newIndex.



Technical name

MoveAnnotationFromOtherParent

Parameters

newParent	NodeId of the node we want to move the annotation to.
newIndex	Index where we want to move movedAnnotation at in annotations of newParent .
oldParent	NodeId of the node we want to move an existing annotation from.
oldIndex	Index where we want to move movedAnnotation from in annotations of oldParent .
movedAnnotation	NodeId of the annotation we want to move from oldParent to newParent .
commandId	CommandId Id of this command.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Errors

- [Unknown node](#) for [oldParent](#)
- [Unknown node](#) for [newParent](#)
- [Unknown node](#) for [movedAnnotation](#)
- The given parent of a node is not its parent [oldParent](#) and [movedAnnotation](#)
- [Index out of range](#) for [newIndex](#) (is beyond the number of annotations of [newParent](#))
- [Index out of range](#) for [oldIndex](#) (is beyond the number of annotations of [oldParent](#))
- [Annotation at index is not the given annotation](#) for [oldIndex](#) and [movedAnnotation](#)
- Old and new parent identical for [oldParent](#) and [newParent](#)
- Old and new index identical for [oldIndex](#) and [newIndex](#)
- [Invalid participation](#)

Related event

[Annotation moved from other parent](#)

5.8.6.5. Move annotation in same parent

Move [existing node](#) [movedAnnotation](#) that's initially at index [oldIndex](#) in [parent](#)'s annotations by [indexOffset](#) positions to index ([oldIndex](#) + [indexOffset](#)).

This conforms to [the action of moving a node in one specific list](#) — see that section for more information, diagrams, and examples.

Technical name

[MoveAnnotationInSameParent](#)

Parameters

parent	NodeId of the node we want to move an annotation in.
oldIndex	NodeId where we want to move movedAnnotation from in annotations of parent .

indexOffset	IndexOffset how much we want to move movedAnnotation in annotations of parent .
movedAnnotation	NodeId of the annotation we want to move.
commandId	CommandId Id of this command.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Errors

- **Unknown node** for **parent**
- **Unknown node** for **movedAnnotation**
- **The given parent of a node is not its parent** for **parent** and **movedAnnotation**
- **Index out of range** for **oldIndex** is not in the range $[0..(\text{the number of annotations on } \text{parent} - 1)]$
- **Index offset is invalid** for **indexOffset** if **indexOffset** equals 0, or if $(\text{oldIndex} + \text{indexOffset})$ is not in the range $[0..(\text{the number of annotations on } \text{parent} - 1)]$
- **Annotation at index is not the given annotation** for **oldIndex** and **movedAnnotation**
- **Invalid participation**

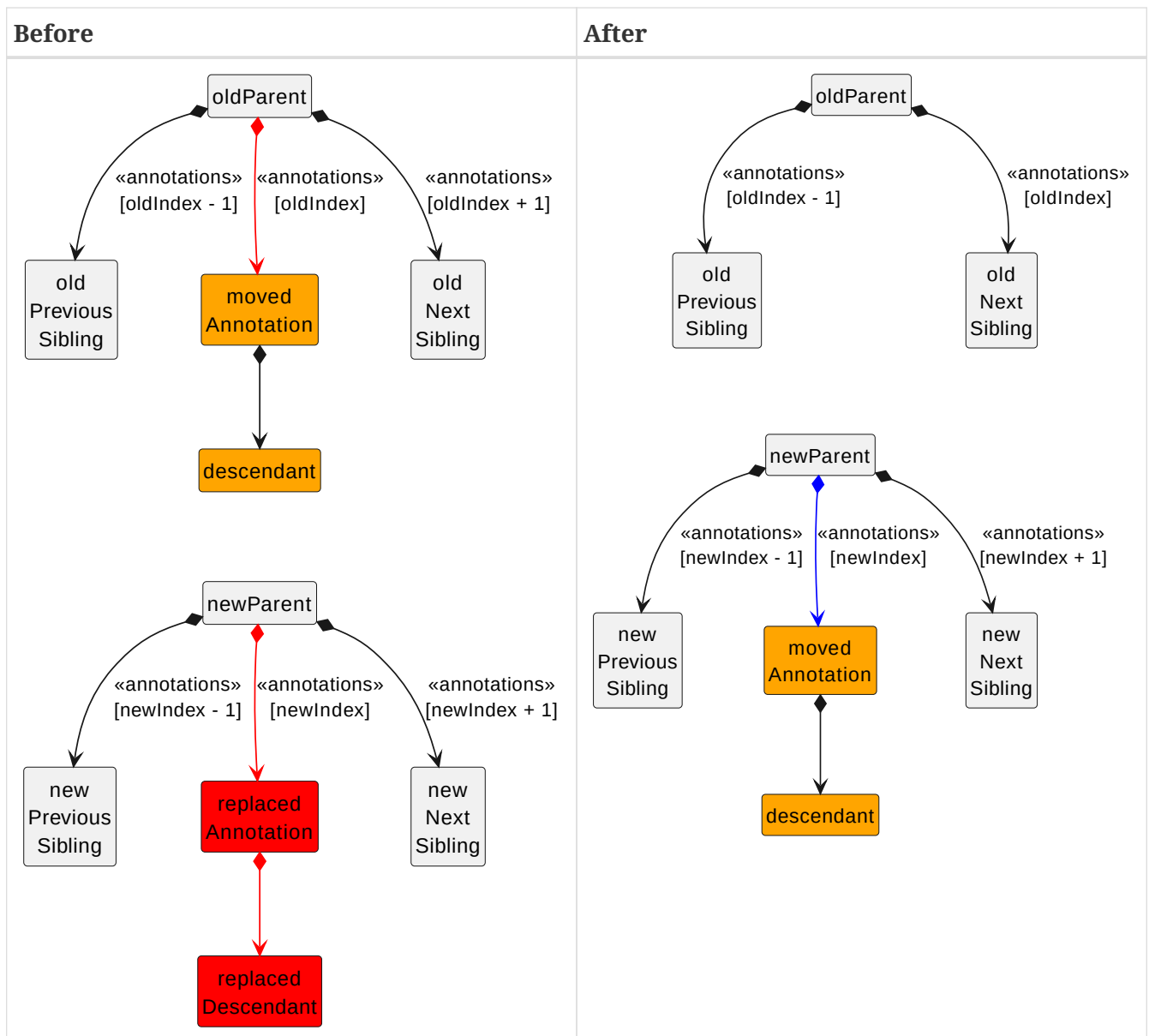
Related event

Annotation moved in same parent

5.8.6.6. Move annotation from other parent and replace existing annotation

Move **existing node** **movedAnnotation** from **oldParent**'s annotations at **oldIndex** to **newIndex** inside **newParent**'s annotations.

Delete **current node** **replacedAnnotation**^[69] at **newParent**'s annotations at **newIndex**, and all its descendants (including annotation instances). Does NOT change references to any of the deleted nodes.^[67]



Technical name

MoveAndReplaceAnnotationFromOtherParent

Parameters

- | | |
|---------------------------|--|
| newParent | NodeId of the node we want to move an annotation to. |
| newIndex | Index where we want to move movedAnnotation at in annotations of newParent . |
| oldParent | NodeId of the node we want to move an existing annotation from. |
| oldIndex | Index where we want to move movedAnnotation from in annotations of oldParent . |
| replacedAnnotation | NodeId of the node we want to replace at newIndex of annotations in newParent . |
| movedAnnotation | NodeId of the node we want to move from annotations in oldParent to annotations in newParent . |

commandId	CommandId Id of this command.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Errors

- [Unknown node](#) for [newParent](#)
- [Unknown node](#) for [oldParent](#)
- [Unknown node](#) for [movedAnnotation](#)
- [Unknown node](#) for [replacedAnnotation](#)
- The given parent of a node is not its parent for [oldParent](#) and [movedAnnotation](#)
- The given parent of a node is not its parent for [newParent](#) and [replacedAnnotation](#)
- [Index out of range](#) for [newIndex](#) (is beyond the number of annotations of [newParent](#))
- [Index out of range](#) for [oldIndex](#) (is beyond the number of annotations of [oldParent](#))
- [Annotation at index is not the given annotation](#) for [newIndex](#) and [replacedAnnotation](#)
- [Annotation at index is not the given annotation](#) for [oldIndex](#) and [movedAnnotation](#)
- Old and new parent identical for [oldParent](#) and [newParent](#)
- Old and new index identical for [oldIndex](#) and [newIndex](#)
- [Invalid participation](#)

Related event

[Annotation moved from other parent and replaced existing annotation](#)

5.8.6.7. Move annotation in same parent and replace existing annotation

Move [existing node](#) [movedAnnotation](#) that's initially at index [oldIndex](#) in [parent](#)'s annotations by [indexOffset](#) positions to index ([oldIndex](#) + [indexOffset](#)), replacing [current node](#) [replacedAnnotation](#)^[69]. This deletes [replacedAnnotation](#), and all its descendants (including annotation instances). Does NOT change references to any of the deleted nodes.^[67]

This conforms to [the action of moving a node and replacing another one in one specific list](#) — see that section for more information, diagrams, and examples.

Technical name

[MoveAndReplaceAnnotationInSameParent](#)

Parameters

parent	NodeId of the node we want to move an annotation in.
oldIndex	NodeId where we want to move movedAnnotation from in annotations of parent .
indexOffset	IndexOffset how much we want to move movedAnnotation in annotations of parent .
movedAnnotation	NodeId of the node we want to move in annotations in parent .

replacedAnnotation **NodeId** of the node we want to replace in annotations in **parent**.

commandId **CommandId** Id of this command.

additionalInfos **AdditionalInfos** Optionally empty list of **additional infos**.

Errors

- **Unknown node** for **parent**
- **Unknown node** for **movedAnnotation**
- **Unknown node** for **replacedAnnotation**
- **The given parent of a node is not its parent** for **parent** and **movedAnnotation**
- **The given parent of a node is not its parent** for **parent** and **replacedAnnotation**
- **Index out of range** for **oldIndex** is not an integer in the range [0..(the number of annotations on **parent** - 1)]
- **Index offset is invalid** for **indexOffset** if **indexOffset** equals 0, or if (**oldIndex** + **indexOffset**) is not in the range [0..(the number of annotations on **parent** - 1)]
- **Annotation at index is not the given annotation** for **oldIndex** and **movedAnnotation** if **movedAnnotation** is not at index **oldIndex** in the annotations on **parent**
- **Annotation at index is not the given annotation** for **oldIndex/indexOffset** and **replacedAnnotation** if **replacedAnnotation** is not at index (**oldIndex** + **indexOffset**) in the annotations on **parent**
- **Invalid participation**

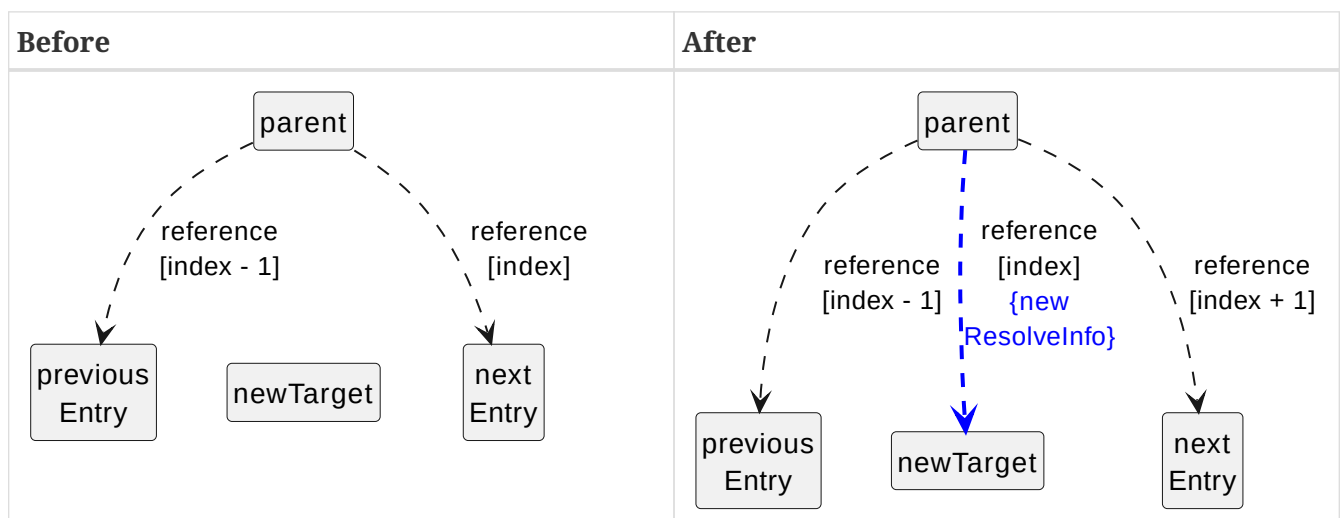
Related event

Annotation moved in same parent and replaced existing annotation

5.8.7. References

5.8.7.1. Add reference

Add **newReference** / **newResolveInfo** to **parent**'s **reference** at **index**.



Technical name

AddReference

Parameters

parent	NodeId of the node we want to add a reference to.
reference	MetaPointer Reference we want to add to parent .
index	Index where we want to add newReference / newResolveInfo at in reference of parent .
newReference	NodeId? of the node we want to set reference in parent to, if any.
newResolveInfo	ResolveInfo? of the node we want to set reference in parent to, if any.
commandId	CommandId Id of this command.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Errors

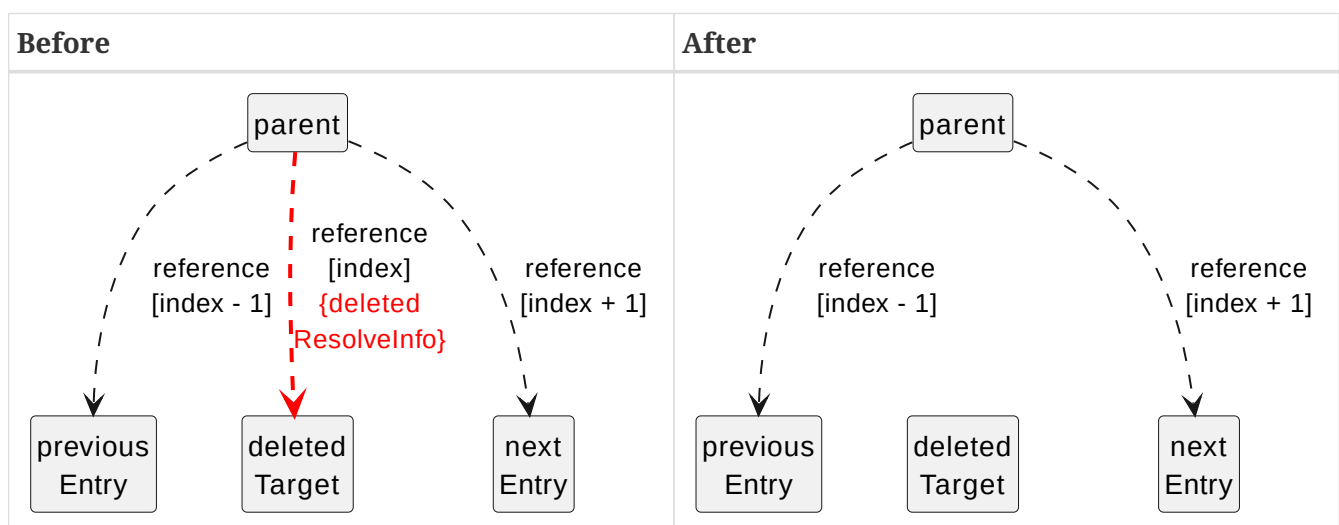
- **Unknown node** for **parent**
- **Unknown reference** for **reference** in **parent**
- **Reference already set** for **reference** in **parent**
- **Index out of range** for **index** (is beyond the number of **reference** entries of **parent**)
- **Undefined reference target** for **newReference**/ **newResolveInfo**
- **Invalid participation**

Related event

Reference added

5.8.7.2. Delete reference

Delete existing entry **deletedReference/deletedResolveInfo**^[69] from **parent**'s **reference** at **index**.



Technical name

DeleteReference

Parameters

parent	NodeId of the node we want to delete a reference from.
reference	MetaPointer Reference we want to delete from parent .
index	Index where we want to delete deletedReference / deletedResolveInfo at in reference of parent .
deletedReference	NodeId ? of the node reference was set to before in parent , if any.
deletedResolveInfo	ResolveInfo ? of the node reference was set to before in parent , if any.
commandId	CommandId Id of this command.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Errors

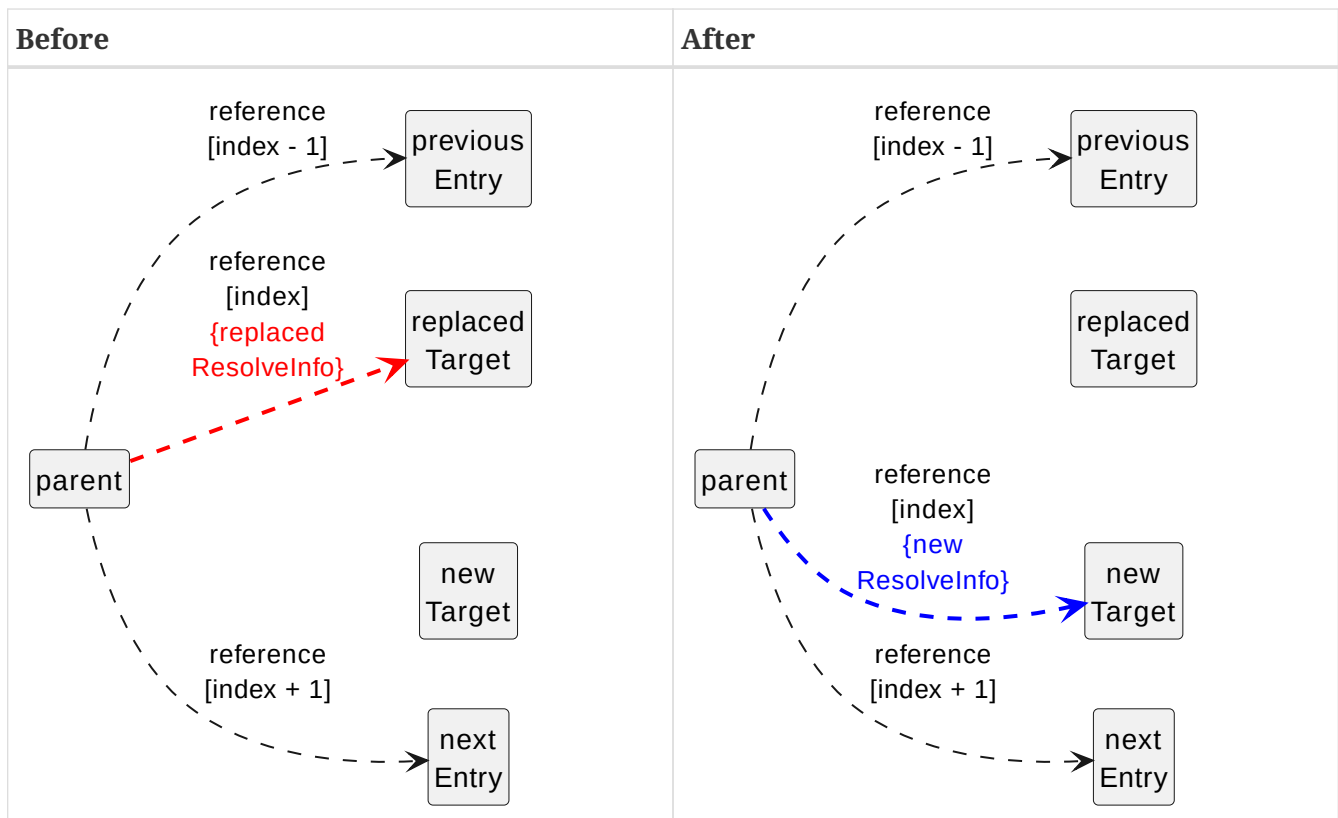
- [Unknown node](#) for **parent**
- [Unknown reference](#) for **reference** in **parent**
- [Reference not set](#) for **reference** in **parent**
- [Index out of range](#) for **index** (is beyond (the number of **reference** entries of **parent**) - 1)
- [Reference at index is not the given reference](#) for **reference/index** and **deletedReference** / **deletedResolveInfo**
- [Undefined reference target](#) for **deletedReference/deletedResolveInfo**
- [Invalid participation](#)

Related event

[Reference deleted](#)

5.8.7.3. Change reference

Replace existing entry **oldReference/oldResolveInfo**^[69] inside **parent's reference** at **index** with **newReference/newResolveInfo**.



Technical name

ChangeReference

Parameters

parent	NodeId of the node we want to change a reference of.
reference	MetaPointer Reference we want to change in parent .
index	Index where we want to change oldReference / oldResolveInfo at in reference of parent .
oldReference	NodeId ? of the node reference is currently set to in parent , if any.
oldResolveInfo	ResolveInfo ? of the node reference is currently set to in parent , if any.
newReference	NodeId ? of the node we want to set reference in parent to, if any.
newResolveInfo	ResolveInfo ? of the node we want to set reference in parent to, if any.
commandId	CommandId Id of this command.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Errors

- [Unknown node](#) for [parent](#)
- [Unknown reference](#) for [reference](#) in [parent](#)
- [Reference not set](#) for [reference](#) in [parent](#)
- [Index out of range](#) for [index](#) (is beyond (the number of [reference](#) entries of [parent](#)) - 1)

- Undefined reference target for `newReference` and `newResolveInfo`
- Reference at index is not the given reference for `reference/index` and `oldReference/oldResolveInfo`
- Invalid participation

Related event

Reference changed

5.8.8. Composite

5.8.8.1. CompositeCommand

Groups several commands into a logical group.^[73] The parts are ordered.

Command groups don't guarantee anything; the repository MAY take the group as a hint to resolve conflicts in a different manner.

Whenever a composite command is syntactically incorrect in one of its `parts`, the repository MUST reject the complete composite command – as if the composite command didn't even start executing – as the command as a whole is (deemed) syntactically incorrect.^[74]

We CAN nest composite commands arbitrarily.^[75]

Semantics:^[54]

- The repository SHOULD process one `CompositeCommand` as one `CompositeEvent`.
- **No** transactional semantics: the repository MUST process all `parts` in order and independent of the outcome of previous ones. The repository MUST be in valid state after having processed each command.
- Any of the commands contained directly in `parts` as well as any command that's contained indirectly through `CompositeCommand` (or a `Custom commands`) MUST NOT be a `Continued Command`. Subsequently, any (directly or indirectly) contained `splittable command` MUST NOT have their `SplitFlag` set to `true`.^[39]
- If several `CompositeCommands` overlap due to repository handling (e.g. conflict resolution), the behavior is **UNDEFINED**.

Technical name

`CompositeCommand`^[76]

Parameters

parts	<code>Command[]</code> The commands this composite combines.
commandId	<code>CommandId</code> Id of this command. Composite commands have their own command id, separate from the command ids of its <code>parts</code> . ^[77]
additionalInfos	<code>AdditionalInfos</code> Optionally empty list of <code>additional infos</code> .

Errors

- accumulation of all errors of **parts**
- Composite command contains **splittable**
- Invalid participation

Related event

CompositeEvent

5.9. Events

5.9.1. Continued Event

A **continued chunk** for one of the following **splittable** events:^[59]

- Partition added **newPartition**
- Child added **newChild**
- Child replaced **newChild**
- Annotation added **newAnnotation**
- Annotation replaced **newAnnotation**

We identify the **splittable** event by its **continuedEventSequenceNumber**.

Technical name

ContinuedEvent

Parameters

chunk	DeltaChunk The continued chunk. The combination of all continued chunks MUST adhere to the same constraints as continuedEventSequenceNumber 's chunk.
continuedChunkCompleted	ContinuedChunkCompletedFlag Whether this message completes continuedEventSequenceNumber .
continuedChunkSequenceNumber	ContinuedChunkSequenceNumber ^[29] Number of this message in the sequence of all continued events relating to the same continuedEventSequenceNumber .
continuedEventSequenceNumber	EventSequenceNumber Sequence number of the related splittable event.
originCommands	CommandSource ^[78] The contents of this field MUST be exactly the same as those of the originCommands field of the corresponding event — i.e., the event designated by continuedEventSequenceNumber . The client MUST reject any differences as an error.
sequenceNumber	EventSequenceNumber Event sequence number of this event.

additionalInfos [AdditionalInfos](#) Optionally empty list of [additional infos](#).

5.9.2. Partitions

5.9.2.1. Partition added

[New](#) partition **newPartition** has been added to the repository.

See also [splittable message](#).

Technical name

PartitionAdded

Parameters

newPartition [single chunk](#) (if subscribed via [List and subscribe partitions](#) or [Subscribe to partition changes](#)) or [shallow chunk](#) (if subscribed via [Inform about partition changes](#)) up to **depthLimit**.

NOTE

This parameter differs from the [corresponding command parameter](#): The command's parameter always contains a whole subtree, whereas the event's parameter may contain only the top of the subtree.^[36]

split? [SplitFlag](#) Whether **newPartition** is complete within this message.

originCommands [CommandSource\[\]](#) List of (**participationId**, **commandId**) pairs this event is based on.

sequenceNumber [EventSequenceNumber](#) Event sequence number of this event.

additionalInfos [AdditionalInfos](#) Optionally empty list of [additional infos](#).

Related command

[Add partition](#)

5.9.2.2. Partition deleted

Partition **deletedPartition**, and all its **deletedDescendants**^[79], have been deleted from the repository.

This client is now unsubscribed from any changes to **deletedPartition** or any of its descendants.

Technical name

PartitionDeleted

Parameters

deletedPartition [NodeId](#) of the partition that has been deleted.

deletedDescendants [NodeId\[\]](#) List of all node ids that used to be descendants (including

children and annotations) of `deletedDescendants`. Does not include `deletedDescendants`.

originCommands	<code>CommandSource[]</code> List of (<code>participationId</code> , <code>commandId</code>) pairs this event is based on.
sequenceNumber	<code>EventSequenceNumber</code> Event sequence number of this event.
additionalInfos	<code>AdditionalInfos</code> Optionally empty list of <code>additional infos</code> .

Related command

[Delete partition](#)

5.9.3. Nodes

5.9.3.1. Classifier changed

Classifier of `node` has been changed from `oldClassifier` to `newClassifier`.

Technical name

`ClassifierChanged`

Parameters

node	<code>NodeId</code> of the node whose classifier has been changed.
newClassifier	<code>MetaPointer</code> New <code>Classifier</code> of <code>node</code> .
oldClassifier	<code>MetaPointer</code> Old <code>Classifier</code> of <code>node</code> .
originCommands	<code>CommandSource[]</code> List of (<code>participationId</code> , <code>commandId</code>) pairs this event is based on.
sequenceNumber	<code>EventSequenceNumber</code> Event sequence number of this event.
additionalInfos	<code>AdditionalInfos</code> Optionally empty list of <code>additional infos</code> .

Related command

[Change classifier](#)

5.9.4. Properties

5.9.4.1. Property added

`property` of `node` has been set to `newValue`.

Technical name

`PropertyAdded`

Parameters

node	<code>NodeId</code> of the node the property has been added to.
-------------	---

property	MetaPointer Property that has been added to node .
newValue	PropertyValue Value property in node has been set to.
originCommands	CommandSource [] List of (participationId , commandId) pairs this event is based on.
sequenceNumber	EventSequenceNumber Event sequence number of this event.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Related command

[Add property](#)

5.9.4.2. Property deleted

Existing **property** of **node** with **oldValue** has been deleted.

Technical name

PropertyDeleted

Parameters

node	NodeId of the node the property has been deleted from.
property	MetaPointer Property that has been deleted from node .
oldValue	PropertyValue Value property in node had before.
originCommands	CommandSource [] List of (participationId , commandId) pairs this event is based on.
sequenceNumber	EventSequenceNumber Event sequence number of this event.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Related command

[Delete property](#)

5.9.4.3. Property changed

Existing **property** of **node** has been changed from **oldValue** to **newValue**.

Technical name

PropertyChanged

Parameters

node	NodeId of the node whose property has been changed.
property	MetaPointer Property that has been changed in node .
newValue	PropertyValue Value property in node has been changed to.

oldValue	PropertyValue Value property in node had before.
originCommands	CommandSource [] List of (participationId , commandId) pairs this event is based on.
sequenceNumber	EventSequenceNumber Event sequence number of this event.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Related command

[Change property](#)

5.9.5. Children

5.9.5.1. Child added

New **node** **newChild** has been added to **parent's containment** at **index**.

All other children inside **parent's containment** with index $\geq$ **index** have been moved to next higher index.

See also [splittable message](#).

Technical name

ChildAdded

Parameters

parent	NodeId of the node the new child was added to.
newChild	single chunk Subtree that formed the new child. The <i>anchor node</i> MUST contain the parent as <i>parent</i> . ^[68] All nodes in that subtree MUST be new nodes . ^[80]
containment	MetaPointer Containment newChild was added to in parent .
index	Index where newChild was added in containment of parent .
split?	SplitFlag Whether newChild is complete within this message.
originCommands	CommandSource [] List of (participationId , commandId) pairs this event is based on.
sequenceNumber	EventSequenceNumber Event sequence number of this event.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Related command

[Add child](#)

5.9.5.2. Child deleted

Existing node `deletedChild`, and all its `deletedDescendants`^[79], have been deleted from `parent`'s `containment` at `index`.^[81] All other children inside `parent`'s `containment` with `index > index` have been moved to next lower index.

Technical name

`ChildDeleted`

Parameters

deletedChild	<code>NodeId</code> of the node that was deleted in <code>containment</code> of <code>parent</code> at <code>index</code> .
deletedDescendants	<code>NodeId[]</code> List of all node ids that used to be descendants (including children and annotations) of <code>deletedChild</code> . Does not include <code>deletedChild</code> .
parent	<code>NodeId</code> of the node a child was deleted from.
containment	<code>MetaPointer Containment</code> <code>deletedChild</code> was deleted from in <code>parent</code> .
index	<code>Index</code> where <code>deletedChild</code> was deleted from in <code>containment</code> of <code>parent</code> .
originCommands	<code>CommandSource[]</code> List of (<code>participationId</code> , <code>commandId</code>) pairs this event is based on.
sequenceNumber	<code>EventSequenceNumber</code> Event sequence number of this event.
additionalInfos	<code>AdditionalInfos</code> Optionally empty list of <code>additional infos</code> .

Related command

`Delete child`

5.9.5.3. Child replaced

Existing node `replacedChild`, and all its `replacedDescendants`^[79], inside `parent`'s `containment` at `index` has been replaced with `new` or `reused` node `newChild`.

`replacedChild`, and all its descendants, have been deleted.

See also `splittable message`.

Technical name

`ChildReplaced`

Parameters

newChild	<code>single chunk</code> Subtree that formed the new child. The <i>anchor node</i> MUST contain the <code>parent</code> as <i>parent</i> . ^[68] All nodes in that subtree MUST be <code>new nodes</code> or <code>reused nodes</code> . ^[80]
replacedChild	<code>NodeId</code> of the node that was replaced in <code>containment</code> of <code>parent</code> at <code>index</code> .

replacedDescendants	NodeId [] List of all node ids that used to be descendants (including children and annotations) of replacedChild . Does not include replacedChild .
parent	NodeId of the node a child was replaced in.
containment	MetaPointer Containment replacedChild was replaced in parent .
index	Index where replacedChild was replaced in containment of parent .
split?	SplitFlag Whether newChild is complete within this message.
originCommands	CommandSource [] List of (participationId , commandId) pairs this event is based on.
sequenceNumber	EventSequenceNumber Event sequence number of this event.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Related command

[Replace child](#)

5.9.5.4. Child moved from containment in other parent

[Existing node](#) [movedChild](#) (previously inside [oldParent](#)'s [oldContainment](#) at [oldIndex](#)) has been moved inside [newParent](#)'s [newContainment](#) at [newIndex](#).

All other children inside [oldParent](#)'s [oldContainment](#) with index > [oldIndex](#) have been moved to next lower index.

All other children inside [newParent](#)'s [newContainment](#) with index >= [newIndex](#) have been moved to next higher index.

Technical name

[ChildMovedFromContainmentInOtherParent](#)^[70]

Parameters

newParent	NodeId of the node the child was moved to.
newContainment	MetaPointer Containment movedChild was moved to in newParent .
newIndex	Index where movedChild was moved to in newContainment of newParent .
movedChild	NodeId of the node that was moved from oldParent to newParent .
oldParent	NodeId of the node the child was moved from.
oldContainment	MetaPointer Containment movedChild was moved from in oldParent .
oldIndex	Index where movedChild was moved from in oldContainment of oldParent .
originCommands	CommandSource [] List of (participationId , commandId) pairs this event

is based on.

sequenceNumber [EventSequenceNumber](#) Event sequence number of this event.

additionalInfos [AdditionalInfos](#) Optionally empty list of [additional infos](#).

Related command

[Move child from containment in other parent](#)

5.9.5.5. Child moved from other containment in same parent

[Existing node](#) [movedChild](#) (previously inside [parent's oldContainment](#) at [oldIndex](#)) has been moved inside [parent's newContainment](#) at [newIndex](#).

All other children inside [parent's oldContainment](#) with index > [oldIndex](#) have been moved to next lower index.

All other children inside [parent's newContainment](#) with index >= [newIndex](#) have been moved to next higher index.

Technical name

[ChildMovedFromOtherContainmentInSameParent](#)

Parameters

newContainment [MetaPointer Containment](#) [movedChild](#) was moved to in [parent](#).

newIndex [Index](#) where [movedChild](#) was moved to in [newContainment](#) of [parent](#).

movedChild [NodeId](#) of the node that was moved from [oldContainment](#) to [newContainment](#).

parent [NodeId](#) of the node the child was moved in.

oldContainment [MetaPointer Containment](#) [movedChild](#) was moved from in [parent](#).

oldIndex [Index](#) where [movedChild](#) was moved from in [oldContainment](#) of [parent](#).

originCommands [CommandSource\[\]](#) List of ([participationId](#), [commandId](#)) pairs this event is based on.

sequenceNumber [EventSequenceNumber](#) Event sequence number of this event.

additionalInfos [AdditionalInfos](#) Optionally empty list of [additional infos](#).

Related command

[Move child from other containment in same parent](#)

5.9.5.6. Child moved in same containment in same parent

[Existing node](#) [movedChild](#) that was initially at index [oldIndex](#) within the [containment](#) inside [parent](#) was moved by [indexOffset](#) positions to index ([oldIndex](#) + [indexOffset](#)).

This conforms to [the action of moving a node in one specific list](#) — see that section for more information, diagrams, and examples.

Technical name

`ChildMovedInSameContainmentInSameParent`^[71]

Parameters

parent	NodeId of the node the child was moved in.
containment	MetaPointer Containment <code>movedChild</code> was moved in <code>parent</code> .
oldIndex	Index where <code>movedChild</code> was moved from in <code>containment</code> of <code>parent</code> .
indexOffset	IndexOffset how much <code>movedChild</code> has moved in <code>containment</code> of <code>parent</code> .
movedChild	NodeId of the node that was moved in <code>containment</code> .
originCommands	CommandSource [] List of (<code>participationId</code> , <code>commandId</code>) pairs this event is based on.
sequenceNumber	EventSequenceNumber Event sequence number of this event.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Related command

[Move child in same containment in same parent](#)

5.9.5.7. Child moved from containment in other parent and replaced existing child

[Existing node](#) `movedChild` (previously inside `oldParent`'s `oldContainment` at `oldIndex`) has replaced the [existing](#) `replacedChild` inside `newParent`'s `newContainment` at `newIndex`. `replacedChild`, and all its `replacedDescendants`^[79], have been deleted.

All other children inside `oldParent`'s `oldContainment` with index > `oldIndex` have been moved to next lower index.

No other children inside `newParent`'s `newContainment` have been moved.

Technical name

`ChildMovedAndReplacedFromContainmentInOtherParent`^[70]

Parameters

newParent	NodeId of the node <code>movedChild</code> was moved to.
newContainment	MetaPointer Containment <code>movedChild</code> was moved to in <code>newParent</code> .
newIndex	Index where <code>movedChild</code> was moved to in <code>newContainment</code> of <code>newParent</code> .
movedChild	NodeId of the node that was moved from <code>oldContainment</code> in <code>oldParent</code> to <code>newContainment</code> in <code>newParent</code> .
oldParent	NodeId of the node <code>movedChild</code> was moved from.

oldContainment	MetaPointer Containment movedChild was moved from in oldParent .
oldIndex	Index where movedChild was moved from in oldContainment of oldParent .
replacedChild	NodeId of the node that was replaced at newIndex of newContainment in newParent .
replacedDescendants	NodeId[] List of all node ids that used to be descendants (including children and annotations) of replacedChild . Does not include replacedChild .
originCommands	CommandSource[] List of (participationId , commandId) pairs this event is based on.
sequenceNumber	EventSequenceNumber Event sequence number of this event.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Related command

[Move child from containment in other parent and replace existing child](#)

5.9.5.8. Child moved from other containment in same parent and replaced existing child

[Existing node](#) **movedChild** (previously inside **parent's oldContainment** at **oldIndex**) has replaced the [existing replacedChild](#) inside **parent's newContainment** at **newIndex**. **replacedChild**, and all its **replacedDescendants**^[79], have been deleted.

All other children inside **parent's oldContainment** with index > **oldIndex** have been moved to next lower index.

No other children inside **parent's newContainment** have been moved.

Technical name

ChildMovedAndReplacedFromOtherContainmentInSameParent

Parameters

newContainment	MetaPointer Containment movedChild was moved to in parent .
newIndex	Index where movedChild was moved to in newContainment of parent .
movedChild	NodeId of the node that was moved from oldContainment to newContainment in parent .
parent	NodeId of the node the child was moved in.
oldContainment	MetaPointer Containment movedChild was moved from in parent .
oldIndex	Index where movedChild was moved from in oldContainment of parent .
replacedChild	NodeId of the node that was replaced at newIndex of newContainment in parent .

replacedDescendants	NodeId [] List of all node ids that used to be descendants (including children and annotations) of replacedChild . Does not include replacedChild .
originCommands	CommandSource [] List of (participationId , commandId) pairs this event is based on.
sequenceNumber	EventSequenceNumber Event sequence number of this event.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Related command

[Move child from other containment in same parent and replace existing child](#)

5.9.5.9. Child moved in same containment in same parent and replaced existing child

[Existing node](#) [movedChild](#) that was initially at index [oldIndex](#) within the [containment](#) inside [parent](#) was moved by [indexOffset](#) positions to index ([oldIndex](#) + [indexOffset](#)), replacing [current child](#) [replacedChild](#)^[69] at the target index.^[72] [replacedChild](#) subsequently gets deleted, including all its descendants (including annotation instances). This does NOT change references to any of the deleted nodes.^[67]

This conforms to [the action of moving a node and replacing another one in one specific list](#) — see that section for more information, diagrams, and examples.

Technical name

[ChildMovedAndReplacedInSameContainmentInSameParent](#)^[71]

Parameters

parent	NodeId of the node the child was moved in.
containment	MetaPointer Containment movedChild was moved and replacedChild was replaced in parent .
oldIndex	Index where movedChild was moved from in containment of parent .
indexOffset	IndexOffset how much movedChild was moved in containment of parent . (oldIndex + indexOffset) MUST NOT be < 0, or >= (the number of children inside parent 's containment).
movedChild	NodeId of the node that was moved in containment in parent .
replacedChild	NodeId of the node that was replaced in containment in parent .
replacedDescendants	NodeId [] List of all node ids that used to be descendants (including children and annotations) of replacedChild . Does not include replacedChild .
originCommands	CommandSource [] List of (participationId , commandId) pairs this

event is based on.

sequenceNumber	EventSequenceNumber Event sequence number of this event.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Related command

[Move child in same containment in same parent and replace existing child](#)

5.9.6. Annotations

5.9.6.1. Annotation added

New [node](#) [newAnnotation](#) has been added to [parent](#)'s annotations at [index](#).

All other annotations inside [parent](#)'s annotations with index $\geq$ [index](#) have been moved to next higher index.

See also [splittable message](#).

Technical name

[AnnotationAdded](#)

Parameters

parent	NodeId of the node the new annotation was added to.
newAnnotation	single chunk Subtree that formed the new annotation. The <i>anchor node</i> MUST contain the parent as <i>parent</i> . ^[68] All nodes in that subtree MUST be new nodes . ^[80]
index	Index where newAnnotation was added in annotations of parent .
split?	SplitFlag Whether newAnnotation is complete within this message.
originCommands	CommandSource [] List of (participationId , commandId) pairs this event is based on.
sequenceNumber	EventSequenceNumber Event sequence number of this event.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Related command

[Add annotation](#)

5.9.6.2. Annotation deleted

[Existing node](#) [deletedAnnotation](#), and all its [deletedDescendants](#)^[79], have been deleted from [parent](#)'s annotations at [index](#).^[81] All other annotations inside [parent](#)'s annotations with index $>$ [index](#) have been moved to next lower index.

Technical name

AnnotationDeleted

Parameters

deletedAnnotation	NodeId of the node that was deleted in annotations of parent at index .
deletedDescendants	NodeId [] List of all node ids that used to be descendants (including children and annotations) of deletedAnnotation . Does not include deletedAnnotation .
parent	NodeId of the node an annotation was deleted from.
index	Index where deletedAnnotation was deleted from in annotations of parent .
originCommands	CommandSource [] List of (participationId , commandId) pairs this event is based on.
sequenceNumber	EventSequenceNumber Event sequence number of this event.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Related command

[Delete annotation](#)

5.9.6.3. Annotation replaced

Existing [node](#) [replacedAnnotation](#), and all its [replacedDescendants](#)^[79], inside [parent](#)'s annotations at [index](#) has been replaced with [new](#) or [reused](#) node [newAnnotation](#).

[replacedAnnotation](#), and all its descendants, have been deleted.

See also [splittable message](#).

Technical name

AnnotationReplaced

Parameters

newAnnotation	single chunk Subtree that formed the new annotation. The <i>anchor node</i> MUST contain the parent as <i>parent</i> . ^[68] All nodes in that subtree MUST either be new nodes or reused nodes . ^[80]
replacedAnnotation	NodeId of the node that was replaced in annotations of parent at index .
replacedDescendants	NodeId [] List of all node ids that used to be descendants (including children and annotations) of replacedAnnotation . Does not include replacedAnnotation .
parent	NodeId of the node an annotation was replaced in.
index	Index where replacedAnnotation was replaced in annotations of

parent.

split?	SplitFlag Whether newAnnotation is complete within this message.
originCommands	CommandSource [] List of (participationId , commandId) pairs this event is based on.
sequenceNumber	EventSequenceNumber Event sequence number of this event.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Related command

[Replace annotation](#)

5.9.6.4. Annotation moved from other parent

[Existing node](#) [movedAnnotation](#) (previously inside [oldParent](#)'s annotations at [oldIndex](#)) has been moved inside [newParent](#)'s annotations at [newIndex](#).

All other annotations inside [oldParent](#)'s annotations with index > [oldIndex](#) have been moved to next lower index.

All other annotations inside [newParent](#)'s annotations with index >= [newIndex](#) have been moved to next higher index.

Technical name

[AnnotationMovedFromOtherParent](#)

Parameters

newParent	NodeId of the node the annotation was moved to.
newIndex	Index where movedAnnotation was moved to in annotations of newParent .
movedAnnotation	NodeId of the annotation that was moved from oldParent to newParent .
oldParent	NodeId of the node the annotation was moved from.
oldIndex	Index where movedAnnotation was moved from in annotations of oldParent .
originCommands	CommandSource [] List of (participationId , commandId) pairs this event is based on.
sequenceNumber	EventSequenceNumber Event sequence number of this event.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Related command

[Move annotation from other parent](#)

5.9.6.5. Annotation moved in same parent

Existing node `movedAnnotation` that was initially at index `oldIndex` in `parent`'s annotations was moved by `indexOffset` positions to index $(oldIndex + indexOffset)$.

This conforms to [the action of moving a node in one specific list](#) — see that section for more information, diagrams, and examples.

Technical name

`AnnotationMovedInSameParent`

Parameters

parent	<code>NodeId</code> of the node the annotation was moved in.
oldIndex	<code>NodeId</code> where <code>movedAnnotation</code> was moved from in annotations of <code>parent</code> .
indexOffset	<code>IndexOffset</code> ^[82] how much <code>movedAnnotation</code> was moved in annotations of <code>parent</code> .
movedAnnotation	<code>NodeId</code> of the annotation that was moved.
originCommands	<code>CommandSource</code> [] List of $(participationId, commandId)$ pairs this event is based on.
sequenceNumber	<code>EventSequenceNumber</code> Event sequence number of this event.
additionalInfos	<code>AdditionalInfos</code> Optionally empty list of additional infos .

Related command

[Move annotation in same parent](#)

5.9.6.6. Annotation moved from other parent and replaced existing annotation

Existing node `movedAnnotation` (previously inside `oldParent`'s annotations at `oldIndex`) has replaced the [existing](#) `replacedAnnotation` inside `newParent`'s annotations at `newIndex`. `replacedAnnotation`, and all its `replacedDescendants`^[79], have been deleted.

All other annotations inside `oldParent`'s annotations with index $> oldIndex$ have been moved to next lower index.

No other annotations inside `newParent`'s annotations have been moved.

Technical name

`AnnotationMovedAndReplacedFromOtherParent`

Parameters

newParent	<code>NodeId</code> of the node the annotation was moved to.
newIndex	<code>Index</code> where <code>movedAnnotation</code> was moved to in annotations of <code>newParent</code> .

movedAnnotation	NodeId of the node that was moved from annotations in oldParent to annotations in newParent .
oldParent	NodeId of the node the annotation was moved from.
oldIndex	Index where movedAnnotation was moved from in annotations of oldParent .
replacedAnnotation	NodeId of the node that was replaced at newIndex of annotations in newParent .
replacedDescendants	NodeId [] List of all node ids that used to be descendants (including children and annotations) of replacedAnnotation . Does not include replacedAnnotation .
originCommands	CommandSource [] List of (participationId , commandId) pairs this event is based on.
sequenceNumber	EventSequenceNumber Event sequence number of this event.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Related command

[Move annotation from other parent and replace existing annotation](#)

5.9.6.7. Annotation moved in same parent and replaced existing annotation

[Existing node](#) [movedAnnotation](#) that was initially at index [oldIndex](#) in [parent](#)'s annotations was moved by [indexOffset](#) positions to index ([oldIndex](#) + [indexOffset](#)), replacing [current node replacedAnnotation](#)^[69]. This deletes [replacedAnnotation](#), and all its descendants (including annotation instances). Does NOT change references to any of the deleted nodes.^[67]

This conforms to [the action of moving a node and replacing another one in one specific list](#) — see that section for more information, diagrams, and examples.

Technical name

[AnnotationMovedAndReplacedInSameParent](#)

Parameters

parent	NodeId of the node the annotation was moved in.
oldIndex	NodeId where movedAnnotation was moved from in annotations of parent .
indexOffset	IndexOffset how much movedAnnotation was moved in annotations of parent .
movedAnnotation	NodeId of the node that was moved in annotations in parent .
replacedAnnotation	NodeId of the node that was replaced in annotations in parent .

replacedDescendants	NodeId [] List of all node ids that used to be descendants (including children and annotations) of replacedAnnotation . Does not include replacedAnnotation .
originCommands	CommandSource [] List of (participationId , commandId) pairs this event is based on.
sequenceNumber	EventSequenceNumber Event sequence number of this event.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Related command

[Move annotation in same parent and replace existing annotation](#)

5.9.7. References

5.9.7.1. Reference added

Reference with [newReference](#)/[newResolveInfo](#) has been added to [parent](#)'s [reference](#) at [index](#). All other entries inside [parent](#)'s [reference](#) with index $\geq$ [index](#) have been moved to next higher index.

Technical name

[ReferenceAdded](#)

Parameters

parent	NodeId of the node the reference has been added to.
reference	MetaPointer Reference that has been added to parent .
index	Index where newReference / newResolveInfo has been added at in reference of parent .
newReference	NodeId ? of the node reference in parent has been set to, if any.
newResolveInfo	ResolveInfo ? of the node reference in parent has been set to, if any.
originCommands	CommandSource [] List of (participationId , commandId) pairs this event is based on.
sequenceNumber	EventSequenceNumber Event sequence number of this event.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Related command

[Add reference](#)

5.9.7.2. Reference deleted

Existing reference with [deletedReference](#)/[deletedResolveInfo](#) has been deleted from [parent](#)'s [reference](#) at [index](#). All other entries inside [parent](#)'s [reference](#) with index $>$ [index](#) have been moved to next lower index.

Technical name

ReferenceDeleted

Parameters

parent	NodeId of the node the reference has been deleted from.
reference	MetaPointer Reference that has been deleted from parent .
index	Index where deletedReference / deletedResolveInfo has been deleted at in reference of parent .
deletedReference	NodeId ? of the node reference was set to before in parent , if any.
deletedResolveInfo	ResolveInfo ? of the node reference was set to before in parent , if any.
originCommands	CommandSource [] List of (participationId , commandId) pairs this event is based on.
sequenceNumber	EventSequenceNumber Event sequence number of this event.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Related command

[Delete reference](#)

5.9.7.3. Reference changed

Existing reference with **oldReference**/**oldResolveInfo** inside **parent**'s annotations at **index** has been replaced with **newReference**/**newResolveInfo**.

Technical name

ReferenceChanged

Parameters

parent	NodeId of the node whose reference has been changed.
reference	MetaPointer Reference that has been changed in parent .
index	Index where oldReference / oldResolveInfo has been changed at in reference of parent .
newReference	NodeId ? of the node reference in parent has been set to, if any.
newResolveInfo	ResolveInfo ? of the node reference in parent has been set to, if any.
oldReference	NodeId ? of the node reference was previously set to in parent , if any.
oldResolveInfo	ResolveInfo ? of the node reference was previously set to in parent , if any.
originCommands	CommandSource [] List of (participationId , commandId) pairs this event is based on.

sequenceNumber [EventSequenceNumber](#) Event sequence number of this event.

additionalInfos [AdditionalInfos](#) Optionally empty list of [additional infos](#).

Related command

[Change reference](#)

5.9.8. Miscellaneous

5.9.8.1. CompositeEvent

The events in **parts** have happened in the given order.^[73]

We CAN nest composite events arbitrarily^[75], and CAN include [ErrorEvent](#) and/or [No-op Event](#)^[83].

The repository MUST NOT create composite events on its own — (only as a result of composite commands).^[77]

Semantics:^[54]

1. A [CompositeEvent](#) SHOULD be the result of processing exactly one [CompositeCommand](#).
2. **No** transactional semantics: any client MUST process all **parts** in order and independent of the outcome of previous ones. Nodes detached and re-attached during a composite event SHOULD keep their identity in clients — this is effectively [reusing nodes](#).
 - Any of the events contained directly in **parts** as well as any event that's contained indirectly through [CompositeEvent](#) (or a [Custom events](#)) MUST NOT be a [Continued Event](#). Subsequently, any (directly or indirectly) contained [splittable event](#) MUST NOT have their [SplitFlag](#) set to **true**.^[39]
 - Ids of nodes that are deleted – as in: detached and not re-attached; see also [this paragraph in the Bulk API](#) – by any of the **parts** are considered free *only after* the outermost [CompositeEvent](#) has been processed.

Technical name

[CompositeEvent](#) ^[84]

Parameters

parts [Event](#)[] Events contained in **parts** MUST appear in ascending order of their sequence numbers.^[85]

originCommands [CommandSource](#)[] List of ([participationId](#), [commandId](#)) pairs this event is based on.

The following constraints apply:^[77]

- The **originCommands** of a composite event MUST only mention command ids of composite commands — (owing to semantics item 1).
- Composite commands MUST only be mentioned as origins of composite events — not of their **parts**. As an example of relations of

composite command **A** with sub-command **B** to composite event **C** and sub-event **D**:

- **C** MUST mention **A** in its own [originCommands](#)
- **D** MUST mention **B** in its [originCommands](#)
- **D** MUST NOT mention **A** in its [originCommands](#)

(Note that command **B** can be composite, in which case event **D** would be composite as well.)

sequenceNumber	EventSequenceNumber Event sequence number of this event. Composite events have their own sequence number, separate from the sequence numbers of its parts . ^[85] This sequence number MUST precede the sequence numbers of all its parts . ^[85]
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Related command

[CompositeCommand](#)

5.9.8.2. No-op Event

Nothing happened as result of one or more command(s).^[86]

See also [unsuccessful command](#).

Technical name

NoOpEvent^[87]

Parameters

originCommands	CommandSource [] List of (participationId , commandId) pairs this event is based on.
sequenceNumber	EventSequenceNumber Event sequence number of this event.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Related command

none

5.9.8.3. ErrorEvent

Repository couldn't handle one or more command(s).^{[88][58]}

The repository MAY send an error event to only one participation.^[35]

See also [Error response](#) and [unsuccessful command](#).

Technical name

ErrorEvent

Parameters

errorCode	ErrorCode Code of the reported error.
message	ErrorMessage Human-readable message of the error.
originCommands	CommandSource [] List of (participationId , commandId) pairs this event is based on.
sequenceNumber	EventSequenceNumber Event sequence number of this event.
additionalInfos	AdditionalInfos Optionally empty list of additional infos .

Related command

none

Appendix A: Delta JSON Schema

The LionWeb [delta JSON Schema](#).

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://lionweb.io/delta.schema.json",
  "title": "LionWeb Delta Protocol",
  "description": "Version: 0",
  "type": "object",
  "$comment": "using anyOf for this sum type, because because
CustomQuery{Request|Response} are indistinguishable; for all other sum types, using
oneOf to make matching as exclusive as possible",
  "anyOf": [
    {
      "$ref": "#/$defs/QueryRequests"
    },
    {
      "$ref": "#/$defs/QueryResponses"
    },
    {
      "$ref": "#/$defs/Commands"
    },
    {
      "$ref": "#/$defs/Events"
    }
  ],
  "$defs": {
    "id": {
      "type": "string",
      "minLength": 1,
      "pattern": "^[a-zA-Z0-9_-]+$"
    },
    "deltaProtocolVersion": {
```

```

    "enum": [
      "2026.1"
    ],
  },
  "key": {
    "$ref": "#/$defs/id"
  },
  "version": {
    "type": "string",
    "minLength": 1
  },
  "metaPointer": {
    "type": "object",
    "properties": {
      "language": {
        "$ref": "#/$defs/key"
      },
      "version": {
        "$ref": "#/$defs/version"
      },
      "key": {
        "$ref": "#/$defs/key"
      }
    }
  },
  "required": [
    "language",
    "version",
    "key"
  ],
  "additionalProperties": false,
  "minProperties": 3,
  "maxProperties": 3
},
"depthLimit": {
  "type": "integer",
  "minimum": 0
},
"split": {
  "type": "boolean"
},
"continuedChunkCompleted": {
  "type": "boolean"
},
"continuedChunkSequenceNumber": {
  "type": "integer",
  "minimum": 0
},
"DeltaChunk": {
  "type": "object",
  "properties": {
    "nodes": {

```

```

        "type": "array",
        "items": {
            "$ref": "#/$defs/SerializedNode"
        },
        "uniqueItems": true
    },
    },
    "required": [
        "nodes"
    ],
    "additionalProperties": false,
    "minProperties": 1,
    "maxProperties": 1
},
"SerializedNode": {
    "type": "object",
    "properties": {
        "id": {
            "$ref": "#/$defs/targetNode"
        },
        "classifier": {
            "$ref": "#/$defs/metaPointer"
        },
        "properties": {
            "type": "array",
            "items": {
                "$ref": "#/$defs/SerializedProperty"
            }
        },
        "containments": {
            "type": "array",
            "items": {
                "$ref": "#/$defs/SerializedContainment"
            }
        },
        "references": {
            "type": "array",
            "items": {
                "$ref": "#/$defs/SerializedReference"
            }
        },
        "annotations": {
            "type": "array",
            "items": {
                "$ref": "#/$defs/id"
            },
            "uniqueItems": true
        },
        "parent": {
            "oneOf": [
                {

```

```

        "$ref": "#/$defs/id"
      },
      {
        "type": "null"
      }
    ]
  },
  "required": [
    "id",
    "classifier",
    "properties",
    "containments",
    "references",
    "annotations",
    "parent"
  ],
  "additionalProperties": false,
  "minProperties": 7,
  "maxProperties": 7
},
"SerializedProperty": {
  "type": "object",
  "properties": {
    "property": {
      "$ref": "#/$defs/metaPointer"
    },
    "value": {
      "oneOf": [
        {
          "$ref": "#/$defs/propertyValue"
        },
        {
          "type": "null"
        }
      ]
    }
  }
},
"required": [
  "property",
  "value"
],
"additionalProperties": false,
"minProperties": 2,
"maxProperties": 2
},
"SerializedContainment": {
  "type": "object",
  "properties": {
    "containment": {
      "$ref": "#/$defs/metaPointer"
    }
  }
}

```

```

    },
    "children": {
      "type": "array",
      "items": {
        "$ref": "#/$defs/id"
      },
      "uniqueItems": true
    }
  },
  "required": [
    "containment",
    "children"
  ],
  "additionalProperties": false,
  "minProperties": 2,
  "maxProperties": 2
},
"SerializedReference": {
  "type": "object",
  "properties": {
    "reference": {
      "$ref": "#/$defs/metaPointer"
    },
    "targets": {
      "type": "array",
      "items": {
        "$ref": "#/$defs/SerializedReferenceTarget"
      }
    }
  }
},
"required": [
  "reference",
  "targets"
],
"additionalProperties": false,
"minProperties": 2,
"maxProperties": 2
},
"SerializedReferenceTarget": {
  "type": "object",
  "properties": {
    "resolveInfo": {
      "$ref": "#/$defs/optionalResolveInfo"
    },
    "reference": {
      "$ref": "#/$defs/optionalTargetNode"
    }
  }
},
"required": [
  "resolveInfo",
  "reference"
]

```

```

    ],
    "additionalProperties": false,
    "minProperties": 2,
    "maxProperties": 2
  },
  "queryId": {
    "$ref": "#/$defs/id"
  },
  "commandId": {
    "$ref": "#/$defs/id"
  },
  "commandSource": {
    "type": "object",
    "properties": {
      "participationId": {
        "$ref": "#/$defs/participationId"
      },
      "commandId": {
        "$ref": "#/$defs/commandId"
      }
    }
  },
  "required": [
    "participationId",
    "commandId"
  ],
  "additionalProperties": false,
  "minProperties": 2,
  "maxProperties": 2
},
"commandSources": {
  "type": "array",
  "items": {
    "$ref": "#/$defs/commandSource"
  },
  "uniqueItems": true
},
"participationId": {
  "$ref": "#/$defs/id"
},
"repositoryId": {
  "$ref": "#/$defs/id"
},
"clientId": {
  "$ref": "#/$defs/id"
},
"targetNode": {
  "$ref": "#/$defs/id"
},
"optionalTargetNode": {
  "oneOf": [
    {

```

```

        "$ref": "#/$defs/targetNode"
    },
    {
        "type": "null"
    }
]
},
"targetNodes": {
    "type": "array",
    "items": {
        "$ref": "#/$defs/targetNode"
    },
    "uniqueItems": true
},
"resolveInfo": {
    "type": "string"
},
"optionalResolveInfo": {
    "oneOf": [
        {
            "$ref": "#/$defs/resolveInfo"
        },
        {
            "type": "null"
        }
    ]
},
"eventSequenceNumber": {
    "type": "integer",
    "minimum": 0
},
"propertyValue": {
    "type": "string"
},
"index": {
    "type": "integer",
    "minimum": 0
},
"indexOffset": {
    "type": "integer"
},
"additionalInfoKind": {
    "$ref": "#/$defs/id"
},
"errorCode": {
    "$ref": "#/$defs/id"
},
"customMessageKind": {
    "type": "string",
    "minLength": 8,
    "pattern": "^Custom_[a-zA-Z0-9_-]+$"
}

```

```

},
"additionalInfoDataKey": {
  "$ref": "#/$defs/id"
},
"additionalInfos": {
  "type": "array",
  "items": {
    "$ref": "#/$defs/AdditionalInfo"
  }
},
"AdditionalInfo": {
  "type": "object",
  "properties": {
    "kind": {
      "$ref": "#/$defs/additionalInfoKind"
    },
    "distribute": {
      "$ref": "#/$defs/additionalInfoDistribute"
    },
    "message": {
      "type": "string"
    },
    "data": {
      "type": "object",
      "additionalProperties": {
        "type": "string"
      }
    }
  },
  "required": [
    "kind",
    "message",
    "data"
  ],
  "additionalProperties": false,
  "minProperties": 3,
  "maxProperties": 4
},
"additionalInfoDistribute": {
  "type": "boolean",
  "default": false
},
"QueryRequests": {
  "oneOf": [
    {
      "$ref": "#/$defs/SubscribeToChangingPartitionsRequest"
    },
    {
      "$ref": "#/$defs/InformAboutChangingPartitionsRequest"
    },
    {

```

```

    "$ref": "#/$defs/SubscribeToPartitionContentsRequest"
  },
  {
    "$ref": "#/$defs/UnsubscribeFromPartitionContentsRequest"
  },
  {
    "$ref": "#/$defs/SignOnRequest"
  },
  {
    "$ref": "#/$defs/SignOffRequest"
  },
  {
    "$ref": "#/$defs/ReconnectRequest"
  },
  {
    "$ref": "#/$defs/GetAvailableIdsRequest"
  },
  {
    "$ref": "#/$defs/ListPartitionsRequest"
  },
  {
    "$ref": "#/$defs/ListAndSubscribePartitionsRequest"
  },
  {
    "$ref": "#/$defs/CustomQueryRequest"
  }
]
},
"QueryResponses": {
  "oneOf": [
    {
      "$ref": "#/$defs/NonContinuedQueryResponses"
    },
    {
      "$ref": "#/$defs/ContinuedQueryResponse"
    }
  ]
},
"NonContinuedQueryResponses": {
  "oneOf": [
    {
      "$ref": "#/$defs/SubscribeToChangingPartitionsResponse"
    },
    {
      "$ref": "#/$defs/InformAboutChangingPartitionsResponse"
    },
    {
      "$ref": "#/$defs/SubscribeToPartitionContentsResponse"
    },
    {
      "$ref": "#/$defs/UnsubscribeFromPartitionContentsResponse"
    }
  ]
}

```

```

    },
    {
      "$ref": "#/$defs/SignOnResponse"
    },
    {
      "$ref": "#/$defs/SignOffResponse"
    },
    {
      "$ref": "#/$defs/ReconnectResponse"
    },
    {
      "$ref": "#/$defs/GetAvailableIdsResponse"
    },
    {
      "$ref": "#/$defs/ListPartitionsResponse"
    },
    {
      "$ref": "#/$defs/ListAndSubscribePartitionsResponse"
    },
    {
      "$ref": "#/$defs/ErrorResponse"
    },
    {
      "$ref": "#/$defs/CustomQueryResponse"
    }
  ]
},
"SubscribeToChangingPartitionsRequest": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "SubscribeToChangingPartitionsRequest"
    },
    "creation": {
      "type": "boolean"
    },
    "deletion": {
      "type": "boolean"
    },
    "queryId": {
      "$ref": "#/$defs/queryId"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "creation",
    "deletion",
    "queryId",

```

```

        "additionalInfos"
    ],
    "additionalProperties": false,
    "minProperties": 5,
    "maxProperties": 5
},
"SubscribeToChangingPartitionsResponse": {
    "type": "object",
    "properties": {
        "messageKind": {
            "const": "SubscribeToChangingPartitionsResponse"
        },
        "queryId": {
            "$ref": "#/$defs/queryId"
        },
        "additionalInfos": {
            "$ref": "#/$defs/additionalInfos"
        }
    },
    "required": [
        "messageKind",
        "queryId",
        "additionalInfos"
    ],
    "additionalProperties": false,
    "minProperties": 3,
    "maxProperties": 3
},
"InformAboutChangingPartitionsRequest": {
    "type": "object",
    "properties": {
        "messageKind": {
            "const": "InformAboutChangingPartitionsRequest"
        },
        "creation": {
            "type": "boolean"
        },
        "deletion": {
            "type": "boolean"
        },
        "depthLimit": {
            "$ref": "#/$defs/depthLimit"
        },
        "queryId": {
            "$ref": "#/$defs/queryId"
        },
        "additionalInfos": {
            "$ref": "#/$defs/additionalInfos"
        }
    },
    "required": [

```

```

        "messageKind",
        "creation",
        "deletion",
        "depthLimit",
        "queryId",
        "additionalInfos"
    ],
    "additionalProperties": false,
    "minProperties": 6,
    "maxProperties": 6
},
"InformAboutChangingPartitionsResponse": {
    "type": "object",
    "properties": {
        "messageKind": {
            "const": "InformAboutChangingPartitionsResponse"
        },
        "queryId": {
            "$ref": "#/$defs/queryId"
        },
        "additionalInfos": {
            "$ref": "#/$defs/additionalInfos"
        }
    },
    "required": [
        "messageKind",
        "queryId",
        "additionalInfos"
    ],
    "additionalProperties": false,
    "minProperties": 3,
    "maxProperties": 3
},
"SubscribeToPartitionContentsRequest": {
    "type": "object",
    "properties": {
        "messageKind": {
            "const": "SubscribeToPartitionContentsRequest"
        },
        "partition": {
            "$ref": "#/$defs/targetNode"
        },
        "queryId": {
            "$ref": "#/$defs/queryId"
        },
        "additionalInfos": {
            "$ref": "#/$defs/additionalInfos"
        }
    },
    "required": [
        "messageKind",

```

```

        "partition",
        "queryId",
        "additionalInfos"
    ],
    "additionalProperties": false,
    "minProperties": 4,
    "maxProperties": 4
},
"SubscribeToPartitionContentsResponse": {
    "type": "object",
    "properties": {
        "messageKind": {
            "const": "SubscribeToPartitionContentsResponse"
        },
        "contents": {
            "$ref": "#/$defs/DeltaChunk"
        },
        "split": {
            "$ref": "#/$defs/split"
        },
        "queryId": {
            "$ref": "#/$defs/queryId"
        },
        "additionalInfos": {
            "$ref": "#/$defs/additionalInfos"
        }
    },
    "required": [
        "messageKind",
        "contents",
        "queryId",
        "additionalInfos"
    ],
    "additionalProperties": false,
    "minProperties": 4,
    "maxProperties": 5
},
"UnsubscribeFromPartitionContentsRequest": {
    "type": "object",
    "properties": {
        "messageKind": {
            "const": "UnsubscribeFromPartitionContentsRequest"
        },
        "partition": {
            "$ref": "#/$defs/targetNode"
        },
        "queryId": {
            "$ref": "#/$defs/queryId"
        },
        "additionalInfos": {
            "$ref": "#/$defs/additionalInfos"
        }
    }
}

```

```

    },
    "required": [
        "messageKind",
        "partition",
        "queryId",
        "additionalInfos"
    ],
    "additionalProperties": false,
    "minProperties": 4,
    "maxProperties": 4
},
"UnsubscribeFromPartitionContentsResponse": {
    "type": "object",
    "properties": {
        "messageKind": {
            "const": "UnsubscribeFromPartitionContentsResponse"
        },
        "queryId": {
            "$ref": "#/$defs/queryId"
        },
        "additionalInfos": {
            "$ref": "#/$defs/additionalInfos"
        }
    },
    "required": [
        "messageKind",
        "queryId",
        "additionalInfos"
    ],
    "additionalProperties": false,
    "minProperties": 3,
    "maxProperties": 3
},
"SignOnRequest": {
    "type": "object",
    "properties": {
        "messageKind": {
            "const": "SignOnRequest"
        },
        "deltaProtocolVersion": {
            "$ref": "#/$defs/deltaProtocolVersion"
        },
        "clientId": {
            "$ref": "#/$defs/clientId"
        },
        "repositoryId": {
            "$ref": "#/$defs/repositoryId"
        },
        "queryId": {
            "$ref": "#/$defs/queryId"
        }
    }
}

```

```

    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "deltaProtocolVersion",
    "clientId",
    "repositoryId",
    "queryId",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 6,
  "maxProperties": 6
},
"SignOnResponse": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "SignOnResponse"
    },
    "participationId": {
      "$ref": "#/$defs/participationId"
    },
    "queryId": {
      "$ref": "#/$defs/queryId"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "participationId",
    "queryId",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 4,
  "maxProperties": 4
},
"SignOffRequest": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "SignOffRequest"
    },
    "queryId": {
      "$ref": "#/$defs/queryId"
    }
  }
}

```

```

    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "queryId",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 3,
  "maxProperties": 3
},
"SignOffResponse": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "SignOffResponse"
    },
    "queryId": {
      "$ref": "#/$defs/queryId"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "queryId",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 3,
  "maxProperties": 3
},
"ReconnectRequest": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "ReconnectRequest"
    },
    "deltaProtocolVersion": {
      "$ref": "#/$defs/deltaProtocolVersion"
    },
    "clientId": {
      "$ref": "#/$defs/clientId"
    },
    "repositoryId": {
      "$ref": "#/$defs/repositoryId"
    }
  },

```

```

    "participationId": {
      "$ref": "#/$defs/participationId"
    },
    "lastReceivedSequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    "queryId": {
      "$ref": "#/$defs/queryId"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "deltaProtocolVersion",
    "clientId",
    "repositoryId",
    "participationId",
    "lastReceivedSequenceNumber",
    "queryId",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 8,
  "maxProperties": 8
},
"ReconnectResponse": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "ReconnectResponse"
    },
    "lastSentSequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    "queryId": {
      "$ref": "#/$defs/queryId"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "lastSentSequenceNumber",
    "queryId",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 4,

```

```

    "maxProperties": 4
  },
  "GetAvailableIdsRequest": {
    "type": "object",
    "properties": {
      "messageKind": {
        "const": "GetAvailableIdsRequest"
      },
      "count": {
        "type": "integer",
        "minimum": 0
      },
      "queryId": {
        "$ref": "#/$defs/queryId"
      },
      "additionalInfos": {
        "$ref": "#/$defs/additionalInfos"
      }
    },
    "required": [
      "messageKind",
      "count",
      "queryId",
      "additionalInfos"
    ],
    "additionalProperties": false,
    "minProperties": 4,
    "maxProperties": 4
  },
  "GetAvailableIdsResponse": {
    "type": "object",
    "properties": {
      "messageKind": {
        "const": "GetAvailableIdsResponse"
      },
      "ids": {
        "$ref": "#/$defs/targetNodes"
      },
      "queryId": {
        "$ref": "#/$defs/queryId"
      },
      "additionalInfos": {
        "$ref": "#/$defs/additionalInfos"
      }
    },
    "required": [
      "messageKind",
      "ids",
      "queryId",
      "additionalInfos"
    ],
  },

```

```

    "additionalProperties": false,
    "minProperties": 4,
    "maxProperties": 4
  },
  "ListPartitionsRequest": {
    "type": "object",
    "properties": {
      "messageKind": {
        "const": "ListPartitionsRequest"
      },
      "depthLimit": {
        "$ref": "#/$defs/depthLimit"
      },
      "queryId": {
        "$ref": "#/$defs/queryId"
      },
      "additionalInfos": {
        "$ref": "#/$defs/additionalInfos"
      }
    },
    "required": [
      "messageKind",
      "depthLimit",
      "queryId",
      "additionalInfos"
    ],
    "additionalProperties": false,
    "minProperties": 4,
    "maxProperties": 4
  },
  "ListPartitionsResponse": {
    "type": "object",
    "properties": {
      "messageKind": {
        "const": "ListPartitionsResponse"
      },
      "partitions": {
        "$ref": "#/$defs/DeltaChunk"
      },
      "split": {
        "$ref": "#/$defs/split"
      },
      "queryId": {
        "$ref": "#/$defs/queryId"
      },
      "additionalInfos": {
        "$ref": "#/$defs/additionalInfos"
      }
    },
    "required": [
      "messageKind",

```

```

        "partitions",
        "queryId",
        "additionalInfos"
    ],
    "additionalProperties": false,
    "minProperties": 4,
    "maxProperties": 5
},
"ListAndSubscribePartitionsRequest": {
    "type": "object",
    "properties": {
        "messageKind": {
            "const": "ListAndSubscribePartitionsRequest"
        },
        "queryId": {
            "$ref": "#/$defs/queryId"
        },
        "additionalInfos": {
            "$ref": "#/$defs/additionalInfos"
        }
    },
    "required": [
        "messageKind",
        "queryId",
        "additionalInfos"
    ],
    "additionalProperties": false,
    "minProperties": 3,
    "maxProperties": 3
},
"ListAndSubscribePartitionsResponse": {
    "type": "object",
    "properties": {
        "messageKind": {
            "const": "ListAndSubscribePartitionsResponse"
        },
        "partitions": {
            "$ref": "#/$defs/DeltaChunk"
        },
        "split": {
            "$ref": "#/$defs/split"
        },
        "queryId": {
            "$ref": "#/$defs/queryId"
        },
        "additionalInfos": {
            "$ref": "#/$defs/additionalInfos"
        }
    },
    "required": [
        "messageKind",

```

```

        "partitions",
        "queryId",
        "additionalInfos"
    ],
    "additionalProperties": false,
    "minProperties": 4,
    "maxProperties": 5
},
"CustomQueryRequest": {
    "type": "object",
    "properties": {
        "messageKind": {
            "$ref": "#/$defs/customMessageKind"
        },
        "queryId": {
            "$ref": "#/$defs/queryId"
        },
        "additionalInfos": {
            "$ref": "#/$defs/additionalInfos"
        }
    },
    "required": [
        "messageKind",
        "queryId",
        "additionalInfos"
    ],
    "additionalProperties": true,
    "minProperties": 3
},
"CustomQueryResponse": {
    "type": "object",
    "properties": {
        "messageKind": {
            "$ref": "#/$defs/customMessageKind"
        },
        "queryId": {
            "$ref": "#/$defs/queryId"
        },
        "additionalInfos": {
            "$ref": "#/$defs/additionalInfos"
        }
    },
    "required": [
        "messageKind",
        "queryId",
        "additionalInfos"
    ],
    "additionalProperties": true,
    "minProperties": 3
},
"ErrorResponse": {

```

```

"type": "object",
"properties": {
  "messageKind": {
    "const": "ErrorResponse"
  },
  "errorCode": {
    "$ref": "#/$defs/errorCode"
  },
  "message": {
    "type": "string"
  },
  "queryId": {
    "$ref": "#/$defs/queryId"
  },
  "additionalInfos": {
    "$ref": "#/$defs/additionalInfos"
  }
},
"required": [
  "messageKind",
  "errorCode",
  "message",
  "queryId",
  "additionalInfos"
],
"additionalProperties": false,
"minProperties": 5,
"maxProperties": 5
},
"ContinuedQueryResponse": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "ContinuedQueryResponse"
    },
    "chunk": {
      "$ref": "#/$defs/DeltaChunk"
    },
    "continuedChunkCompleted": {
      "$ref": "#/$defs/continuedChunkCompleted"
    },
    "continuedChunkSequenceNumber": {
      "$ref": "#/$defs/continuedChunkSequenceNumber"
    },
    "queryId": {
      "$ref": "#/$defs/queryId"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  }
},

```

```

"required": [
  "messageKind",
  "chunk",
  "continuedChunkCompleted",
  "continuedChunkSequenceNumber",
  "queryId",
  "additionalInfos"
],
"additionalProperties": false,
"minProperties": 6,
"maxProperties": 6
},
"Commands": {
  "oneOf": [
    {
      "$ref": "#/$defs/NonContinuedCommands"
    },
    {
      "$ref": "#/$defs/ContinuedCommand"
    }
  ]
},
"NonContinuedCommands": {
  "oneOf": [
    {
      "$ref": "#/$defs/AddPartition"
    },
    {
      "$ref": "#/$defs/DeletePartition"
    },
    {
      "$ref": "#/$defs/ChangeClassifier"
    },
    {
      "$ref": "#/$defs/AddProperty"
    },
    {
      "$ref": "#/$defs/DeleteProperty"
    },
    {
      "$ref": "#/$defs/ChangeProperty"
    },
    {
      "$ref": "#/$defs/AddChild"
    },
    {
      "$ref": "#/$defs/DeleteChild"
    },
    {
      "$ref": "#/$defs/ReplaceChild"
    }
  ],

```

```

{
  "$ref": "#/$defs/MoveChildFromContainmentInOtherParent"
},
{
  "$ref": "#/$defs/MoveChildFromOtherContainmentInSameParent"
},
{
  "$ref": "#/$defs/MoveChildInSameContainmentInSameParent"
},
{
  "$ref": "#/$defs/MoveAndReplaceChildFromContainmentInOtherParent"
},
{
  "$ref": "#/$defs/MoveAndReplaceChildFromOtherContainmentInSameParent"
},
{
  "$ref": "#/$defs/MoveAndReplaceChildInSameContainmentInSameParent"
},
{
  "$ref": "#/$defs/AddAnnotation"
},
{
  "$ref": "#/$defs/DeleteAnnotation"
},
{
  "$ref": "#/$defs/ReplaceAnnotation"
},
{
  "$ref": "#/$defs/MoveAnnotationFromOtherParent"
},
{
  "$ref": "#/$defs/MoveAnnotationInSameParent"
},
{
  "$ref": "#/$defs/MoveAndReplaceAnnotationFromOtherParent"
},
{
  "$ref": "#/$defs/MoveAndReplaceAnnotationInSameParent"
},
{
  "$ref": "#/$defs/AddReference"
},
{
  "$ref": "#/$defs/DeleteReference"
},
{
  "$ref": "#/$defs/ChangeReference"
},
{
  "$ref": "#/$defs/CompositeCommand"
},

```

```

    {
      "$ref": "#/$defs/CustomCommand"
    }
  ]
},
"AddPartition": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "AddPartition"
    },
    "newPartition": {
      "$ref": "#/$defs/DeltaChunk"
    },
    "split": {
      "$ref": "#/$defs/split"
    },
    "commandId": {
      "$ref": "#/$defs/commandId"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "newPartition",
    "commandId",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 4,
  "maxProperties": 5
},
"DeletePartition": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "DeletePartition"
    },
    "deletedPartition": {
      "$ref": "#/$defs/targetNode"
    },
    "commandId": {
      "$ref": "#/$defs/commandId"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [

```

```

        "messageKind",
        "deletedPartition",
        "commandId",
        "additionalInfos"
    ],
    "additionalProperties": false,
    "minProperties": 4,
    "maxProperties": 4
},
"ChangeClassifier": {
    "type": "object",
    "properties": {
        "messageKind": {
            "const": "ChangeClassifier"
        },
        "node": {
            "$ref": "#/$defs/targetNode"
        },
        "newClassifier": {
            "$ref": "#/$defs/metaPointer"
        },
        "commandId": {
            "$ref": "#/$defs/commandId"
        },
        "additionalInfos": {
            "$ref": "#/$defs/additionalInfos"
        }
    },
    "required": [
        "messageKind",
        "node",
        "newClassifier",
        "commandId",
        "additionalInfos"
    ],
    "additionalProperties": false,
    "minProperties": 5,
    "maxProperties": 5
},
"AddProperty": {
    "type": "object",
    "properties": {
        "messageKind": {
            "const": "AddProperty"
        },
        "node": {
            "$ref": "#/$defs/targetNode"
        },
        "property": {
            "$ref": "#/$defs/metaPointer"
        }
    },

```

```

    "newValue": {
      "$ref": "#/$defs/propertyValue"
    },
    "commandId": {
      "$ref": "#/$defs/commandId"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "node",
    "property",
    "newValue",
    "commandId",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 6,
  "maxProperties": 6
},
"DeleteProperty": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "DeleteProperty"
    },
    "node": {
      "$ref": "#/$defs/targetNode"
    },
    "property": {
      "$ref": "#/$defs/metaPointer"
    },
    "commandId": {
      "$ref": "#/$defs/commandId"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "node",
    "property",
    "commandId",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 5,
  "maxProperties": 5

```

```

},
"ChangeProperty": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "ChangeProperty"
    },
    "node": {
      "$ref": "#/$defs/targetNode"
    },
    "property": {
      "$ref": "#/$defs/metaPointer"
    },
    "newValue": {
      "$ref": "#/$defs/propertyValue"
    },
    "commandId": {
      "$ref": "#/$defs/commandId"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "node",
    "property",
    "newValue",
    "commandId",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 6,
  "maxProperties": 6
},
"AddChild": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "AddChild"
    },
    "parent": {
      "$ref": "#/$defs/targetNode"
    },
    "newChild": {
      "$ref": "#/$defs/DeltaChunk"
    },
    "containment": {
      "$ref": "#/$defs/metaPointer"
    },
    "index": {

```

```

    "$ref": "#/$defs/index"
  },
  "split": {
    "$ref": "#/$defs/split"
  },
  "commandId": {
    "$ref": "#/$defs/commandId"
  },
  "additionalInfos": {
    "$ref": "#/$defs/additionalInfos"
  }
},
"required": [
  "messageKind",
  "parent",
  "newChild",
  "containment",
  "index",
  "commandId",
  "additionalInfos"
],
"additionalProperties": false,
"minProperties": 7,
"maxProperties": 8
},
"DeleteChild": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "DeleteChild"
    },
    "parent": {
      "$ref": "#/$defs/targetNode"
    },
    "containment": {
      "$ref": "#/$defs/metaPointer"
    },
    "index": {
      "$ref": "#/$defs/index"
    },
    "deletedChild": {
      "$ref": "#/$defs/targetNode"
    },
    "commandId": {
      "$ref": "#/$defs/commandId"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [

```

```

        "messageKind",
        "parent",
        "containment",
        "index",
        "deletedChild",
        "commandId",
        "additionalInfos"
    ],
    "additionalProperties": false,
    "minProperties": 7,
    "maxProperties": 7
},
"ReplaceChild": {
    "type": "object",
    "properties": {
        "messageKind": {
            "const": "ReplaceChild"
        },
        "parent": {
            "$ref": "#/$defs/targetNode"
        },
        "newChild": {
            "$ref": "#/$defs/DeltaChunk"
        },
        "containment": {
            "$ref": "#/$defs/metaPointer"
        },
        "index": {
            "$ref": "#/$defs/index"
        },
        "replacedChild": {
            "$ref": "#/$defs/targetNode"
        },
        "split": {
            "$ref": "#/$defs/split"
        },
        "commandId": {
            "$ref": "#/$defs/commandId"
        },
        "additionalInfos": {
            "$ref": "#/$defs/additionalInfos"
        }
    },
    "required": [
        "messageKind",
        "parent",
        "newChild",
        "containment",
        "index",
        "replacedChild",
        "commandId",

```

```

    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 8,
  "maxProperties": 9
},
"MoveChildFromContainmentInOtherParent": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "MoveChildFromContainmentInOtherParent"
    },
    "newParent": {
      "$ref": "#/$defs/targetNode"
    },
    "newContainment": {
      "$ref": "#/$defs/metaPointer"
    },
    "newIndex": {
      "$ref": "#/$defs/index"
    },
    "oldParent": {
      "$ref": "#/$defs/targetNode"
    },
    "oldContainment": {
      "$ref": "#/$defs/metaPointer"
    },
    "oldIndex": {
      "$ref": "#/$defs/index"
    },
    "movedChild": {
      "$ref": "#/$defs/targetNode"
    },
    "commandId": {
      "$ref": "#/$defs/commandId"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "newParent",
    "newContainment",
    "newIndex",
    "oldParent",
    "oldIndex",
    "oldContainment",
    "movedChild",
    "commandId",
    "additionalInfos"
  ]
}

```

```

    ],
    "additionalProperties": false,
    "minProperties": 10,
    "maxProperties": 10
  },
  "MoveChildFromOtherContainmentInSameParent": {
    "type": "object",
    "properties": {
      "messageKind": {
        "const": "MoveChildFromOtherContainmentInSameParent"
      },
      "parent": {
        "$ref": "#/$defs/targetNode"
      },
      "newContainment": {
        "$ref": "#/$defs/metaPointer"
      },
      "newIndex": {
        "$ref": "#/$defs/index"
      },
      "oldContainment": {
        "$ref": "#/$defs/metaPointer"
      },
      "oldIndex": {
        "$ref": "#/$defs/index"
      },
      "movedChild": {
        "$ref": "#/$defs/targetNode"
      },
      "commandId": {
        "$ref": "#/$defs/commandId"
      },
      "additionalInfos": {
        "$ref": "#/$defs/additionalInfos"
      }
    },
    "required": [
      "messageKind",
      "parent",
      "newContainment",
      "newIndex",
      "oldContainment",
      "oldIndex",
      "movedChild",
      "commandId",
      "additionalInfos"
    ],
    "additionalProperties": false,
    "minProperties": 9,
    "maxProperties": 9
  },

```

```

"MoveChildInSameContainmentInSameParent": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "MoveChildInSameContainmentInSameParent"
    },
    "parent": {
      "$ref": "#/$defs/targetNode"
    },
    "containment": {
      "$ref": "#/$defs/metaPointer"
    },
    "oldIndex": {
      "$ref": "#/$defs/index"
    },
    "indexOffset": {
      "$ref": "#/$defs/indexOffset"
    },
    "movedChild": {
      "$ref": "#/$defs/targetNode"
    },
    "commandId": {
      "$ref": "#/$defs/commandId"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "parent",
    "containment",
    "oldIndex",
    "indexOffset",
    "movedChild",
    "commandId",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 8,
  "maxProperties": 8
},
"MoveAndReplaceChildFromContainmentInOtherParent": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "MoveAndReplaceChildFromContainmentInOtherParent"
    },
    "newParent": {
      "$ref": "#/$defs/targetNode"
    },
  },

```

```

    "newContainment": {
      "$ref": "#/$defs/metaPointer"
    },
    "newIndex": {
      "$ref": "#/$defs/index"
    },
    "oldParent": {
      "$ref": "#/$defs/targetNode"
    },
    "oldContainment": {
      "$ref": "#/$defs/metaPointer"
    },
    "oldIndex": {
      "$ref": "#/$defs/index"
    },
    "replacedChild": {
      "$ref": "#/$defs/targetNode"
    },
    "movedChild": {
      "$ref": "#/$defs/targetNode"
    },
    "commandId": {
      "$ref": "#/$defs/commandId"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "newParent",
    "newContainment",
    "newIndex",
    "oldParent",
    "oldContainment",
    "oldIndex",
    "replacedChild",
    "movedChild",
    "commandId",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 11,
  "maxProperties": 11
},
"MoveAndReplaceChildFromOtherContainmentInSameParent": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "MoveAndReplaceChildFromOtherContainmentInSameParent"
    },
  },

```

```

    "parent": {
      "$ref": "#/$defs/targetNode"
    },
    "newContainment": {
      "$ref": "#/$defs/metaPointer"
    },
    "newIndex": {
      "$ref": "#/$defs/index"
    },
    "oldContainment": {
      "$ref": "#/$defs/metaPointer"
    },
    "oldIndex": {
      "$ref": "#/$defs/index"
    },
    "replacedChild": {
      "$ref": "#/$defs/targetNode"
    },
    "movedChild": {
      "$ref": "#/$defs/targetNode"
    },
    "commandId": {
      "$ref": "#/$defs/commandId"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "parent",
    "newContainment",
    "newIndex",
    "oldContainment",
    "oldIndex",
    "replacedChild",
    "movedChild",
    "commandId",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 10,
  "maxProperties": 10
},
"MoveAndReplaceChildInSameContainmentInSameParent": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "MoveAndReplaceChildInSameContainmentInSameParent"
    },
    "parent": {

```

```

    "$ref": "#/$defs/targetNode"
  },
  "containment": {
    "$ref": "#/$defs/metaPointer"
  },
  "oldIndex": {
    "$ref": "#/$defs/index"
  },
  "indexOffset": {
    "$ref": "#/$defs/indexOffset"
  },
  "replacedChild": {
    "$ref": "#/$defs/targetNode"
  },
  "movedChild": {
    "$ref": "#/$defs/targetNode"
  },
  "commandId": {
    "$ref": "#/$defs/commandId"
  },
  "additionalInfos": {
    "$ref": "#/$defs/additionalInfos"
  }
},
"required": [
  "messageKind",
  "parent",
  "containment",
  "oldIndex",
  "indexOffset",
  "replacedChild",
  "movedChild",
  "commandId",
  "additionalInfos"
],
"additionalProperties": false,
"minProperties": 9,
"maxProperties": 9
},
"AddAnnotation": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "AddAnnotation"
    },
    "parent": {
      "$ref": "#/$defs/targetNode"
    },
    "newAnnotation": {
      "$ref": "#/$defs/DeltaChunk"
    }
  },

```

```

    "index": {
      "$ref": "#/$defs/index"
    },
    "split": {
      "$ref": "#/$defs/split"
    },
    "commandId": {
      "$ref": "#/$defs/commandId"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "parent",
    "newAnnotation",
    "index",
    "commandId",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 6,
  "maxProperties": 7
},
"DeleteAnnotation": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "DeleteAnnotation"
    },
    "parent": {
      "$ref": "#/$defs/targetNode"
    },
    "index": {
      "$ref": "#/$defs/index"
    },
    "deletedAnnotation": {
      "$ref": "#/$defs/targetNode"
    },
    "commandId": {
      "$ref": "#/$defs/commandId"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "parent",
    "index",

```

```

        "deletedAnnotation",
        "commandId",
        "additionalInfos"
    ],
    "additionalProperties": false,
    "minProperties": 6,
    "maxProperties": 6
},
"ReplaceAnnotation": {
    "type": "object",
    "properties": {
        "messageKind": {
            "const": "ReplaceAnnotation"
        },
        "parent": {
            "$ref": "#/$defs/targetNode"
        },
        "newAnnotation": {
            "$ref": "#/$defs/DeltaChunk"
        },
        "index": {
            "$ref": "#/$defs/index"
        },
        "replacedAnnotation": {
            "$ref": "#/$defs/targetNode"
        },
        "split": {
            "$ref": "#/$defs/split"
        },
        "commandId": {
            "$ref": "#/$defs/commandId"
        },
        "additionalInfos": {
            "$ref": "#/$defs/additionalInfos"
        }
    },
    "required": [
        "messageKind",
        "parent",
        "newAnnotation",
        "index",
        "replacedAnnotation",
        "commandId",
        "additionalInfos"
    ],
    "additionalProperties": false,
    "minProperties": 7,
    "maxProperties": 8
},
"MoveAnnotationFromOtherParent": {
    "type": "object",

```

```

"properties": {
  "messageKind": {
    "const": "MoveAnnotationFromOtherParent"
  },
  "newParent": {
    "$ref": "#/$defs/targetNode"
  },
  "newIndex": {
    "$ref": "#/$defs/index"
  },
  "oldParent": {
    "$ref": "#/$defs/targetNode"
  },
  "oldIndex": {
    "$ref": "#/$defs/index"
  },
  "movedAnnotation": {
    "$ref": "#/$defs/targetNode"
  },
  "commandId": {
    "$ref": "#/$defs/commandId"
  },
  "additionalInfos": {
    "$ref": "#/$defs/additionalInfos"
  }
},
"required": [
  "messageKind",
  "newParent",
  "newIndex",
  "oldParent",
  "oldIndex",
  "movedAnnotation",
  "commandId",
  "additionalInfos"
],
"additionalProperties": false,
"minProperties": 8,
"maxProperties": 8
},
"MoveAnnotationInSameParent": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "MoveAnnotationInSameParent"
    },
    "parent": {
      "$ref": "#/$defs/targetNode"
    },
    "oldIndex": {
      "$ref": "#/$defs/index"
    }
  }
}

```

```

    },
    "indexOffset": {
      "$ref": "#/$defs/indexOffset"
    },
    "movedAnnotation": {
      "$ref": "#/$defs/targetNode"
    },
    "commandId": {
      "$ref": "#/$defs/commandId"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "parent",
    "oldIndex",
    "indexOffset",
    "movedAnnotation",
    "commandId",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 7,
  "maxProperties": 7
},
"MoveAndReplaceAnnotationFromOtherParent": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "MoveAndReplaceAnnotationFromOtherParent"
    },
    "newParent": {
      "$ref": "#/$defs/targetNode"
    },
    "newIndex": {
      "$ref": "#/$defs/index"
    },
    "oldParent": {
      "$ref": "#/$defs/targetNode"
    },
    "oldIndex": {
      "$ref": "#/$defs/index"
    },
    "replacedAnnotation": {
      "$ref": "#/$defs/targetNode"
    },
    "movedAnnotation": {
      "$ref": "#/$defs/targetNode"
    }
  },

```

```

    "commandId": {
      "$ref": "#/$defs/commandId"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "newParent",
    "newIndex",
    "oldParent",
    "oldIndex",
    "replacedAnnotation",
    "movedAnnotation",
    "commandId",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 9,
  "maxProperties": 9
},
"MoveAndReplaceAnnotationInSameParent": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "MoveAndReplaceAnnotationInSameParent"
    },
    "parent": {
      "$ref": "#/$defs/targetNode"
    },
    "oldIndex": {
      "$ref": "#/$defs/index"
    },
    "indexOffset": {
      "$ref": "#/$defs/indexOffset"
    },
    "replacedAnnotation": {
      "$ref": "#/$defs/targetNode"
    },
    "movedAnnotation": {
      "$ref": "#/$defs/targetNode"
    },
    "commandId": {
      "$ref": "#/$defs/commandId"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [

```

```

        "messageKind",
        "parent",
        "oldIndex",
        "indexOffset",
        "replacedAnnotation",
        "movedAnnotation",
        "commandId",
        "additionalInfos"
    ],
    "additionalProperties": false,
    "minProperties": 8,
    "maxProperties": 8
},
"AddReference": {
    "type": "object",
    "properties": {
        "messageKind": {
            "const": "AddReference"
        },
        "parent": {
            "$ref": "#/$defs/targetNode"
        },
        "reference": {
            "$ref": "#/$defs/metaPointer"
        },
        "index": {
            "$ref": "#/$defs/index"
        },
        "newReference": {
            "$ref": "#/$defs/targetNode"
        },
        "newResolveInfo": {
            "$ref": "#/$defs/resolveInfo"
        },
        "commandId": {
            "$ref": "#/$defs/commandId"
        },
        "additionalInfos": {
            "$ref": "#/$defs/additionalInfos"
        }
    },
    "required": [
        "messageKind",
        "parent",
        "reference",
        "index",
        "commandId",
        "additionalInfos"
    ],
    "additionalProperties": false,
    "minProperties": 6,

```

```

    "maxProperties": 8
  },
  "DeleteReference": {
    "type": "object",
    "properties": {
      "messageKind": {
        "const": "DeleteReference"
      },
      "parent": {
        "$ref": "#/$defs/targetNode"
      },
      "reference": {
        "$ref": "#/$defs/metaPointer"
      },
      "index": {
        "$ref": "#/$defs/index"
      },
      "deletedReference": {
        "$ref": "#/$defs/targetNode"
      },
      "deletedResolveInfo": {
        "$ref": "#/$defs/resolveInfo"
      },
      "commandId": {
        "$ref": "#/$defs/commandId"
      },
      "additionalInfos": {
        "$ref": "#/$defs/additionalInfos"
      }
    },
    "required": [
      "messageKind",
      "parent",
      "reference",
      "index",
      "commandId",
      "additionalInfos"
    ],
    "additionalProperties": false,
    "minProperties": 6,
    "maxProperties": 8
  },
  "ChangeReference": {
    "type": "object",
    "properties": {
      "messageKind": {
        "const": "ChangeReference"
      },
      "parent": {
        "$ref": "#/$defs/targetNode"
      },
    },

```

```

    "reference": {
      "$ref": "#/$defs/metaPointer"
    },
    "index": {
      "$ref": "#/$defs/index"
    },
    "oldReference": {
      "$ref": "#/$defs/targetNode"
    },
    "oldResolveInfo": {
      "$ref": "#/$defs/resolveInfo"
    },
    "newReference": {
      "$ref": "#/$defs/targetNode"
    },
    "newResolveInfo": {
      "$ref": "#/$defs/resolveInfo"
    },
    "commandId": {
      "$ref": "#/$defs/commandId"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "parent",
    "reference",
    "index",
    "commandId",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 6,
  "maxProperties": 10
},
"CompositeCommand": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "CompositeCommand"
    },
    "parts": {
      "type": "array",
      "items": {
        "$ref": "#/$defs/NonContinuedCommands"
      }
    },
    "commandId": {
      "$ref": "#/$defs/commandId"
    }
  }
}

```

```

    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "parts",
    "commandId",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 4,
  "maxProperties": 4
},
"ContinuedCommand": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "ContinuedCommand"
    },
    "chunk": {
      "$ref": "#/$defs/DeltaChunk"
    },
    "continuedChunkCompleted": {
      "$ref": "#/$defs/continuedChunkCompleted"
    },
    "continuedChunkSequenceNumber": {
      "$ref": "#/$defs/continuedChunkSequenceNumber"
    },
    "commandId": {
      "$ref": "#/$defs/commandId"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "chunk",
    "continuedChunkCompleted",
    "continuedChunkSequenceNumber",
    "commandId",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 6,
  "maxProperties": 6
},
"CustomCommand": {
  "type": "object",

```

```

"properties": {
  "messageKind": {
    "$ref": "#/$defs/customMessageKind"
  },
  "commandId": {
    "$ref": "#/$defs/commandId"
  },
  "additionalInfos": {
    "$ref": "#/$defs/additionalInfos"
  }
},
"required": [
  "messageKind",
  "commandId",
  "additionalInfos"
],
"additionalProperties": true,
"minProperties": 3
},
"Events": {
  "oneOf": [
    {
      "$ref": "#/$defs/NonContinuedEvents"
    },
    {
      "$ref": "#/$defs/ContinuedEvent"
    }
  ]
},
"NonContinuedEvents": {
  "oneOf": [
    {
      "$ref": "#/$defs/PartitionAdded"
    },
    {
      "$ref": "#/$defs/PartitionDeleted"
    },
    {
      "$ref": "#/$defs/ClassifierChanged"
    },
    {
      "$ref": "#/$defs/PropertyAdded"
    },
    {
      "$ref": "#/$defs/PropertyDeleted"
    },
    {
      "$ref": "#/$defs/PropertyChanged"
    },
    {
      "$ref": "#/$defs/ChildAdded"
    }
  ]
}

```

```

},
{
  "$ref": "#/$defs/ChildDeleted"
},
{
  "$ref": "#/$defs/ChildReplaced"
},
{
  "$ref": "#/$defs/ChildMovedFromContainmentInOtherParent"
},
{
  "$ref": "#/$defs/ChildMovedFromOtherContainmentInSameParent"
},
{
  "$ref": "#/$defs/ChildMovedInSameContainmentInSameParent"
},
{
  "$ref": "#/$defs/ChildMovedAndReplacedFromContainmentInOtherParent"
},
{
  "$ref": "#/$defs/ChildMovedAndReplacedFromOtherContainmentInSameParent"
},
{
  "$ref": "#/$defs/ChildMovedAndReplacedInSameContainmentInSameParent"
},
{
  "$ref": "#/$defs/AnnotationAdded"
},
{
  "$ref": "#/$defs/AnnotationDeleted"
},
{
  "$ref": "#/$defs/AnnotationReplaced"
},
{
  "$ref": "#/$defs/AnnotationMovedFromOtherParent"
},
{
  "$ref": "#/$defs/AnnotationMovedInSameParent"
},
{
  "$ref": "#/$defs/AnnotationMovedAndReplacedFromOtherParent"
},
{
  "$ref": "#/$defs/AnnotationMovedAndReplacedInSameParent"
},
{
  "$ref": "#/$defs/ReferenceAdded"
},
{
  "$ref": "#/$defs/ReferenceDeleted"
}

```

```

    },
    {
      "$ref": "#/$defs/ReferenceChanged"
    },
    {
      "$ref": "#/$defs/NoOpEvent"
    },
    {
      "$ref": "#/$defs/CompositeEvent"
    },
    {
      "$ref": "#/$defs/ErrorEvent"
    },
    {
      "$ref": "#/$defs/CustomEvent"
    }
  ]
},
"PartitionAdded": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "PartitionAdded"
    },
    "newPartition": {
      "$ref": "#/$defs/DeltaChunk"
    },
    "split": {
      "$ref": "#/$defs/split"
    },
    "originCommands": {
      "$ref": "#/$defs/commandSources"
    },
    "sequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "newPartition",
    "originCommands",
    "sequenceNumber",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 5,
  "maxProperties": 6
},

```

```

"PartitionDeleted": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "PartitionDeleted"
    },
    "deletedPartition": {
      "$ref": "#/$defs/targetNode"
    },
    "deletedDescendants": {
      "$ref": "#/$defs/targetNodes"
    },
    "originCommands": {
      "$ref": "#/$defs/commandSources"
    },
    "sequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "deletedPartition",
    "deletedDescendants",
    "originCommands",
    "sequenceNumber",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 6,
  "maxProperties": 6
},
"ClassifierChanged": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "ClassifierChanged"
    },
    "node": {
      "$ref": "#/$defs/targetNode"
    },
    "newClassifier": {
      "$ref": "#/$defs/metaPointer"
    },
    "oldClassifier": {
      "$ref": "#/$defs/metaPointer"
    },
    "originCommands": {
      "$ref": "#/$defs/commandSources"
    }
  }
}

```

```

    },
    "sequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "node",
    "newClassifier",
    "oldClassifier",
    "originCommands",
    "sequenceNumber",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 7,
  "maxProperties": 7
},
"PropertyAdded": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "PropertyAdded"
    },
    "node": {
      "$ref": "#/$defs/targetNode"
    },
    "property": {
      "$ref": "#/$defs/metaPointer"
    },
    "newValue": {
      "$ref": "#/$defs/propertyValue"
    },
    "originCommands": {
      "$ref": "#/$defs/commandSources"
    },
    "sequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "node",
    "property",
    "newValue",

```

```

        "originCommands",
        "sequenceNumber",
        "additionalInfos"
    ],
    "additionalProperties": false,
    "minProperties": 7,
    "maxProperties": 7
},
"PropertyDeleted": {
    "type": "object",
    "properties": {
        "messageKind": {
            "const": "PropertyDeleted"
        },
        "node": {
            "$ref": "#/$defs/targetNode"
        },
        "property": {
            "$ref": "#/$defs/metaPointer"
        },
        "oldValue": {
            "$ref": "#/$defs/propertyValue"
        },
        "originCommands": {
            "$ref": "#/$defs/commandSources"
        },
        "sequenceNumber": {
            "$ref": "#/$defs/eventSequenceNumber"
        },
        "additionalInfos": {
            "$ref": "#/$defs/additionalInfos"
        }
    },
    "required": [
        "messageKind",
        "node",
        "property",
        "oldValue",
        "originCommands",
        "sequenceNumber",
        "additionalInfos"
    ],
    "additionalProperties": false,
    "minProperties": 7,
    "maxProperties": 7
},
"PropertyChanged": {
    "type": "object",
    "properties": {
        "messageKind": {
            "const": "PropertyChanged"

```

```

    },
    "node": {
      "$ref": "#/$defs/targetNode"
    },
    "property": {
      "$ref": "#/$defs/metaPointer"
    },
    "oldValue": {
      "$ref": "#/$defs/propertyValue"
    },
    "newValue": {
      "$ref": "#/$defs/propertyValue"
    },
    "originCommands": {
      "$ref": "#/$defs/commandSources"
    },
    "sequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "node",
    "property",
    "oldValue",
    "newValue",
    "originCommands",
    "sequenceNumber",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 8,
  "maxProperties": 8
},
"ChildAdded": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "ChildAdded"
    },
    "parent": {
      "$ref": "#/$defs/targetNode"
    },
    "newChild": {
      "$ref": "#/$defs/DeltaChunk"
    },
    "containment": {
      "$ref": "#/$defs/metaPointer"
    }
  }
}

```

```

    },
    "index": {
      "$ref": "#/$defs/index"
    },
    "split": {
      "$ref": "#/$defs/split"
    },
    "originCommands": {
      "$ref": "#/$defs/commandSources"
    },
    "sequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "parent",
    "newChild",
    "containment",
    "index",
    "originCommands",
    "sequenceNumber",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 8,
  "maxProperties": 9
},
"ChildDeleted": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "ChildDeleted"
    },
    "deletedChild": {
      "$ref": "#/$defs/targetNode"
    },
    "deletedDescendants": {
      "$ref": "#/$defs/targetNodes"
    },
    "parent": {
      "$ref": "#/$defs/targetNode"
    },
    "containment": {
      "$ref": "#/$defs/metaPointer"
    },
    "index": {
      "$ref": "#/$defs/index"
    }
  }
}

```

```

    },
    "originCommands": {
      "$ref": "#/$defs/commandSources"
    },
    },
    "sequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "deletedChild",
    "deletedDescendants",
    "parent",
    "containment",
    "index",
    "originCommands",
    "sequenceNumber",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 9,
  "maxProperties": 9
},
"ChildReplaced": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "ChildReplaced"
    },
  },
  "newChild": {
    "$ref": "#/$defs/DeltaChunk"
  },
  },
  "replacedChild": {
    "$ref": "#/$defs/targetNode"
  },
  },
  "replacedDescendants": {
    "$ref": "#/$defs/targetNodes"
  },
  },
  "parent": {
    "$ref": "#/$defs/targetNode"
  },
  },
  "containment": {
    "$ref": "#/$defs/metaPointer"
  },
  },
  "index": {
    "$ref": "#/$defs/index"
  },
  },
  "split": {

```

```

    "$ref": "#/$defs/split"
  },
  "originCommands": {
    "$ref": "#/$defs/commandSources"
  },
  "sequenceNumber": {
    "$ref": "#/$defs/eventSequenceNumber"
  },
  "additionalInfos": {
    "$ref": "#/$defs/additionalInfos"
  }
},
"required": [
  "messageKind",
  "newChild",
  "replacedChild",
  "replacedDescendants",
  "parent",
  "containment",
  "index",
  "originCommands",
  "sequenceNumber",
  "additionalInfos"
],
"additionalProperties": false,
"minProperties": 10,
"maxProperties": 11
},
"ChildMovedFromContainmentInOtherParent": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "ChildMovedFromContainmentInOtherParent"
    },
    "newParent": {
      "$ref": "#/$defs/targetNode"
    },
    "newContainment": {
      "$ref": "#/$defs/metaPointer"
    },
    "newIndex": {
      "$ref": "#/$defs/index"
    },
    "movedChild": {
      "$ref": "#/$defs/targetNode"
    },
    "oldParent": {
      "$ref": "#/$defs/targetNode"
    },
    "oldContainment": {
      "$ref": "#/$defs/metaPointer"
    }
  }
}

```

```

    },
    "oldIndex": {
      "$ref": "#/$defs/index"
    },
    },
    "originCommands": {
      "$ref": "#/$defs/commandSources"
    },
    },
    "sequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "newParent",
    "newContainment",
    "newIndex",
    "movedChild",
    "oldParent",
    "oldContainment",
    "oldIndex",
    "originCommands",
    "sequenceNumber",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 11,
  "maxProperties": 11
},
"ChildMovedFromOtherContainmentInSameParent": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "ChildMovedFromOtherContainmentInSameParent"
    },
    },
    "newContainment": {
      "$ref": "#/$defs/metaPointer"
    },
    },
    "newIndex": {
      "$ref": "#/$defs/index"
    },
    },
    "movedChild": {
      "$ref": "#/$defs/targetNode"
    },
    },
    "parent": {
      "$ref": "#/$defs/targetNode"
    },
    },
    "oldContainment": {
      "$ref": "#/$defs/metaPointer"
    }
  }
}

```

```

    },
    "oldIndex": {
      "$ref": "#/$defs/index"
    },
    },
    "originCommands": {
      "$ref": "#/$defs/commandSources"
    },
    },
    "sequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "newContainment",
    "newIndex",
    "movedChild",
    "parent",
    "oldContainment",
    "oldIndex",
    "originCommands",
    "sequenceNumber",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 10,
  "maxProperties": 10
},
"ChildMovedInSameContainmentInSameParent": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "ChildMovedInSameContainmentInSameParent"
    },
    },
    "movedChild": {
      "$ref": "#/$defs/targetNode"
    },
    },
    "parent": {
      "$ref": "#/$defs/targetNode"
    },
    },
    "containment": {
      "$ref": "#/$defs/metaPointer"
    },
    },
    "oldIndex": {
      "$ref": "#/$defs/index"
    },
    },
    "indexOffset": {
      "$ref": "#/$defs/indexOffset"
    },
    },

```

```

    "originCommands": {
      "$ref": "#/$defs/commandSources"
    },
    "sequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "movedChild",
    "parent",
    "containment",
    "oldIndex",
    "indexOffset",
    "originCommands",
    "sequenceNumber",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 9,
  "maxProperties": 9
},
"ChildMovedAndReplacedFromContainmentInOtherParent": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "ChildMovedAndReplacedFromContainmentInOtherParent"
    },
    "newParent": {
      "$ref": "#/$defs/targetNode"
    },
    "newContainment": {
      "$ref": "#/$defs/metaPointer"
    },
    "newIndex": {
      "$ref": "#/$defs/index"
    },
    "movedChild": {
      "$ref": "#/$defs/targetNode"
    },
    "oldParent": {
      "$ref": "#/$defs/targetNode"
    },
    "oldContainment": {
      "$ref": "#/$defs/metaPointer"
    },
    "oldIndex": {
      "$ref": "#/$defs/index"
    }
  }
}

```

```

    },
    "replacedChild": {
      "$ref": "#/$defs/targetNode"
    },
    "replacedDescendants": {
      "$ref": "#/$defs/targetNodes"
    },
    "originCommands": {
      "$ref": "#/$defs/commandSources"
    },
    "sequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "newParent",
    "newContainment",
    "newIndex",
    "movedChild",
    "oldParent",
    "oldContainment",
    "oldIndex",
    "replacedChild",
    "replacedDescendants",
    "originCommands",
    "sequenceNumber",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 13,
  "maxProperties": 13
},
"ChildMovedAndReplacedFromOtherContainmentInSameParent": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "ChildMovedAndReplacedFromOtherContainmentInSameParent"
    },
    "newContainment": {
      "$ref": "#/$defs/metaPointer"
    },
    "newIndex": {
      "$ref": "#/$defs/index"
    },
    "movedChild": {
      "$ref": "#/$defs/targetNode"
    }
  },

```

```

    "parent": {
      "$ref": "#/$defs/targetNode"
    },
    "oldContainment": {
      "$ref": "#/$defs/metaPointer"
    },
    "oldIndex": {
      "$ref": "#/$defs/index"
    },
    "replacedChild": {
      "$ref": "#/$defs/targetNode"
    },
    "replacedDescendants": {
      "$ref": "#/$defs/targetNodes"
    },
    "originCommands": {
      "$ref": "#/$defs/commandSources"
    },
    "sequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "newContainment",
    "newIndex",
    "movedChild",
    "parent",
    "oldContainment",
    "oldIndex",
    "replacedChild",
    "replacedDescendants",
    "originCommands",
    "sequenceNumber",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 12,
  "maxProperties": 12
},
"ChildMovedAndReplacedInSameContainmentInSameParent": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "ChildMovedAndReplacedInSameContainmentInSameParent"
    },
    "movedChild": {
      "$ref": "#/$defs/targetNode"
    }
  }
}

```

```

    },
    "parent": {
      "$ref": "#/$defs/targetNode"
    },
    "containment": {
      "$ref": "#/$defs/metaPointer"
    },
    "oldIndex": {
      "$ref": "#/$defs/index"
    },
    "indexOffset": {
      "$ref": "#/$defs/indexOffset"
    },
    "replacedChild": {
      "$ref": "#/$defs/targetNode"
    },
    "replacedDescendants": {
      "$ref": "#/$defs/targetNodes"
    },
    "originCommands": {
      "$ref": "#/$defs/commandSources"
    },
    "sequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "movedChild",
    "parent",
    "containment",
    "oldIndex",
    "indexOffset",
    "replacedChild",
    "replacedDescendants",
    "originCommands",
    "sequenceNumber",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 11,
  "maxProperties": 11
},
"AnnotationAdded": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "AnnotationAdded"
    }
  }
}

```

```

    },
    "parent": {
      "$ref": "#/$defs/targetNode"
    },
    "newAnnotation": {
      "$ref": "#/$defs/DeltaChunk"
    },
    "index": {
      "$ref": "#/$defs/index"
    },
    "split": {
      "$ref": "#/$defs/split"
    },
    "originCommands": {
      "$ref": "#/$defs/commandSources"
    },
    "sequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "parent",
    "newAnnotation",
    "index",
    "originCommands",
    "sequenceNumber",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 7,
  "maxProperties": 8
},
"AnnotationDeleted": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "AnnotationDeleted"
    },
    "parent": {
      "$ref": "#/$defs/targetNode"
    },
    "deletedAnnotation": {
      "$ref": "#/$defs/targetNode"
    },
    "deletedDescendants": {
      "$ref": "#/$defs/targetNodes"
    }
  },

```

```

    "index": {
      "$ref": "#/$defs/index"
    },
    "originCommands": {
      "$ref": "#/$defs/commandSources"
    },
    "sequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "parent",
    "deletedAnnotation",
    "deletedDescendants",
    "index",
    "originCommands",
    "sequenceNumber",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 8,
  "maxProperties": 8
},
"AnnotationReplaced": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "AnnotationReplaced"
    },
    "newAnnotation": {
      "$ref": "#/$defs/DeltaChunk"
    },
    "replacedAnnotation": {
      "$ref": "#/$defs/targetNode"
    },
    "replacedDescendants": {
      "$ref": "#/$defs/targetNodes"
    },
    "parent": {
      "$ref": "#/$defs/targetNode"
    },
    "index": {
      "$ref": "#/$defs/index"
    },
    "split": {
      "$ref": "#/$defs/split"
    }
  },

```

```

    "originCommands": {
      "$ref": "#/$defs/commandSources"
    },
    "sequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "newAnnotation",
    "replacedAnnotation",
    "replacedDescendants",
    "parent",
    "index",
    "originCommands",
    "sequenceNumber",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 9,
  "maxProperties": 10
},
"AnnotationMovedFromOtherParent": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "AnnotationMovedFromOtherParent"
    },
    "newParent": {
      "$ref": "#/$defs/targetNode"
    },
    "newIndex": {
      "$ref": "#/$defs/index"
    },
    "movedAnnotation": {
      "$ref": "#/$defs/targetNode"
    },
    "oldParent": {
      "$ref": "#/$defs/targetNode"
    },
    "oldIndex": {
      "$ref": "#/$defs/index"
    },
    "originCommands": {
      "$ref": "#/$defs/commandSources"
    },
    "sequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    }
  }
}

```

```

    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "newParent",
    "newIndex",
    "movedAnnotation",
    "oldParent",
    "oldIndex",
    "originCommands",
    "sequenceNumber",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 9,
  "maxProperties": 9
},
"AnnotationMovedInSameParent": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "AnnotationMovedInSameParent"
    },
    "movedAnnotation": {
      "$ref": "#/$defs/targetNode"
    },
    "parent": {
      "$ref": "#/$defs/targetNode"
    },
    "oldIndex": {
      "$ref": "#/$defs/index"
    },
    "indexOffset": {
      "$ref": "#/$defs/indexOffset"
    },
    "originCommands": {
      "$ref": "#/$defs/commandSources"
    },
    "sequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "movedAnnotation",

```

```

        "parent",
        "oldIndex",
        "indexOffset",
        "originCommands",
        "sequenceNumber",
        "additionalInfos"
    ],
    "additionalProperties": false,
    "minProperties": 8,
    "maxProperties": 8
},
"AnnotationMovedAndReplacedFromOtherParent": {
    "type": "object",
    "properties": {
        "messageKind": {
            "const": "AnnotationMovedAndReplacedFromOtherParent"
        },
        "newParent": {
            "$ref": "#/$defs/targetNode"
        },
        "newIndex": {
            "$ref": "#/$defs/index"
        },
        "movedAnnotation": {
            "$ref": "#/$defs/targetNode"
        },
        "oldParent": {
            "$ref": "#/$defs/targetNode"
        },
        "oldIndex": {
            "$ref": "#/$defs/index"
        },
        "replacedAnnotation": {
            "$ref": "#/$defs/targetNode"
        },
        "replacedDescendants": {
            "$ref": "#/$defs/targetNodes"
        },
        "originCommands": {
            "$ref": "#/$defs/commandSources"
        },
        "sequenceNumber": {
            "$ref": "#/$defs/eventSequenceNumber"
        },
        "additionalInfos": {
            "$ref": "#/$defs/additionalInfos"
        }
    },
    "required": [
        "messageKind",
        "newParent",

```

```

    "newIndex",
    "movedAnnotation",
    "oldParent",
    "oldIndex",
    "replacedAnnotation",
    "replacedDescendants",
    "originCommands",
    "sequenceNumber",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 11,
  "maxProperties": 11
},
"AnnotationMovedAndReplacedInSameParent": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "AnnotationMovedAndReplacedInSameParent"
    },
    "movedAnnotation": {
      "$ref": "#/$defs/targetNode"
    },
    "parent": {
      "$ref": "#/$defs/targetNode"
    },
    "oldIndex": {
      "$ref": "#/$defs/index"
    },
    "indexOffset": {
      "$ref": "#/$defs/indexOffset"
    },
    "replacedAnnotation": {
      "$ref": "#/$defs/targetNode"
    },
    "replacedDescendants": {
      "$ref": "#/$defs/targetNodes"
    },
    "originCommands": {
      "$ref": "#/$defs/commandSources"
    },
    "sequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "movedAnnotation",

```

```

    "parent",
    "oldIndex",
    "indexOffset",
    "replacedAnnotation",
    "replacedDescendants",
    "originCommands",
    "sequenceNumber",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 10,
  "maxProperties": 10
},
"ReferenceAdded": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "ReferenceAdded"
    },
    "parent": {
      "$ref": "#/$defs/targetNode"
    },
    "reference": {
      "$ref": "#/$defs/metaPointer"
    },
    "index": {
      "$ref": "#/$defs/index"
    },
    "newReference": {
      "$ref": "#/$defs/targetNode"
    },
    "newResolveInfo": {
      "$ref": "#/$defs/resolveInfo"
    },
    "originCommands": {
      "$ref": "#/$defs/commandSources"
    },
    "sequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  }
},
"required": [
  "messageKind",
  "parent",
  "reference",
  "index",
  "originCommands",
  "sequenceNumber",

```

```

    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 7,
  "maxProperties": 9
},
"ReferenceDeleted": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "ReferenceDeleted"
    },
    "parent": {
      "$ref": "#/$defs/targetNode"
    },
    "reference": {
      "$ref": "#/$defs/metaPointer"
    },
    "index": {
      "$ref": "#/$defs/index"
    },
    "deletedReference": {
      "$ref": "#/$defs/targetNode"
    },
    "deletedResolveInfo": {
      "$ref": "#/$defs/resolveInfo"
    },
    "originCommands": {
      "$ref": "#/$defs/commandSources"
    },
    "sequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "parent",
    "reference",
    "index",
    "originCommands",
    "sequenceNumber",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 7,
  "maxProperties": 9
},
"ReferenceChanged": {

```

```

"type": "object",
"properties": {
  "messageKind": {
    "const": "ReferenceChanged"
  },
  "parent": {
    "$ref": "#/$defs/targetNode"
  },
  "reference": {
    "$ref": "#/$defs/metaPointer"
  },
  "index": {
    "$ref": "#/$defs/index"
  },
  "newReference": {
    "$ref": "#/$defs/targetNode"
  },
  "newResolveInfo": {
    "$ref": "#/$defs/resolveInfo"
  },
  "oldReference": {
    "$ref": "#/$defs/targetNode"
  },
  "oldResolveInfo": {
    "$ref": "#/$defs/resolveInfo"
  },
  "originCommands": {
    "$ref": "#/$defs/commandSources"
  },
  "sequenceNumber": {
    "$ref": "#/$defs/eventSequenceNumber"
  },
  "additionalInfos": {
    "$ref": "#/$defs/additionalInfos"
  }
},
"required": [
  "messageKind",
  "parent",
  "reference",
  "index",
  "originCommands",
  "sequenceNumber",
  "additionalInfos"
],
"additionalProperties": false,
"minProperties": 7,
"maxProperties": 11
},
"CompositeEvent": {
  "type": "object",

```

```

"properties": {
  "messageKind": {
    "const": "CompositeEvent"
  },
  "parts": {
    "type": "array",
    "items": {
      "$ref": "#/$defs/NonContinuedEvents"
    }
  },
  "originCommands": {
    "$ref": "#/$defs/commandSources"
  },
  "sequenceNumber": {
    "$ref": "#/$defs/eventSequenceNumber"
  },
  "additionalInfos": {
    "$ref": "#/$defs/additionalInfos"
  }
},
"required": [
  "messageKind",
  "parts",
  "originCommands",
  "sequenceNumber",
  "additionalInfos"
],
"additionalProperties": false,
"minProperties": 5,
"maxProperties": 5
},
"NoOpEvent": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "NoOpEvent"
    },
    "originCommands": {
      "$ref": "#/$defs/commandSources"
    },
    "sequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "originCommands",
    "sequenceNumber",

```

```

    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 4,
  "maxProperties": 4
},
"ErrorEvent": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "ErrorEvent"
    },
    "errorCode": {
      "$ref": "#/$defs/errorCode"
    },
    "message": {
      "type": "string"
    },
    "originCommands": {
      "$ref": "#/$defs/commandSources"
    },
    "sequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "errorCode",
    "message",
    "originCommands",
    "sequenceNumber",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 6,
  "maxProperties": 6
},
"ContinuedEvent": {
  "type": "object",
  "properties": {
    "messageKind": {
      "const": "ContinuedEvent"
    },
    "chunk": {
      "$ref": "#/$defs/DeltaChunk"
    },
    "continuedChunkCompleted": {
      "$ref": "#/$defs/continuedChunkCompleted"
    }
  }
}

```

```

    },
    "continuedChunkSequenceNumber": {
      "$ref": "#/$defs/continuedChunkSequenceNumber"
    },
    "continuedEventSequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    "originCommands": {
      "$ref": "#/$defs/commandSources"
    },
    "sequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "chunk",
    "continuedChunkCompleted",
    "continuedChunkSequenceNumber",
    "continuedEventSequenceNumber",
    "originCommands",
    "sequenceNumber",
    "additionalInfos"
  ],
  "additionalProperties": false,
  "minProperties": 8,
  "maxProperties": 8
},
"CustomEvent": {
  "type": "object",
  "properties": {
    "messageKind": {
      "$ref": "#/$defs/customMessageKind"
    },
    "originCommands": {
      "$ref": "#/$defs/commandSources"
    },
    "sequenceNumber": {
      "$ref": "#/$defs/eventSequenceNumber"
    },
    "additionalInfos": {
      "$ref": "#/$defs/additionalInfos"
    }
  },
  "required": [
    "messageKind",
    "originCommands",
    "sequenceNumber",

```

```

        "additionalInfos"
    ],
    "additionalProperties": true,
    "minProperties": 4
}
}
}

```

- [1] Define Delta Participation (was: Session) #337
- [2] https://en.wikipedia.org/wiki/Command_Query_Responsibility_Segregation
- [3] Do we separate basic model sync ("delta") from updates on changes ("notification") a la CQRS? #291
- [4] Is participation handling a query or something else? #350
- [5] Each event must list all command ids it results from #306
- [6] Repo must send all updates to all clients #299
- [7] How fine-grained do we want to split up commands? #317
- [8] How fine-grained do we want reference commands? #329
- [9] Refactor and Simplify Reference Delta's #431
- [10] Moving a child between parents, replacing a child at the destination #321
- [11] Clarifying terms "repository" vs. "server" vs. "forest" #401
- [12] Repository takes the final decision on any change to its contents #339
- [13] Repository doesn't maintain an understanding of the state of a client's model #297
- [14] Should we have an event sequence number? #341
- [15] Assumed connection properties of default delta serialization #429
- [16] <https://websockets.spec.whatwg.org/>
- [17] How do LW specs relate to specific bindings / implementations #428
- [18] Channel MUST guarantee delivering messages in the order sent #485
- [19] Do we always apply commands of one client in the order they arrive at the repo? #484
- [20] Can a client ask the repo if the participation is still alive? #349
- [21] Event sequence numbers MUST start with 1 for each participation #509
- [22] Scope of event sequence number #343
- [23] How does "physical" message ordering relate to EventSequenceNumber? #483
- [24] How to specify subscription scope? #269
- [25] ChangingPartitions subscriptions questions #438
- [26] Can a client change a node it is not subscribed to? #352
- [27] Each command has a unique id #305
- [28] Do we need a dedicated eventId? #423
- [29] Align ChunkedEvent with other continued messages? #475
- [30] Events should have at least 1 origin command (+ unicity of their IDs) #415
- [31] Who assigns participation id? #345
- [32] Repo must tell client if command applied successfully #298
- [33] Discuss move errors #516
- [34] How to respond to commands that lead to no-op? #511
- [35] Do we broadcast ErrorEvent to everybody or only the originating client? #379
- [36] Provide partition contents only on explicit request #425
- [37] How to merge Continued chunks? #473
- [38] Clients MUST process continued messages in chunk order #529
- [39] Disallow continued chunks inside composites? #462
- [40] Do we allow additional data on events? #331
- [41] Do we allow additional data on commands? #332
- [42] Rename + unify ProtocolMessage? #430

- [43] How do ProtocolMessages of command/event, or queryRequest/queryResponse relate? #421
- [44] Represent additional info data as map / dictionary #471
- [45] We assume all our lists to be ordered.
- [46] Clarify use of "?" for parameters in specification #427
- [47] Representation of empty protocol messages part of delta messages #402
- [48] Does DeltaSerializationChunk have the same format as bulk SerializationChunk? #358
- [49] Rename SerializationChunk to DeltaChunk in delta #474
- [50] Do we allow to add more than one node per AddChild command? #327
- [51] spec `depthLimit` range as $[0..2^{31}-1]$ #535
- [52] Should `ContinuedChunkSequenceNumber` start with 1? #532
- [53] Clarify semantics of Replace (with new content) commands #422
- [54] Clarify CompositeCommand / CompositeEvent semantics #420
- [55] We don't separate connected and signedOn state #530
- [56] Allow custom messages in delta #448
- [57] Should the message kind of custom messages start with `Custom\_` (uppercased)? #476
- [58] Introduce ErrorResponse #405
- [59] Rename Chunked Event / Command / Query Response to "Continued Xxx"? #472
- [60] Can a client change its subscription scope during a connection? #270
- [61] Do we have a "end Participation" / "sign-off" API? #344
- [62] Reconnect request should have clientId and repository id #453
- [63] Reconnect query: Report invalid participation by error or by `participationValid` flag? #354
- [64] ListAndSubscribePartitions definition using undefined term "non-language partitions" #450
- [65] Do we want `listPartitions` query in delta? #361
- [66] AddPartition command should automatically subscribe the sending client to the partition #417
- [67] Deleting a node does not touch existing references to that node #285
- [68] How to mark added node in DeltaSerializationChunk? #357
- [69] Potential issues with delete<child/reference/...> commands #371
- [70] naming: `MoveChildFromOtherContainment` or `MoveChildFromOtherParent`? #513
- [71] naming: `MoveChildFromOtherContainmentInSameParent` or `MoveChildFromOtherContainment`? #514
- [72] MoveAndReplaceChildInSameContainment unclear / incorrect #469
- [73] Delta needs to support composite commands / events #281
- [74] Syntactically Incorrect Composite Command - what to do #454
- [75] Can we nest composite commands / events? #360
- [76] This command is the only one to have a **Command** suffix, which is necessary to distinguish it from its **CompositeEvent** counterpart. All other commands can be distinguished from their event counterparts by looking at whether the technical name reads as an imperative ("add child" — for commands), or in past perfect tense ("child [has been] added" — for events).
- [77] Does a composite command have its own commandId? #353
- [78] ContinuedEvent should have originCommands property #477
- [79] What type and contents should the "deletedChild" property of the ChildDeleted event have #370
- [80] If we add a subtree, do we emit one add per node or only one for the (added) root node of the tree? #326
- [81] If we remove a node via delta protocol, do we also need to send events for all descendants? #286
- [82] Align MoveChildInSameContainment to offsetIndex? #482
- [83] Can `ErrorEvent` / `NoOpEvent` appear inside `CompositeEvent`? #491
- [84] This event is the only one to have a **Event** suffix, which is necessary to distinguish it from its **CompositeCommand** counterpart. All other events can be distinguished from their command counterparts by looking at whether the technical name reads as an imperative ("add child" — for commands), or in past perfect tense ("child [has been] added" — for events). (The **NoOp** and **Error** don't have command counterparts.)
- [85] Does a CompositeEvent have its own sequence number? #351
- [86] How to report a command has completely been discarded, without any resulting event? #314
- [87] Naming: `NoOp` or `NoOpEvent`? #492
- [88] Repository error event #316